



Universidad Peruana de Ciencias Aplicadas

Ingeniería de Software

Periodo: 202610

1ASI0732 | Diseño de Experimentos de Ingeniería de Software

NRC: 16879

Docente: Alex Humberto S  nchez Ponce

Informe del Trabajo Final

Startup: Stoq

Producto: StockWise

Integrantes

U   Luciana Carolina Choquehuanca Nu  ez

U   Ronald Joel Peralta Chipa

U202210973 U   Sanchez Rios, Camila Cristina

U   Fabiola Del Rocio Salda  a Ayala

U  

Abril, 2026

Registro de Versiones del Informe

Version	Fecha	Autor	Descripcion de modificaciones
---------	-------	-------	-------------------------------

Project Report Collaboration Insights

Enlace del repositorio del informe

Enlace de los repositorios de la organizacion

Contenido

Tabla de contenidos

- [Registro de Versiones del Informe](#)
- [Project Report Collaboration Insights](#)
 - [Enlace del repositorio del informe](#)
 - [Enlace de los repositorios de la organizacion](#)
- [Contenido](#)
 - [Tabla de contenidos](#)
- [Student Outcome](#)
- [Part I: As-Is Software Project](#)
- [Capitulo I: Introduccion](#)
 - [1.1. Startup Profile](#)
 - [1.1.1. Descripcion de la Startup](#)
 - [1.1.2. Perfiles de integrantes del equipo](#)
 - [1.2. Solution Profile](#)
 - [1.2.1. Antecedentes y problematica](#)
 - [1.2.2. Lean UX Process](#)
 - [1.2.2.1. Lean UX Problem Statements](#)
 - [1.2.2.2. Lean UX Assumptions](#)
 - [1.2.2.3. Lean UX Hypothesis Statements](#)
 - [1.2.2.4. Lean UX Canvas](#)
 - [1.3. Segmentos objetivo](#)
- [Capitulo II: Requirements Elicitation & Analysis](#)
 - [2.1. Competidores](#)
 - [2.1.1. Analisis competitivo](#)
 - [2.1.2. Estrategias y tacticas frente a competidores](#)
 - [2.2. Entrevistas](#)
 - [2.2.1. Diseno de entrevistas](#)
 - [2.2.2. Registro de entrevistas](#)
 - [2.2.3. Analisis de entrevistas](#)
 - [2.3. Needfinding](#)
 - [2.3.1. User Personas](#)
 - [2.3.2. User Task Matrix](#)
 - [2.3.3. User Journey Mapping](#)
 - [2.3.4. Empathy Mapping](#)
 - [2.3.5. As-is Scenario Mapping](#)
 - [2.4. Ubiquitous Language](#)
- [Capitulo III: Requirements Specification](#)
 - [3.1. To-Be Scenario Mapping](#)
 - [3.2. User Stories](#)
 - [3.3. Product Backlog](#)
 - [3.4. Impact Mapping](#)
- [Capitulo IV: Product Design](#)

- 4.1. Style Guidelines
 - 4.1.1. General Style Guidelines
 - 4.1.2. Web Style Guidelines
 - 4.1.3. Mobile Style Guidelines
 - 4.1.3.1. iOS Mobile Style Guidelines
 - 4.1.3.2. Android Mobile Style Guidelines
- 4.2. Information Architecture
 - 4.2.1. Organization Systems
 - 4.2.2. Labeling Systems
 - 4.2.3. SEO Tags and Meta Tags
 - 4.2.4. Searching Systems
 - 4.2.5. Navigation Systems
- 4.3. Landing Page UI Design
 - 4.3.1. Landing Page Wireframe
 - 4.3.2. Landing Page Mock-up
- 4.4. Mobile Applications UX/UI Design
 - 4.4.1. Mobile Applications Wireframes
 - 4.4.2. Mobile Applications Wireflow Diagrams
 - 4.4.3. Mobile Applications Mock-ups
 - 4.4.4. Mobile Applications User Flow Diagrams
- 4.5. Mobile Applications Prototyping
 - 4.5.1. Android Mobile Applications Prototyping
 - 4.5.2. iOS Mobile Applications Prototyping
- 4.6. Web Applications UX/UI Design
 - 4.6.1. Web Applications Wireframes
 - 4.6.2. Web Applications Wireflow Diagrams
 - 4.6.3. Web Applications Mock-ups
 - 4.6.4. Web Applications User Flow Diagrams
- 4.7. Web Applications Prototyping
- 4.8. Domain-Driven Software Architecture
 - 4.8.1. Software Architecture Context Diagram
 - 4.8.2. Software Architecture Container Diagrams
 - 4.8.3. Software Architecture Components Diagrams
- 4.9. Software Object-Oriented Design
 - 4.9.1. Class Diagrams
 - 4.9.2. Class Dictionary
- 4.10. Database Design
 - 4.10.1. Relational/Non-Relational Database Diagram
- Capitulo V: Product Implementation
 - 5.1. Software Configuration Management
 - 5.1.1. Software Development Environment Configuration
 - 5.1.2. Source Code Management
 - 5.1.3. Source Code Style Guide & Conventions
 - 5.1.4. Software Deployment Configuration
 - 5.2. Product Implementation & Deployment
 - 5.2.1. Sprint Backlogs

- 5.2.2. Implemented Landing Page Evidence
 - 5.2.3. Implemented Frontend-Web Application Evidence
 - 5.2.4. Acuerdo de Servicio - SaaS
 - 5.2.5. Implemented Native-Mobile Application Evidence
 - 5.2.6. Implemented RESTful API and/or Serverless Backend Evidence
 - 5.2.7. RESTful API documentation
 - 5.2.8. Team Collaboration Insights
- 5.3. Video About-the-Product
- Part II: Verification, Validation & Pipeline
- Capitulo VI: Product Verification & Validation
 - 6.1. Testing Suites & Validation
 - 6.1.1. Core Entities Unit Tests
 - 6.1.2. Core Integration Tests
 - Información General Test-1
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - Escenario 4
 - Información General Test-2
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - Información General Test-3
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - Información General Test-4
 - Escenario 1
 - Escenario 2
 - Información General Test-5
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - 6.1.3. Core Behavior-Driven Development
 - Evidence 1: Product Registration Feature File
 - Evidence 2: Low Stock Alert Feature File
 - Evidence 3: BDD Test Execution Results
 - 6.1.4. Core System Tests
 - 6.2. Static testing & Verification
 - 6.2.1. Static Code Analysis
 - 6.2.1.1. .Coding standard & Code conventions
 - 6.2.1.2. Code Quality & Code Security.
 - 6.2.2. Reviews
 - 6.2.2. Reviews
 - Peer Code Reviews
 - Sprint Reviews

- Design & Architecture Reviews
- Quality Assurance Reviews
- Resultado de las revisiones
- 6.3. Validation Interviews.
 - 6.3.1. Diseño de Entrevistas.
 - 6.3.2. Registro de Entrevistas.
 - 6.3.3. Evaluaciones segun heurísticas.
- 6.4. Auditoria de Experiencias de Usuario.
 - 6.4.1. Auditoria realizada.
 - 6.4.1.1. Información del grupo auditado.
 - 6.4.1.2. Cronograma de auditoría realizada
 - 6.4.1.3. Contenido de la auditoría realizada
 - Tareas evaluadas
 - Escala de severidad utilizada
 - Resumen de hallazgos identificados
 - Detalle de hallazgos
 - Hallazgo H-01
 - Hallazgo H-02
 - Hallazgo H-03
 - Hallazgo H-04
 - Hallazgo H-05
 - Hallazgo H-06
 - Hallazgo H-07
 - Hallazgo H-08
 - Hallazgo H-09
 - 6.4.2. Auditoria recibida.
 - 6.4.2.1. Información del grupo auditor.
 - 6.4.2.2. Cronograma de auditoría recibida.
 - 6.4.2.3. Contenido de auditoría recibida.
 - Hallazgo H-01
 - Hallazgo H-02
 - Hallazgo H-03
 - Hallazgo H-04
 - Hallazgo H-05
 - Hallazgo H-06
 - Hallazgo H-07
 - Hallazgo H-08
 - Hallazgo H-09
 - Hallazgo H-10
 - Hallazgo H-11
 - Hallazgo H-12
 - Hallazgo H-13
 - Hallazgo H-14
 - Hallazgo H-15
 - Hallazgo H-16
 - Hallazgo H-17

- Hallazgo H-18
 - Hallazgo H-19
 - 6.4.2.4. Resumen de modificaciones para subsanar hallazgos.
- Capitulo VII: DevOps Practices
 - 7.1. Continuous Integration
 - 7.1.1. Tools and Practices
 - 7.1.2. Build & Test Suite Pipeline Components
 - 7.2. Continuous Delivery
 - 7.2.1. Tools and Practices
 - 7.2.2. Stages Deployment Pipeline Components
 - 7.3. Continuous deployment
 - 7.3.1. Tools and Practices
 - 7.3.2. Production Deployment Pipeline Components
 - 7.4. Continuous Monitoring
 - 7.4.1. Tools and Practices
 - 7.4.2. Monitoring Pipeline Components
 - 7.4.3. Alerting Pipeline Components
 - 7.4.4. Notification Pipeline Components
- Part III: Experiment-Driven Lifecycle
- Capitulo VIII: Experiment-Driven Development
 - 8.1. Experiment Planning
 - 8.1.1. As-Is Summary
 - 8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims
 - 8.1.3. Experiment-Ready Questions
 - 8.1.4. Question Backlog
 - 8.1.5. Experiment Cards
 - 8.2. Experiment Design
 - 8.2.1. Hypotheses
 - 8.2.2. Domain Business Metrics
 - 8.2.3. Measures
 - 8.2.4. Conditions
 - 8.2.5. Scale Calculations and Decisions
 - 8.2.6. Methods Selection
 - 8.2.7. Data Analytics: Goals, KPIs and Metrics Selection
 - 8.2.8. Web and Mobile Tracking Plan
 - 8.3. Experimentation
 - 8.3.1. To-Be User Stories
 - 8.3.2. To-Be Product Backlog
 - 8.3.3. Pipeline-supported, Experiment-Driven To-Be Software Platform Lifecycle
 - 8.3.3.1. To-Be Sprint Backlogs
 - 8.3.3.2. Implemented To-Be Landing Page Evidence
 - 5.2.3. Implemented Frontend-Web Application Evidence
 - 8.3.3.4. Implemented To-Be Native-Mobile Application Evidence
 - 8.3.3.5. Implemented To-Be RESTful API and/or Serverless Backend Evidence
 - 8.3.3.6. Team Collaboration Insights
 - 8.3.4. To-Be Validation Interviews

- 8.3.4.1. Diseno de Entrevistas
 - 8.3.4.2. Registro de Entrevistas
- 8.4. Experiment Aftermath & Analysis
 - 8.4.1. Analysis and Interpretation of Results
 - 8.4.2. Re-scored and Re-prioritized Question Backlog
- 8.5. Continuous Learning
 - 8.5.1. Shareback Session Artifacts: Learning Workflow
- 8.6. To-Be Software Platform Pre-launch
 - 8.6.1. About-the-Product Intro Video
- Conclusiones
- Conclusiones y recomendaciones
- Bibliografia
- Anexos

Student Outcome

ABET – EAC - Student Outcome 4

Criterio: La capacidad de reconocer responsabilidades éticas y profesionales en situaciones de ingeniería y hacer juicios informados, que deben considerar el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales.

En el siguiente cuadro se describe las acciones realizadas y enunciados de conclusiones por parte del grupo, que permiten sustentar el haber alcanzado el logro del ABET – EAC - Student Outcome 4.

Criterio específico	Acciones realizadas	Conclusiones
Reconoce responsabilidad ética y profesional en situaciones de ingeniería de software	Miranda Ayasta, Rogger Faryd	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Vargas Javier, Jose Enrique	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Sanchez Rios, Camila	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Luciana Carolina Choquehuanca Nuñez	
	AV1:	
	TB1:	

AV2:

TB2:

**Ronald Joel Peralta
Chipa**

AV1:

TB1:

AV2:

TB2:

**Emite juicios informados considerando el impacto de las
soluciones de ingeniería de software en contextos
globales, económicos, ambientales y sociales**

**Miranda Ayasta,
Roger Faryd**

AV1:

TB1:

AV2:

TB2:

**Vargas Javier, Jose
Enrique**

AV1:

TB1:

AV2:

TB2:

Sanchez Rios, Camila

AV1:

TB1:

AV2:

TB2:

**Luciana Carolina
Choquehuanca Nuñez**

AV1:

TB1:

AV2:

TB2:

**Ronald Joel Peralta
Chipa**

AV1:

TB1:

AV2:

TB2:

Part I: As-Is Software Project

Capitulo I: Introduccion

1.1. Startup Profile

1.1.1. Descripcion de la Startup

1.1.2. Perfiles de integrantes del equipo

1.2. Solution Profile

1.2.1. Antecedentes y problematica

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statements

1.2.2.2. Lean UX Assumptions

1.2.2.3. Lean UX Hypothesis Statements

1.2.2.4. Lean UX Canvas

1.3. Segmentos objetivo

Capitulo II: Requirements Elicitation & Analysis

2.1. Competidores

2.1.1. Analisis competitivo

2.1.2. Estrategias y tacticas frente a competidores

2.2. Entrevistas

2.2.1. Diseno de entrevistas

2.2.2. Registro de entrevistas

2.2.3. Analisis de entrevistas

2.3. Needfinding

2.3.1. User Personas

2.3.2. User Task Matrix

2.3.3. User Journey Mapping

2.3.4. Empathy Mapping

2.3.5. As-is Scenario Mapping

2.4. Ubiquitous Language

Capitulo III: Requirements Specification

3.1. To-Be Scenario Mapping

3.2. User Stories

3.3. Product Backlog

3.4. Impact Mapping

Capitulo IV: Product Design

4.1. Style Guidelines

4.1.1. General Style Guidelines

4.1.2. Web Style Guidelines

4.1.3. Mobile Style Guidelines

4.1.3.1. iOS Mobile Style Guidelines

4.1.3.2. Android Mobile Style Guidelines

4.2. Information Architecture

4.2.1. Organization Systems

4.2.2. Labeling Systems

4.2.3. SEO Tags and Meta Tags

4.2.4. Searching Systems

4.2.5. Navigation Systems

4.3. Landing Page UI Design

4.3.1. Landing Page Wireframe

4.3.2. Landing Page Mock-up

4.4. Mobile Applications UX/UI Design

4.4.1. Mobile Applications Wireframes

4.4.2. Mobile Applications Wireflow Diagrams

4.4.3. Mobile Applications Mock-ups

4.4.4. Mobile Applications User Flow Diagrams

4.5. Mobile Applications Prototyping

4.5.1. Android Mobile Applications Prototyping

4.5.2. iOS Mobile Applications Prototyping

4.6. Web Applications UX/UI Design

4.6.1. Web Applications Wireframes

4.6.2. Web Applications Wireflow Diagrams

4.6.3. Web Applications Mock-ups

4.6.4. Web Applications User Flow Diagrams

4.7. Web Applications Prototyping

4.8. Domain-Driven Software Architecture

4.8.1. Software Architecture Context Diagram

4.8.2. Software Architecture Container Diagrams

4.8.3. Software Architecture Components Diagrams

4.9. Software Object-Oriented Design

4.9.1. Class Diagrams

4.9.2. Class Dictionary

4.10. Database Design

4.10.1. Relational/Non-Relational Database Diagram

Capitulo V: Product Implementation

5.1. Software Configuration Management

5.1.1. Software Development Environment Configuration

5.1.2. Source Code Management

5.1.3. Source Code Style Guide & Conventions

5.1.4. Software Deployment Configuration

5.2. Product Implementation & Deployment

5.2.1. Sprint Backlogs

5.2.2. Implemented Landing Page Evidence

5.2.3. Implemented Frontend-Web Application Evidence

5.2.4. Acuerdo de Servicio - SaaS

5.2.5. Implemented Native-Mobile Application Evidence

5.2.6. Implemented RESTful API and/or Serverless Backend Evidence

5.2.7. RESTful API documentation

5.2.8. Team Collaboration Insights

5.3. Video About-the-Product

Part II: Verification, Validation & Pipeline

Capítulo VI: Product Verification & Validation

6.1. Testing Suites & Validation

6.1.1. Core Entities Unit Tests

Se muestra evidencia de los test a las historias de usuario del proyecto

US01 - Crear producto válido: valida que el agregado Product se construye correctamente con datos completos y normaliza los valores de texto.

```
@Test
// US01 - Registrar producto nuevo: creación válida.
void shouldCreateProductWithValidData() {
    CreateProductCommand command = new CreateProductCommand(
        name: " Arroz Extra ",
        description: " Bolsa de 5kg ",
        categoryId: "1",
        providerId: "10",
        minStock: 8,
        unitPrice: 12.5,
        isActive: true
    );

    Product product = new Product(command);

    assertNotNull(product);
    assertEquals(expected: "Arroz Extra", product.getName());
    assertEquals(expected: "Bolsa de 5kg", product.getDescription());
    assertEquals(expected: "1", product.getCategoryId());
    assertEquals(expected: "10", product.getProviderId());
    assertEquals(expected: 8, product.getMinStock());
    assertEquals(expected: 12.5, product.getUnitPrice());
    assertEquals(expected: true, product.getIsActive());
}
```

US01 / US19 - Stock inicial y mínimo: valida que minStock acepta valores válidos, incluyendo cero, y rechaza negativos.

```
@Test
// US01 + US19 - stock inicial/minStock válido en 0.
void shouldAllowZeroMinStockAsInitialStockThreshold() {
    CreateProductCommand command = new CreateProductCommand(
        name: "Aceite",
        description: "Botella 1l",
        categoryId: "2",
        providerId: "11",
        minStock: 0,
        unitPrice: 7.2,
        isActive: true
    );

    Product product = new Product(command);
    assertEquals(expected: 0, product.getMinStock());
}
```

US19 - Configurar stock mínimo: valida que el producto permita actualizar el límite mínimo de stock correctamente.

```
@Test
// US19 - Configurar stock mínimo: guardar nuevo límite.
void shouldUpdateMinStockWhenCommandIsValid() {
    Product product = new Product(new CreateProductCommand(
        name: "Fideos",
        description: "Paquete",
        categoryId: "3",
        providerId: "12",
        minStock: 4,
        unitPrice: 3.5,
        isActive: true
    ));

    UpdateProductCommand update = new UpdateProductCommand(
        productId: 1L,
        name: "Fideos Premium",
        description: "Paquete grande",
        categoryId: "3",
        providerId: "12",
        minStock: 9,
        unitPrice: 4.0,
        isActive: true
    );

    product.updateProduct(update);

    assertEquals(expected: "Fideos Premium", product.getName());
    assertEquals(expected: 9, product.getMinStock());
    assertEquals(expected: 4.0, product.getUnitPrice());
}
```

US03 - Registrar salida de producto: valida que Batch descuenta stock correctamente al registrar una salida.

```
@Test
// US03 - Registrar salida de producto: estado base para operar stock.
void shouldCreateBatchWithValidData() {
    Date now = new Date();
    Date expiration = new Date(now.getTime() + 86_400_000L);

    Batch batch = new Batch(new CreateBatchCommand(productId: 1L, quantity: 20, expiration, now));

    assertEquals(expected: 1L, batch.getProductId());
    assertEquals(expected: 20, batch.getQuantity());
}

@Test
// US03 - Registrar salida de producto: descuento correcto del stock.
void shouldDecreaseStockForProductOutput() {
    Date now = new Date();
    Date expiration = new Date(now.getTime() + 86_400_000L);
    Batch batch = new Batch(new CreateBatchCommand(productId: 5L, quantity: 15, expiration, now));

    batch.reduceQuantity(amount: 6);

    assertEquals(expected: 9, batch.getQuantity());
}
```

US14 - Registrar devolución: valida que el stock aumente al registrar una devolución.

```
@Test
// US14 - Registrar devolución: aumentar stock.
void shouldIncreaseStockWhenRegisteringReturn() {
    Date now = new Date();
    Date expiration = new Date(now.getTime() + 86_400_000L);
    Batch batch = new Batch(new CreateBatchCommand(productId: 7L, quantity: 10, expiration, now));

    int returnedQuantity = 3;
    batch.setQuantity(batch.getQuantity() + returnedQuantity);

    assertEquals(expected: 13, batch.getQuantity());
}
```

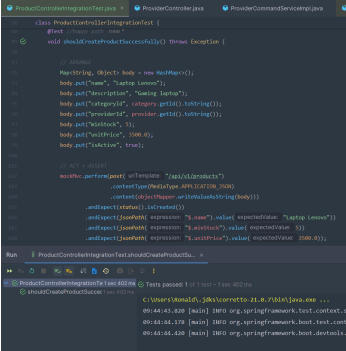
6.1.2. Core Integration Tests

En esta sección se presentan las pruebas de integración desarrolladas para validar la correcta comunicación e interacción entre los componentes principales de StockWise, garantizando el funcionamiento coordinado y consistente de los distintos módulos que conforman el sistema.

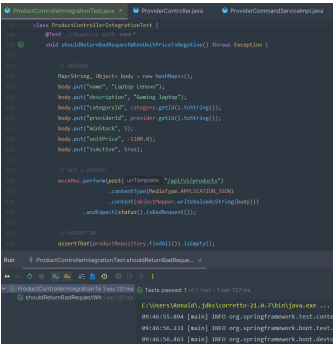
Información General Test-1

Elemento	Descripción
Clase de Test	ProductControllerIntegrationTest
Módulo(s)	inventory, shared, iam - US12, US13
Descripción General	Validar la integración completa del módulo de productos dentro del sistema StockWise, comprobando que las operaciones relacionadas con la gestión de productos funcionen correctamente desde la capa HTTP hasta la persistencia en base de datos.

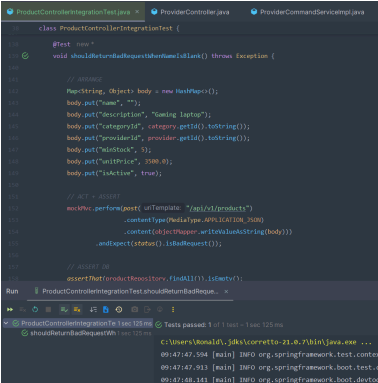
Escenario 1

Evidencia	Descripción
	Este escenario valida que el sistema pueda registrar correctamente un nuevo producto cuando se envían datos válidos, verificando el flujo completo desde la solicitud HTTP hasta la persistencia en base de datos. Se comprueba la integración entre controller, servicios de aplicación, aggregate Product, repositorios JPA y la base de datos H2. Esta validación es importante porque asegura que la funcionalidad principal de creación de productos opere correctamente y que la información registrada mantenga consistencia dentro del inventario.

Escenario 2

Evidencia	Descripción
	Este escenario valida que el sistema rechace productos con un precio unitario inválido, específicamente valores negativos, comprobando que las reglas de negocio definidas en el dominio sean respetadas antes de persistir información. La prueba garantiza que las validaciones del aggregate y el manejo de excepciones funcionen correctamente a través de toda la integración del sistema. Esto es importante para proteger la integridad de los datos financieros y evitar inconsistencias en cálculos de ventas, reportes y valorización de inventario.

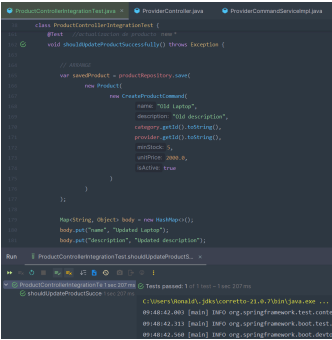
Escenario 3

Evidencia	Descripción
	Este escenario verifica que el sistema no permita registrar productos sin nombre o con nombres vacíos, validando las restricciones obligatorias del dominio y el correcto manejo de respuestas HTTP ante entradas inválidas. La prueba asegura que los datos esenciales del catálogo sean obligatorios y que el sistema prevenga registros incompletos que puedan afectar búsquedas, clasificación de productos y operaciones posteriores dentro del inventario.

Escenario 4

Evidencia

Descripción



Este escenario valida que el sistema pueda actualizar correctamente la información de un producto existente, verificando la integración entre el endpoint REST, los servicios de actualización, el aggregate Product, el repositorio JPA y la persistencia en base de datos. Se comprueba que los cambios enviados sean reflejados correctamente y que las validaciones del dominio continúen aplicándose durante la edición. Esta validación es importante porque garantiza que la información del catálogo pueda mantenerse actualizada y consistente a lo largo del tiempo.

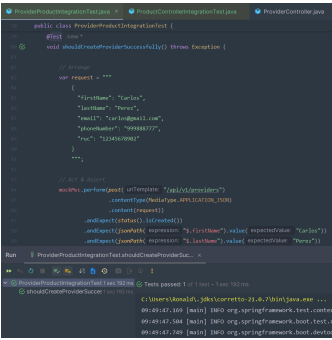
Información General Test-2

Elemento	Descripción
Clase de Test	ProviderProductIntegrationTest
Módulo(s)	inventory, shared - US24, US26
Descripción General	Validar la integración entre los módulos de proveedores y productos, asegurando que las relaciones funcionales entre ambos aggregates se comporten correctamente dentro del sistema.

Escenario 1

Evidencia

Descripción

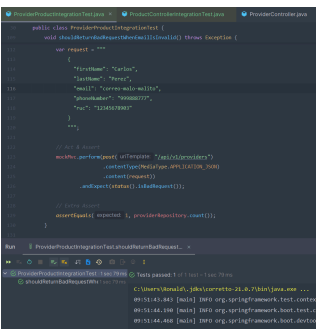


Este escenario valida que el sistema pueda registrar correctamente un nuevo proveedor utilizando datos válidos, comprobando el flujo completo desde la solicitud HTTP hasta la persistencia en base de datos. La prueba verifica la integración entre controller, servicios de aplicación, aggregate Provider, value objects, repositorios JPA y la base de datos H2. Esta validación es importante porque garantiza que los proveedores puedan ser gestionados correctamente y que la información crítica de abastecimiento quede almacenada de manera consistente y segura.

Escenario 2

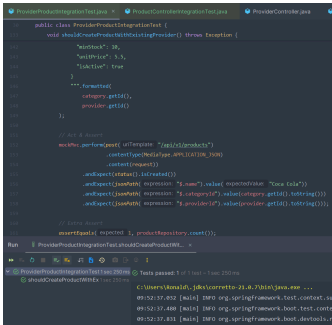
Evidencia

Descripción



Este escenario valida que el sistema rechace proveedores con correos electrónicos inválidos, comprobando que las reglas de validación implementadas en los value objects del dominio funcionen correctamente durante todo el flujo de integración. La prueba asegura que las excepciones generadas por datos inconsistentes sean manejadas adecuadamente y que la información inválida no llegue a persistirse en la base de datos. Esta validación es importante porque protege la calidad de los datos y evita problemas posteriores relacionados con notificaciones, contacto con proveedores y trazabilidad del sistema.

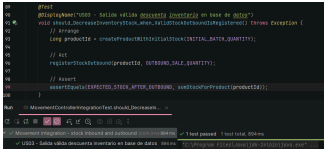
Escenario 3

Evidencia	Descripción
	Este escenario valida la integración funcional entre los módulos de productos y proveedores, verificando que un producto solo pueda registrarse cuando se encuentra asociado a un proveedor existente dentro del sistema. La prueba comprueba la interacción entre múltiples aggregates, servicios de aplicación, repositorios y validaciones de negocio relacionadas con integridad referencial. Esta validación es importante porque asegura consistencia entre entidades del inventario y evita relaciones inválidas que puedan afectar procesos de compras, abastecimiento y trazabilidad de productos.

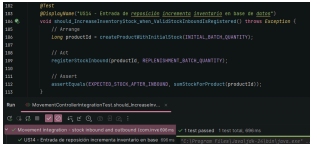
Información General Test-3

Elemento	Descripción
Clase de Test	MovementControllerIntegrationTest
Módulo(s)	inventory, sales - US03, US14
Descripción General	Validar la integración entre los módulos de inventario y ventas, comprobando que los movimientos de entrada y salida de stock actualicen correctamente la cantidad disponible en base de datos, y que el sistema rechace operaciones que excedan el stock disponible.

Escenario 1

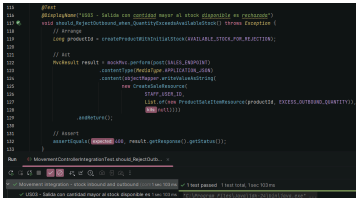
Evidencia	Descripción
	Este escenario valida que el sistema descuente correctamente el stock de un producto cuando se registra una venta válida, verificando el flujo completo desde la solicitud HTTP hasta la persistencia en base de datos H2. Se comprueba la integración entre el controlador de ventas, los servicios de aplicación, el repositorio de lotes y la lógica de descuento del aggregate. Esta validación es importante porque garantiza que cada transacción de salida mantenga la consistencia del inventario y refleje el stock real disponible para operaciones posteriores.

Escenario 2

Evidencia	Descripción
	Este escenario valida que el sistema incremente correctamente el stock de un producto cuando se registra un lote de reposición, comprobando la integración entre el endpoint de lotes, los servicios de aplicación, el repositorio JPA y la base de datos H2. La prueba verifica que la cantidad del nuevo lote se sume correctamente al stock existente del producto. Esta validación es importante porque asegura que el proceso de reabastecimiento opere de

Evidencia	Descripción
	forma confiable y que el inventario refleje en todo momento las entradas registradas por el equipo de almacén.

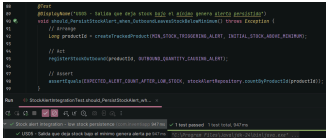
Escenario 3

Evidencia	Descripción
	Este escenario verifica que el sistema rechace una solicitud de venta cuya cantidad supera el stock disponible del producto, comprobando que las reglas de negocio del dominio sean aplicadas correctamente antes de persistir cualquier cambio. La prueba garantiza que el sistema retorne un error HTTP 400 ante este tipo de operación inválida. Esta validación es importante porque protege la integridad del inventario y evita que el sistema registre ventas que no pueden ser satisfechas con el stock existente.

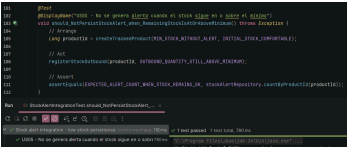
Información General Test-4

Elemento	Descripción
Clase de Test	StockAlertIntegrationTest
Módulo(s)	inventory, alertstock - US05
Descripción General	Validar la integración entre los módulos de inventario y alertas de stock, comprobando que el sistema genere y persista automáticamente una alerta cuando una salida de stock deja el nivel por debajo del mínimo configurado, y que no genere alertas innecesarias cuando el stock se mantiene dentro del rango aceptable.

Escenario 1

Evidencia	Descripción
	Este escenario valida que el sistema genere y persista automáticamente una alerta de stock bajo cuando una venta reduce el inventario por debajo del mínimo configurado para el producto, comprobando la integración entre el módulo de ventas, el servicio de alertas y el repositorio de alertas en base de datos H2. Esta validación es importante porque garantiza que el mecanismo de alerta temprana funcione de forma automática y confiable, permitiendo al equipo de almacén reaccionar oportunamente ante situaciones de desabastecimiento.

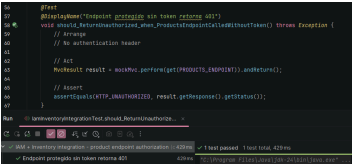
Escenario 2

Evidencia	Descripción
	Este escenario verifica que el sistema no genere alertas de stock cuando una salida de inventario deja el nivel igual o por encima del mínimo configurado, comprobando que la lógica de disparo de alertas sea precisa y no produzca falsos positivos. La prueba consulta directamente el repositorio de alertas para confirmar que ningún registro fue persistido. Esta validación es importante porque evita que el equipo de almacén reciba notificaciones innecesarias que puedan generar ruido operativo y reducir la efectividad del sistema de alertas.

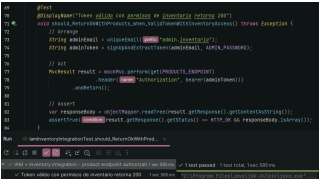
Información General Test-5

Elemento	Descripción
Clase de Test	IamInventoryIntegrationTest
Módulo(s)	iam, inventory - US01, US12
Descripción General	Validar la integración entre el módulo de autenticación y autorización (IAM) y los endpoints de inventario, comprobando que el sistema proteja correctamente el acceso a los recursos según el token y los permisos del usuario autenticado.

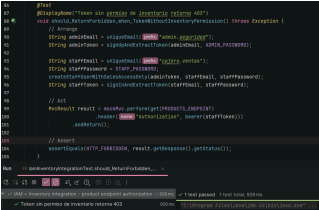
Escenario 1

Evidencia	Descripción
	Este escenario valida que el sistema rechace con HTTP 401 cualquier solicitud al endpoint de productos que no incluya un token de autenticación, comprobando que los filtros de seguridad estén activos y configurados correctamente. Esta validación es importante porque garantiza que los recursos del inventario estén protegidos frente a accesos anónimos y que el sistema cumpla con los requisitos de seguridad definidos para la plataforma SaaS.

Escenario 2

Evidencia	Descripción
	Este escenario valida que un usuario autenticado con permisos de inventario pueda acceder correctamente al endpoint de productos, recibiendo una respuesta HTTP 200 con el listado correspondiente. La prueba comprueba la integración completa entre el proceso de registro, la generación del token JWT y la autorización en el endpoint protegido. Esta validación es importante porque asegura que el flujo de autenticación funcione de extremo a extremo y que los usuarios con los permisos correctos puedan operar sin restricciones.

Escenario 3

Evidencia	Descripción
	Este escenario verifica que un usuario autenticado pero sin permisos de inventario reciba una respuesta HTTP 403 al intentar acceder al endpoint de productos, comprobando que el sistema aplique correctamente las reglas de autorización por rol. La prueba crea un usuario con acceso exclusivo a ventas e intenta acceder a un recurso restringido. Esta validación es importante porque garantiza el principio de mínimo privilegio en la plataforma, protegiendo los datos del inventario frente a accesos no autorizados de usuarios con roles insuficientes.

6.1.3. Core Behavior-Driven Development

En esta sección se presentan las pruebas Behavior-Driven Development desarrolladas para validar el comportamiento esperado de las funcionalidades principales de StockWise desde la perspectiva del usuario. El desarrollo de estas pruebas se basó en las User Stories priorizadas del Product Backlog, específicamente US01 - Registrar producto nuevo y US05 - Generar alertas por bajo stock, ambas pertenecientes al epic EP01 - Funciones básicas de inventario. Estas historias fueron seleccionadas porque representan funciones centrales del sistema de inventario.

Evidence 1: Product Registration Feature File

```
Feature: Registro de productos

    Para mantener un control actualizado y confiable de las existencias desde su ingreso
    Como Usuario de inventario
    Quiero registrar un nuevo producto en mi inventario

Scenario: Registro exitoso de producto
    Given el usuario proporciona información válida del producto
    When el usuario confirma el registro
    Then el sistema valida la información
    And registra los datos de manera segura
    And actualiza el inventario con el nuevo producto

Scenario: Información incompleta o inconsistente
    Given existe información de producto incompleta o inconsistente
    When el sistema procesa la solicitud de registro
    Then rechaza la operación
    And genera una notificación indicando la necesidad de completar la información requerida

Scenario: Producto duplicado
    Given existe una solicitud de registro con un producto ya registrado
    When el sistema procesa la información recibida
    Then impide el registro duplicado
    And genera una notificación indicando que el producto ya se encuentra en el inventario
```

Evidence 2: Low Stock Alert Feature File

```
Feature: Alertas por bajo stock

Para realizar el reabastecimiento oportuno y evitar quiebres de inventario
Como Usuario de inventario
Quiero recibir alertas cuando un producto se encuentra por debajo del stock mínimo

Scenario: Activación automática de alerta por bajo stock
  Given un producto tiene configurado un límite mínimo de stock
  When el sistema detecta que la cantidad disponible desciende por debajo de ese valor
  Then el sistema genera una alerta automática
  And comunica al usuario la necesidad de reabastecer el producto

Scenario: Personalización del límite de alerta
  Given el usuario desea modificar el valor mínimo de stock
  When el usuario establece un nuevo parámetro de alerta
  Then el sistema guarda el nuevo valor
  And aplica las alertas futuras con base en el límite configurado

Scenario: Desactivación temporal de alertas
  Given el usuario necesita suspender las notificaciones por mantenimiento o revisión de inventario
  When el usuario desactiva temporalmente las alertas
  Then el sistema detiene la emisión de nuevas alertas
  And registra la acción para su trazabilidad
```

Evidence 3: BDD Test Execution Results

```
Project ▾ pom.xml (stocktrack-backend) product-registration.feature low-stock-alert.feature CucumberTest.java x ProductRegistrationSteps.java
Run CucumberTest x
CucumberTest (com.inventiapp.stocktrack: 144ms) ✓ 6 tests passed 6 tests total, 144ms
  ✓ Alertas por bajo stock 117ms
    ✓ Activación automática de alerta por bajo stock 96ms
    ✓ Personalización del límite de alerta 11ms
    ✓ Desactivación temporal de alertas 10ms
  ✓ Registro de productos 27ms
    ✓ Registro exitoso de producto 9ms
    ✓ Información incompleta o inconsistente 8ms
    ✓ Producto duplicado 10ms

Scenario: Activación automática de alerta por bajo stock # features/low-stock-alert.feature:7
  Given un producto tiene configurado un límite mínimo de stock # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.unProductoTieneConfiguradoUnLi
  When el sistema detecta que la cantidad disponible desciende por debajo de ese valor # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaDetectaCantidadDispon
  Then el sistema genera una alerta automática # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaGeneraUnaAlertaAutoma
  And comunica al usuario la necesidad de reabastecer el producto # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.comunicaAlUsuarioLaNecesidadDe

Scenario: Personalización del límite de alerta # features/low-stock-alert.feature:13
  Given el usuario desea modificar el valor mínimo de stock # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioDeseaModificarElValorMinimoDeStock()
  When el usuario establece un nuevo parámetro de alerta # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioEstableceUnNuevoParametroDeAlerta()
  Then el sistema guarda el nuevo valor # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaGuardaElNuevoValor()
  And aplica las alertas futuras con base en el límite configurado # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.aplicaLasAlertasFuturasConBaseEnElLimiteConfigurad

Scenario: Desactivación temporal de alertas # features/low-stock-alert.feature:19
  Given el usuario necesita suspender las notificaciones por mantenimiento o revisión de inventario # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioNecesita
  When el usuario desactiva temporalmente las alertas # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioDesactiv
  Then el sistema detiene la emisión de nuevas alertas # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaDetieneE
  And registra la acción para su trazabilidad # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.registraLaAccionP

Scenario: Registro exitoso de producto # features/product-registration.feature:7
  Given el usuario proporciona información válida del producto # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioProporcionaInformacionValidaDelProducto()
  When el usuario confirma el registro # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioConfirmaElRegistro()
  Then el sistema valida la información # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaValidaLaInformacion()
  And registra los datos de manera segura # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.registraLosDatosDeManeraSegura()
  And actualiza el inventario con el nuevo producto # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.actualizaElInventarioConELNuevoProducto()

Scenario: Información incompleta o inconsistente # features/product-registration.feature:14
  Given existe información de producto incompleta o inconsistente # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.existeInformacionDeProduct
  When el sistema procesa la solicitud de registro # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaProcesaLaSolicitu
  Then rechaza la operación # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.rechazaLaOperacion()
  And genera una notificación indicando la necesidad de completar la información requerida # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.generaNotificacionIndicand

Scenario: Producto duplicado # features/product-registration.feature:20
  Given existe una solicitud de registro con un producto ya registrado # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.existeUnaSolicitudDeRegistru
  When el sistema procesa la información recibida # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaProcesaLaInformacio
  Then impide el registro duplicado # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.impideElRegistroDuplicado()
  And genera una notificación indicando que el producto ya se encuentra en el inventario # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.generaNotificacionProductoYa
```

6.1.4. Core System Tests

Feature: Confirmar salida y descontar inventario

Para mantener la exactitud del stock y asegurar el descuento correcto de los lotes

Como Cajero

Quiero confirmar una salida de productos en el sistema

Scenario: Venta básica con consumo secuencial de lotes

Given el sistema cuenta con un producto activo con stock distribuido en múltiples lotes

When el usuario procesa una venta por una cantidad que supera el primer lote pero es cubierta por el segundo

Then el sistema valida y procesa la solicitud exitosamente

And agota por completo el inventario del primer lote

And descuenta la cantidad restante del segundo lote

And registra la venta de manera segura

```
@Test
@DisplayName("Debe completar flujo de venta basico")
void testSimpleSalesFlow() throws Exception {

    categoryId = extractIdFromResponse(postRequest( endpoint: "/api/v1/categories",
        new CreateCategoryResource( name: "Electronics", description: "Electronics products")));

    providerId = extractIdFromResponse(postRequest( endpoint: "/api/v1/providers",
        new CreateProviderResource( firstName: "Miguel", lastName: "Suarez", phoneNumber: "1234567890", email: "tech@email.com", ruc: "20123456789")));

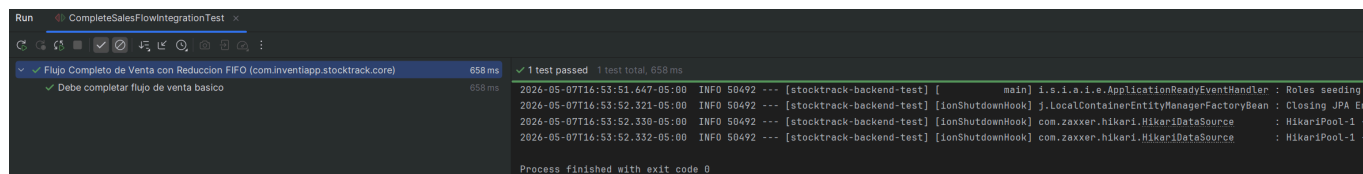
    productId = extractIdFromResponse(postRequest( endpoint: "/api/v1/products",
        new CreateProductResource( name: "USB Keyboard", description: "Mechanical RGB Keyboard",
            String.valueOf(categoryId), String.valueOf(providerId), minStock: 5, unitPrice: 29.99, isActive: true)));

    Date now = new Date();
    Long batch1Id = extractIdFromResponse(postRequest( endpoint: "/api/v1/batches",
        new CreateBatchResource(productId, quantity: 50, addDays(25), now)));
    Long batch2Id = extractIdFromResponse(postRequest( endpoint: "/api/v1/batches",
        new CreateBatchResource(productId, quantity: 30, addDays(40), now)));

    postRequest( endpoint: "/api/v1/sales", new CreateSaleResource( staffUserId: 1L, List.of(new ProductSaleItemResource(productId, quantity: 60)), kits: null));

    // Se valida solo el resultado clave del flujo
    getAndValidate( endpoint: "/api/v1/batches/" + batch1Id, expectedQuantity: 0);
    getAndValidate( endpoint: "/api/v1/batches/" + batch2Id, expectedQuantity: 20);

    mockMvc.perform(get( uriTemplate: "/api/v1/sales"))
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$", hasSize(greaterThanOrEqualTo( value: 1))));
}
```



6.2. Static testing & Verification

Esta sección describe las actividades de control de calidad que se ejecutan directamente sobre el código fuente, especificaciones y artefactos del proyecto sin necesidad de poner el sistema en marcha. Su objetivo es identificar de manera temprana ambigüedades, defectos de diseño, vulnerabilidades de seguridad y desviaciones de las pautas de desarrollo establecidas.

6.2.1. Static Code Analysis

Consiste en la evaluación automatizada del código base utilizando herramientas especializadas. A través de este análisis se audita la mantenibilidad, el cumplimiento de reglas sintácticas, la deuda técnica y los posibles riesgos de seguridad antes de que el código sea integrado a las ramas principales del repositorio.

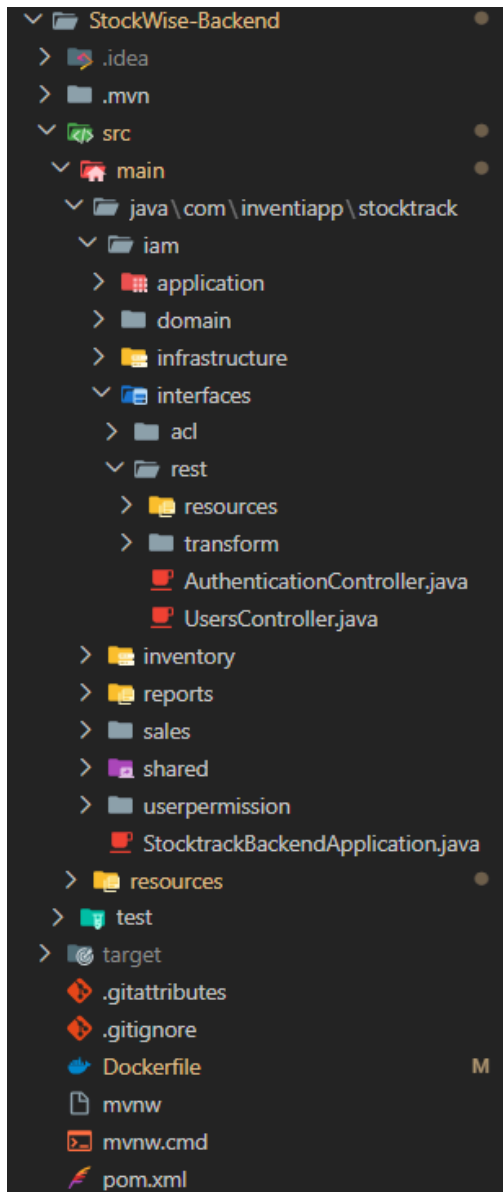
6.2.1.1. Coding standard & Code conventions

Backend - Java/Spring Boot

El backend implementa convenciones de código rigurosas basadas en los estándares oficiales de Spring Framework y las mejores prácticas de la arquitectura limpia. La nomenclatura sigue patrones semánticos y descriptivos: los controladores utilizan el sufijo Controller, los servicios emplean Service o la distinción CommandService / QueryService para segregar las operaciones de lectura y escritura (patrón CQRS), y los componentes de transformación adoptan el sufijo Assembler para la conversión eficiente entre entidades del dominio y recursos REST (DTOs).

La estructura de paquetes refleja fielmente una arquitectura hexagonal, donde cada contexto acotado (Bounded Context) está claramente delimitado en los paquetes domain, application, interfaces.rest e infrastructure, facilitando una estricta separación de responsabilidades. Adicionalmente, todos los métodos públicos incluyen documentación técnica mediante Javadoc, detallando parámetros y valores de retorno. El manejo de excepciones es homogéneo mediante una capa global de captura de errores, la cual procesa excepciones de negocio específicas (tales como ProductAlreadyExistsException o ProductNotFoundException) y las traduce en códigos de estado HTTP semánticos (201, 400, 404, 500).

Las anotaciones nativas de Spring (@RestController, @PostMapping, @GetMapping) y de documentación (@Operation, @ApiResponse) se aplican de forma transversal para generar de manera automática el contrato de la API mediante OpenAPI/Swagger. Finalmente, la inyección de dependencias se realiza exclusivamente a través de constructores, promoviendo la inmutabilidad de los componentes y simplificando el diseño de pruebas unitarias.



```

@RestController
@RequestMapping(value = "/api/v1/authentication", produces = MediaType.APPLICATION_JSON_VALUE)
@Tag(name = "Authentication", description = "Authentication Endpoints")
public class AuthenticationController {

    private final UserCommandService userCommandService;
    private final TokenService tokenService;

    /**
     * Constructor
     * @param userCommandService The {@link UserCommandService} instance
     * @param tokenService The {@link TokenService} instance
     */
    public AuthenticationController(UserCommandService userCommandService, TokenService tokenService) {
        this.userCommandService = userCommandService;
        this.tokenService = tokenService;
    }

    /**
     * Sign in endpoint
     * @param signInResource The {@link SignInResource} instance
     * @return A {@link AuthenticatedUserResource} resource for the authenticated user with JWT token, or an unauthorized response if credentials are invalid
     */
    @PostMapping("/sign-in")
    @Operation(summary = "Sign in", description = "Authenticate a user and return a JWT token")
    @ApiResponse(value = {
        @ApiResponse(responseCode = "200", description = "User authenticated successfully"),
        @ApiResponse(responseCode = "401", description = "Invalid credentials")
    })
    public ResponseEntity<AuthenticatedUserResource> signIn(@RequestBody SignInResource signInResource) {
        var signInCommand = SignInCommandFromResourceAssembler.toCommandFromResource(signInResource);
        var result = userCommandService.handle(signInCommand);

        if (result.isEmpty()) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
        }

        var user = result.get().getLeft();
        var token = result.get().getRight();
        var authenticatedUserResource = AuthenticatedUserResourceFromEntityAssembler.toResourceFromEntity(user, token);

        return ResponseEntity.ok(authenticatedUserResource);
    }
}

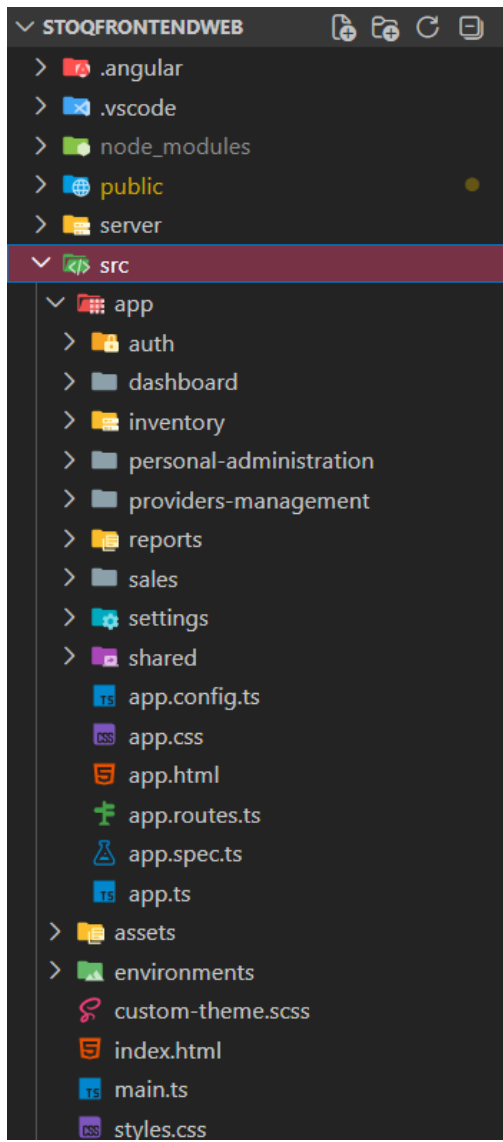
```

Frontend - Angular/TypeScript

El frontend se alinea con las convenciones oficiales recomendadas por el equipo de Angular y las directrices de TypeScript, estructurándose mediante una arquitectura modular orientada a características (Features). Cada módulo de negocio (auth, inventory, dashboard, etc.) replica la filosofía de separación de responsabilidades en las capas de presentation, domain, infrastructure y application.

Los archivos se nombran estrictamente utilizando la convención kebab-case (auth-api.ts, auth-guard.ts, inventory.store.ts), mientras que las clases e interfaces emplean PascalCase (AuthApi, InventoryStore, AuthGuard). Los servicios inyectables se configuran mediante el decorador @Injectable({ providedIn: 'root' }), lo que optimiza el empaquetado mediante tree-shaking y previene la duplicidad de instancias. Se destaca el uso de Angular Signals para la gestión del estado reactivo de la interfaz, adoptando así la API moderna del framework para reducir la complejidad operativa frente a patrones RxJS tradicionales.

La documentación mediante JSDoc es mandatoria en métodos críticos, especialmente en guards y servicios de dominio. Los guards de protección de rutas implementan la interfaz funcional CanActivateFn, validando la vigencia del token JWT, los estados de autenticación y los roles del usuario antes de permitir el acceso a las vistas. El código prohíbe la lógica compleja dentro de los templates HTML, delegando dicha responsabilidad a los componentes y almacenes (stores) fuertemente tipados.



```

export interface StockInfo {
  productId: string;
  currentStock: number;
  lastUpdated: string;
}

> /**...
@Injectable({
  providedIn: 'root'
})
export class InventoryStore {
  private readonly productsSignal = signal<Product[]>([]);
  private readonly categoriesSignal = signal<Category[]>([]);
  private readonly providersSignal = signal<Provider[]>([]);
  private readonly kitsSignal = signal<Kit[]>([]);
  private readonly batchesSignal = signal<Batch[]>([]);
  private readonly loadingSignal = signal<boolean>(false);
  private readonly errorSignal = signal<string | null>(null);

  readonly products = this.productsSignal.asReadonly();
  readonly categories = this.categoriesSignal.asReadonly();
  readonly providers = this.providersSignal.asReadonly();
  readonly kits = this.kitsSignal.asReadonly();
  readonly batches = this.batchesSignal.asReadonly();
  readonly loading = this.loadingSignal.asReadonly();
  readonly error = this.errorSignal.asReadonly();

  /**
   * Computed stock from batches - calculates total quantity per product.
   */
  readonly stock = computed<StockInfo[]>(() => {
    const batches = this.batches();
    const stockMap = new Map<string, { quantity: number; lastDate: string }>();

    batches.forEach(batch => {
      const productId = String(batch.productId);
      const existing = stockMap.get(productId);
      const batchDate = batch.receptionDate || new Date().toISOString();

      if (existing) {
        existing.quantity += batch.quantity;
        if (batchDate > existing.lastDate) {
          existing.lastDate = batchDate;
        }
      }
    });
  });
}

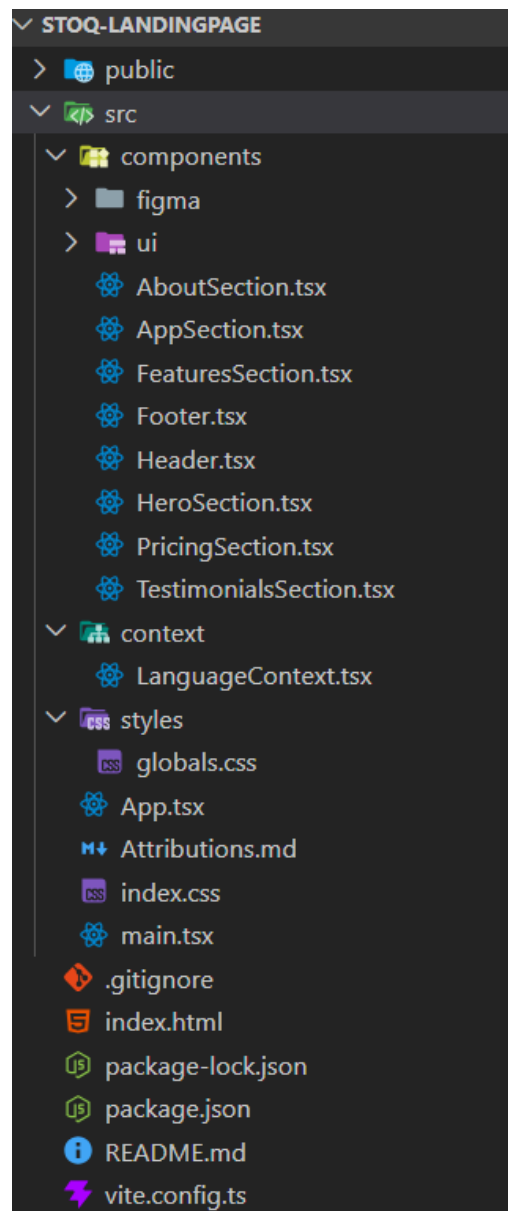
```

Landing Page - React/TypeScript

La landing page se construye mediante componentes funcionales de React basados en TypeScript, asegurando la tipación estática (type safety) en toda la capa de presentación. Los componentes se nombran en PascalCase (Header, HeroSection, PricingSection) y se centralizan dentro del directorio src/components/. Cada componente declara y recibe propiedades (Props) fuertemente tipadas, mitigando errores comunes en tiempo de compilación.

Para el diseño visual se utiliza Tailwind CSS, aplicando clases utilitarias directamente sobre el atributo className, lo que resulta en un estilo cohesivo, ágil y libre de archivos CSS externos innecesarios. Los colores corporativos de la marca se encuentran centralizados en el archivo de configuración de Tailwind, evitando valores arbitrarios dispersos.

La internacionalización del sitio se gestiona a través de un `LanguageContext` que expone el hook personalizado `useLanguage()`, permitiendo a los componentes acceder y alternar el idioma de manera global sin incurrir en prop drilling. El código mantiene un acceso unificado a las variables de entorno y URLs de redirección mediante constantes globales. Asimismo, componentes clave como `ImageWithFallback` implementan patrones defensivos para mitigar fallos en la carga de recursos multimedia externos, garantizando una experiencia de usuario fluida.



6.2.1.2. Code Quality & Code Security.

Backend - Java/Spring Boot

La calidad en el backend se asegura mediante el desacoplamiento de la arquitectura hexagonal y el uso de DTOs, lo que reduce la complejidad ciclomática y la deuda técnica. En el ámbito de la seguridad a nivel de código, el sistema destaca por:

- Control de Acceso: Integración de Spring Security y tokens JWT. Los endpoints se protegen mediante autorización basada en roles con la anotación `@PreAuthorize`.
- Validación Defensiva: Aplicación estricta de Bean Validation (`@Valid`, `@NotNull`) en controladores para sanitizar los datos de entrada y prevenir ataques de inyección (SQLi) o XSS.

- Transparencia: Todos los flujos de autorización y códigos de respuesta están completamente documentados y declarados en el contrato de OpenAPI/Swagger.

Frontend - Angular/TypeScript

La calidad del frontend se basa en una arquitectura modular por features y en el uso estricto de TypeScript (strict: true), garantizando un tipado fuerte que elimina errores en tiempo de compilación. Las prácticas de seguridad y estabilidad aplicadas incluyen:

- Protección de Navegación: Implementación de guards funcionales (CanActivateFn) que interceptan las rutas y validan la vigencia del JWT y los permisos antes de renderizar cualquier vista.
- Gestión de Estado Inmutable: El uso de Angular Signals previene fugas de memoria (memory leaks) y asegura un flujo de datos reactivo y predecible.
- Saneamiento: Confianza en los mecanismos nativos del framework para sanitizar el DOM y neutralizar vulnerabilidades de Client-Side XSS.

Landing Page - React/TypeScript

A pesar de su naturaleza informativa, la landing page se rige por estándares de calidad basados en componentes funcionales reutilizables y tipados estáticos que evitan caídas de la interfaz. Sus medidas de seguridad y buenas prácticas comprenden:

- Navegación Segura: Uso mandatorio del atributo rel="noopener noreferrer" en enlaces externos para mitigar ataques de manipulación de pestañas (Tabnabbing).
- Gestión de Contexto: Centralización de traducciones mediante LanguageContext fuertemente tipado, evitando referencias nulas en la interfaz.
- Aislamiento: Configuración hermética de variables de entorno para evitar la filtración accidental de URLs o credenciales de desarrollo en el build de producción.

6.2.2. Reviews

6.2.2. Reviews

Durante el desarrollo del sistema **NorthEye**, se realizaron revisiones continuas del producto con el objetivo de asegurar la calidad del código, validar el cumplimiento de los requerimientos funcionales y no funcionales, y detectar de manera temprana posibles errores de diseño, implementación o integración.

Estas revisiones se ejecutaron en distintos niveles del ciclo de desarrollo, abarcando tanto el código fuente como los artefactos de diseño y la integración de componentes del sistema (backend, frontend y landing page).

Peer Code Reviews

Cada funcionalidad implementada en el backend (FastAPI) y en el frontend (React/Angular según módulo) pasó por un proceso de revisión entre pares antes de ser integrada a la rama principal del repositorio.

Durante estas revisiones se evaluaron los siguientes aspectos:

- Cumplimiento de las convenciones de codificación establecidas en el proyecto.
- Correcta implementación de la lógica de negocio en servicios y controladores.
- Uso adecuado de DTOs y separación de responsabilidades.
- Legibilidad, mantenibilidad y estructura del código.

- Detección de posibles errores lógicos o redundancias.
- Validación del manejo de excepciones y respuestas HTTP consistentes.

Asimismo, se verificó que los cambios realizados no generen conflictos con funcionalidades existentes y que el código cumpla con los estándares definidos dentro del flujo GitFlow antes de su merge a la rama principal.

Sprint Reviews

Al final de cada sprint del proyecto se realizaron sesiones de Sprint Review con el equipo de desarrollo, en las cuales se presentó el incremento funcional del sistema NorthEye.

En estas sesiones se llevaron a cabo las siguientes actividades:

- Demostración de las funcionalidades implementadas en el sprint.
- Validación del cumplimiento de las historias de usuario definidas en el Product Backlog.
- Revisión del comportamiento del sistema en escenarios reales de uso.
- Recolección de feedback por parte del equipo y ajustes necesarios para el siguiente sprint.
- Validación de la integración entre backend, frontend y servicios REST.

Estas revisiones permitieron asegurar la alineación del producto con los objetivos del proyecto y facilitaron la toma de decisiones sobre mejoras en la arquitectura y experiencia de usuario.

Design & Architecture Reviews

Además de las revisiones de código y funcionalidad, se realizaron revisiones de diseño y arquitectura del sistema, enfocadas en garantizar la coherencia del modelo propuesto.

En este proceso se validaron:

- Consistencia del diseño basado en arquitectura hexagonal en el backend.
- Correcta separación de capas (domain, application, infrastructure, interfaces).
- Definición adecuada de entidades, agregados y relaciones en el modelo de datos.
- Coherencia entre el diseño del frontend y los servicios expuestos por la API.
- Alineación entre los diagramas de arquitectura y la implementación real.

Quality Assurance Reviews

Como parte del aseguramiento de calidad, se realizaron revisiones orientadas a verificar el cumplimiento de criterios de calidad del sistema, incluyendo:

- Correcto funcionamiento de endpoints mediante pruebas manuales.
- Validación de respuestas de la API en escenarios exitosos y de error.
- Revisión de consistencia en la interfaz de usuario.
- Evaluación de usabilidad básica en flujos principales del sistema.
- Verificación de integración entre módulos del sistema.

Resultado de las revisiones

Gracias a las revisiones realizadas a lo largo del desarrollo, se logró:

- Reducir errores en etapas tempranas del ciclo de desarrollo.

- Mejorar la calidad del código antes de su integración.
- Asegurar coherencia entre diseño, implementación y requisitos.
- Incrementar la estabilidad del sistema en cada sprint.
- Mantener trazabilidad entre historias de usuario y entregables funcionales.

6.3. Validation Interviews.

6.3.1. Diseño de Entrevistas.

Segmento #1: Bodegas especializadas por rubro

Presentación del entrevistado:

¿Cuál es tu nombre?

¿Qué edad tienes?

¿Desde cuándo gestionas esta bodega?

¿Cuál es el rubro principal de tu bodega?

El usuario interactúa con la app móvil de la plataforma para gestión de inventarios.

Landing page

Acciones clave dentro del sistema

Navegación interactivo por la interfaz app móvil

User Goal: Registrar

El usuario selecciona la opción "Register", completa los campos solicitados y hace clic en el botón "Registrar".

A continuación, se muestra el panel "Add Card", donde debe llenar los campos relacionados con su tarjeta y correo electrónico.

Una vez que el proceso de pago se complete exitosamente, se notifica al usuario con un mensaje confirmando el vínculo de su tarjeta con la plataforma.

Del mismo modo, si el usuario desea retirar su información o actualizarla, lo podrá hacer a través de su perfil.

Finalmente, hacer clic en el botón "Aceptar".

User Goal: Iniciar sesión

El usuario introduce su correo y contraseña.

Luego hace clic en el botón "Log In".

Después, se le redirige al panel de perfil.

Allí puede editar su información personal y acceder a herramientas según su perfil ("Administrador" o "Empleado").

User Goal: Navegar por el Dashboard

El usuario inicia sesión desde la Landing Page.

Ingresa a la vista principal del Dashboard.

Visualiza el total de productos registrados y la fecha del último proveedor.

Visualiza un resumen de productos próximos a caducar con su respectiva fecha y stock disponible.

Accede a botones de acción rápida como "Historial", "Inventario", "Añadir Productos", "Kits" y "Devolución de productos".

User Goal: Inventario (Producto / Lote)

Ingresa a la sección de Inventario.

Revisa el listado de productos presionando el botón "por producto".

Filtra los productos por categoría, nombre del producto, fecha o stock mínimo.

Consulta el listado con información clave: fecha de entrada, cantidad por unidad, precio, stock mínimo y unidad de medida.

User Goal: Botones Principales (Agregar Producto y Kits)

Pulsa el botón "Agregar Producto".

Rellena los campos solicitados para registrar uno nuevo.

Pulsa el botón "Crear Kit".

Combina productos existentes para crear un kit nuevo.

El usuario pulsa el botón "Añadir Productos" desde el Dashboard.

Visualiza una galería de productos existentes y accede a opciones para editarlos o duplicarlos.

Puede agregar uno nuevo haciendo clic en el botón "+", donde se despliega un formulario con campos como nombre, etiquetas, cantidad, lote, precios, fecha de caducidad y notas.

Desde el menú principal, también accede a la opción "Kits".

En esta sección, selecciona productos existentes y los combina mediante el botón "Seleccionar para kit", indicando cantidad e inventario disponible.

User Goal: Historial de Movimientos

Navega a la sección de Historial.

Visualiza entradas y salidas de productos.

Filtra movimientos por fecha, producto o lote.

El usuario accede a la sección de Historial desde el panel principal.

Filtra los registros por tipo de gestión, categoría, stock promedio y fecha.

Visualiza los movimientos realizados, incluyendo datos como nombre del producto, fecha de consulta, precio unitario, cantidad y total.

Consulta métricas como el stock promedio, estado del producto y stock ideal.

Cuenta con botones para editar o eliminar cada registro y, para los stock promedio, exportar la información y realiza un ticket promedio.

User Goal: Alerta de Stock

El usuario accede a la sección de Alertas desde el menú superior.

Consulta una lista de productos que presentan alertas por stock mínimo o por cercanía a su fecha de vencimiento.

Visualiza detalles como categoría de alerta, tipo de producto, nombre y fecha de alerta.

Puede ingresar a una vista de detalles, generar reportes o confirmar acciones desde botones individuales por producto.

También puede eliminar una alerta específica luego de una confirmación emergente.

Preguntas principales:

- ¿Te resultó fácil encontrar cómo agregar un producto?
- ¿Qué tal fue el proceso para registrar el stock por producto y por lote?
- ¿Te resultó claro el apartado de "Próximos a caducar"? ¿Te pareció útil?
- ¿Probaste la sección de kits? ¿Te resultó útil combinar productos?
- ¿Hubo algo que no entendiste o que te confundió mientras usabas la app?
- ¿Qué te pareció el diseño general de la landing page?
- ¿Tuviste alguna dificultad visual o técnica durante la navegación?

Valoración de experiencia

- Del 1 al 10, ¿qué tan útil te pareció la app para tu bodega?
- ¿Qué funcionalidad te pareció más valiosa?
- ¿Qué función le agregarías sí o sí?
- ¿Recomendarías esta plataforma a otros dueños de bodegas? ¿Por qué?
-

Segmento #2: Startups y emprendedores con necesidades logísticas

Presentación del entrevistado

- ¿Cuál es tu nombre?
- ¿Qué edad tienes?
- ¿Qué tipo de producto vendes o almacenas?
- ¿Tienes local físico, almacén propio o trabajas con terceros?
- Interactúa el usuario con la landing page y la versión web de nuestra plataforma para gestión de stock, registrar compras y generar reportes.

Landing page

Acciones clave dentro del sistema

Navegación interactivo por la interfaz app móvil

User Goal: Registrar

El usuario selecciona la opción "Register", completa los campos solicitados y hace clic en el botón "Registrar".

A continuación, se muestra el panel "Add Card", donde debe llenar los campos relacionados con su tarjeta y correo electrónico.

Una vez que el proceso de pago se complete exitosamente, se notifica al usuario con un mensaje confirmando el vínculo de su tarjeta con la plataforma.

Del mismo modo, si el usuario desea retirar su información o actualizarla, lo podrá hacer a través de su perfil.

Finalmente, hacer clic en el botón "Aceptar".

User Goal: Iniciar sesión

El usuario introduce su correo y contraseña.

Luego hace clic en el botón "Log In".

Después, se le redirige al panel de perfil.

Allí puede editar su información personal y acceder a herramientas según su perfil ("Administrador" o "Empleado").

User Goal: Navegar por el Dashboard

El usuario inicia sesión desde la Landing Page.

Ingresa a la vista principal del Dashboard.

Visualiza el total de productos registrados y la fecha del último proveedor.

Visualiza un resumen de productos próximos a caducar con su respectiva fecha y stock disponible.

Accede a botones de acción rápida como "Historial", "Inventario", "Añadir Productos", "Kits" y "Devolución de productos".

User Goal: Inventario (Lote)

Accede a la sección de Inventario por lote.

Filtra los lotes por nombre del producto, proveedor, fecha de entrada, cantidad o precio.

Consulta el listado con información detallada como proveedor, producto, fecha, cantidad por unidad, precio y unidad de medida.

Gestiona las acciones disponibles para cada lote desde la columna correspondiente.

User Goal: Botones Principales (Agregar Producto y Kits)

Pulsa el botón "Agregar Producto".

Rellena los campos solicitados para registrar uno nuevo.

Pulsa el botón "Crear Kit".

Combina productos existentes para crear un kit nuevo.

El usuario pulsa el botón "Añadir Productos" desde el Dashboard.

Visualiza una galería de productos existentes y accede a opciones para editarlos o duplicarlos.

Puede agregar uno nuevo haciendo clic en el botón "+", donde se despliega un formulario con campos como nombre, etiquetas, cantidad, lote, precios, fecha de caducidad y notas.

Desde el menú principal, también accede a la opción "Kits".

En esta sección, selecciona productos existentes y los combina mediante el botón "Seleccionar para kit", indicando cantidad e inventario disponible.

User Goal: Historial de Movimientos

Navega a la sección de Historial.

Visualiza entradas y salidas de productos.

Filtra movimientos por fecha, producto o lote.

El usuario accede a la sección de Historial desde el panel principal.

Filtra los registros por tipo de gestión, categoría, stock promedio y fecha.

Visualiza los movimientos realizados, incluyendo datos como nombre del producto, fecha de consulta, precio unitario, cantidad y total.

Consulta métricas como el stock promedio, estado del producto y stock ideal.

Cuenta con botones para editar o eliminar cada registro y, para los stock promedio, exportar la información y realiza un ticket promedio.

User Goal: Alerta de Stock

El usuario accede a la sección de Alertas desde el menú superior.

Consulta una lista de productos que presentan alertas por stock mínimo o por cercanía a su fecha de vencimiento.

Visualiza detalles como categoría de alerta, tipo de producto, nombre y fecha de alerta.

Puede ingresar a una vista de detalles, generar reportes o confirmar acciones desde botones individuales por producto.

También puede eliminar una alerta específica luego de una confirmación emergente.

Preguntas principales:

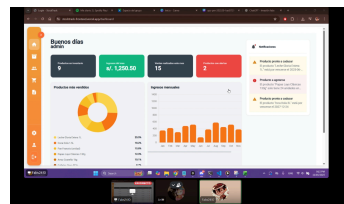
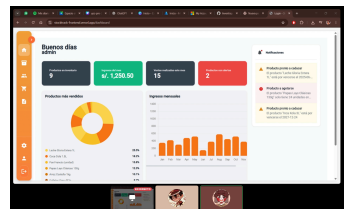
- ¿Qué opinas de las alertas de stock mínimo/máximo y vencimiento? ¿Son suficientes?
- ¿La sección de historial de movimientos te pareció útil para revisar tus registros?
- ¿Te resultó útil la opción de devolución de productos?
- ¿Los reportes del historial son útiles para tomar decisiones?
- ¿Hubo algo que no entendiste o que te confundió mientras usabas la app?
- ¿Qué te pareció el diseño general de la landing page?
- ¿Tuviste alguna dificultad visual o técnica durante la navegación?

Valoración de experiencia

- Del 1 al 10, ¿qué tan útil te pareció la app móvil para tu bodega?
- ¿Qué funcionalidad te pareció más valiosa?
- ¿Qué función le agregarías sí o sí?
- ¿Recomendarías esta plataforma a otros emprendedores? ¿Por qué?

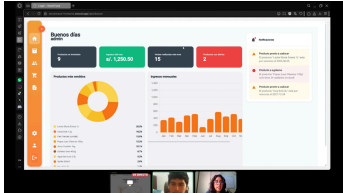
6.3.2. Registro de Entrevistas.

Segmento #1: Dueños de bodegas

Nº	Datos del entrevistado	Resumen de la entrevista	Evidencia de entrevista
1	- Nombre: Ariana Agreda - Edad: 20 años	Ariana ayuda a su familia en el negocio familiar el cual es una bodega. Comenta que le resulta amigable y sencillo el flujo de la aplicación. Le ha gustado las notificaciones acerca de los productos que están por vencerse y agotarse. De igual forma, le ha parecido bueno como funciona lo de ingresar productos. Ella nos sugiere que para los reportes y pantalla de proveedores deberían mostrarse qué productos distribuye cada proveedor para un mayor seguimiento.	 Validation Interview Ariana
2	- Nombre: Mauricio Elera - Edad: 25 años	Mauricio administra el inventario de un negocio en crecimiento y percibe que StockWise le brinda una experiencia profesional y confiable gracias a su diseño ordenado y fácil de navegar. Destaca que las métricas rápidas y las alertas inteligentes le permiten reaccionar a tiempo ante riesgos como el bajo stock o productos por vencer. Considera muy intuitivo el proceso para registrar productos y gestionar roles de personal. Aunque está satisfecho con la estructura general del sistema, señala que un módulo de facturación y la integración con software contable harían la plataforma mucho más completa. Para él, la mayor ventaja de StockWise es centralizar toda la operación en un solo lugar, lo que le ayuda a reducir errores y mantener una gestión mucho más eficiente.	 Validation Interview Mauricio

Nº	Datos del entrevistado	Resumen de la entrevista	Evidencia de entrevista
3	- Nombre: Marcelo Binda - Edad: 26	Marcelo Binda considera que StockWise representa una mejora significativa frente a los métodos tradicionales de gestión de inventario que utiliza actualmente, como cuadernos y hojas de Excel. Destaca que la aplicación le permite tener un mayor control y visibilidad del stock en tiempo real, reduciendo la incertidumbre sobre la disponibilidad de productos. Además, valora positivamente la simplicidad de la interfaz y la rapidez con la que puede realizar tareas básicas como el registro de productos y la consulta del inventario. Señala que la incorporación de alertas automáticas y un dashboard centralizado le ayuda a tomar decisiones más informadas y evitar pérdidas por falta de control. En general, percibe la herramienta como una solución práctica y adecuada para pequeñas bodegas que buscan digitalizar su operación.	 Entrevista 3 - Dueños de bodegas Validation Interview Marcelo (YouTube)

Segmento #2: Startups y emprendedores en expansión con necesidades logísticas

Nº	Datos del entrevistado	Resumen de la entrevista	Evidencia de entrevista
1	- Nombre: Alexander Miranda Vivanco - Edad: 27 años	Alexander posee un emprendimiento de venta de artículos para mascotas. Durante la entrevista reconoció que StockWise le sería de mucha ayuda por la logística del inventario. Las métricas del dashboard le parecieron idóneas. Sugirió facilitar el flujo de cuando se crea un producto y se agrega una reposición. Asimismo, comentó que le sería de mucha utilidad generar reportes acerca de aquellos productos que se hayan vencido. También especificó que poder agregar etiquetas personalizables sería adecuado y recomendó implementar una mejor lógica respecto a la reposición del stock.	 Validation Interview Alexander

6.3.3. Evaluaciones segun heurísticas.

TAREAS A EVALUAR: El alcance de esta evaluación incluye la revisión de la usabilidad de las siguientes tareas:

1. Navegación entre módulos del dashboard (Inicio, Inventario, Proveedores, etc.)
2. Creación de un Kit de productos
3. Búsqueda y filtrado en la sección de Proveedores

- 4. Gestión de usuarios
- 5. Generación de reportes avanzados

No están incluidas en esta versión de la evaluación las siguientes tareas:

- 1. Registro de nuevo usuario
- 2. Proceso de pago
- 3. Integraciones con APIs

ESCALA DE SEVERIDAD:

Nivel	Descripción
1	Problema superficial: puede ser fácilmente superado por el usuario ó ocurre con muy poca frecuencia. No necesita ser arreglado a no ser que exista disponibilidad de tiempo.
2	Problema menor: puede ocurrir un poco más frecuentemente o es un poco más difícil de superar para el usuario. Se le debería asignar una prioridad baja resolverlo de cara al siguiente release.
3	Problema mayor: ocurre frecuentemente o los usuarios no son capaces de resolverlos. Es importante que sean corregidos y se les debe asignar una prioridad alta.
4	Problema muy grave: un error de gran impacto que impide al usuario continuar con el uso de la herramienta. Es imperativo que sea corregido antes del lanzamiento.

TABLA RESUMEN:

#	Problema	Escala de severidad	Heurística/Principio violada(o)
1	Duplicidad redundante de alertas de stock en la vista general del Dashboard	2	Consistencia y estándares / Reducción de carga cognitiva
2	Inoperancia total de la búsqueda y el botón "Filtrar" en el módulo de Proveedores	3	Retroalimentación del sistema / Flexibilidad y eficiencia
3	Falta de responsividad y superposición de textos en el contenedor lateral de Kits	3	Diseño estético y minimalista
4	Bloqueo funcional en el menú de acciones de la tabla de Inventario	4	Control y libertad del usuario
5	Permisi3n de sobreventa de Kits sin stock e insuficiencia en la gesti3n de errores	4	Prevenci3n de errores / Recuperaci3n ante errores

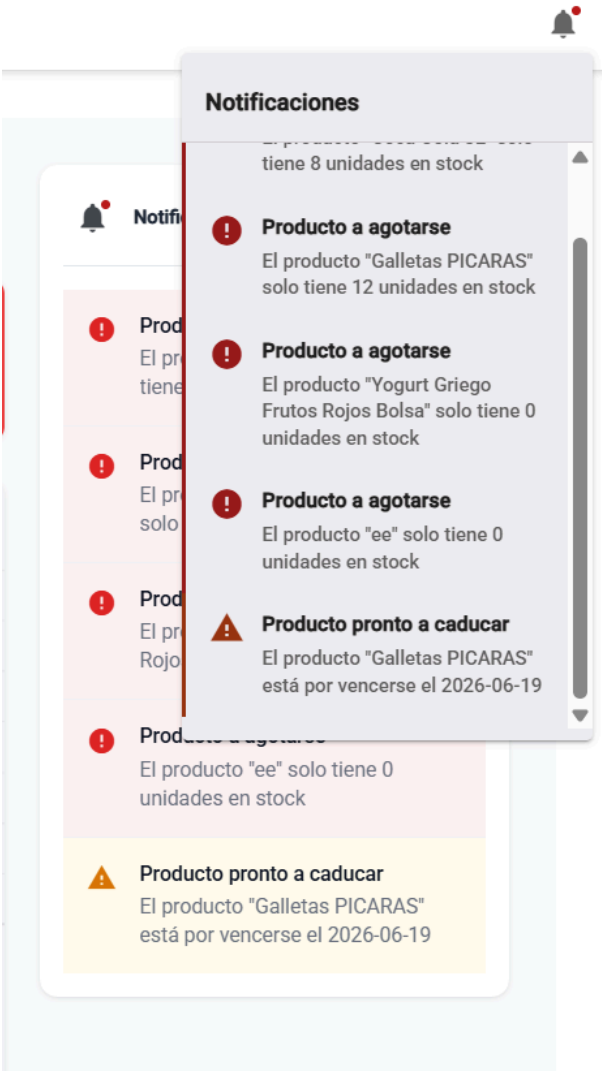
Detalle de hallazgos

- Hallazgo H-01

Campo	Detalle
Tarea evaluada	Visualizaci3n de alertas en el Dashboard

Campo	Detalle
Ubicación	Dashboard general - Icono de notificaciones (Campana superior)
Problema identificado	Duplicidad redundante de alertas de stock en la misma pantalla
Severidad	2
Heurística vulnerada	Consistencia y estándares / Diseño estético y minimalista
Descripción	Al hacer clic en el icono de la campana en la barra superior, el panel flotante de notificaciones se despliega directamente sobre una vista principal que ya tiene como único propósito listar de forma idéntica las alertas de "Producto a agotarse" y "Producto pronto a caducar".
Impacto para el usuario	Genera ruido visual innecesario y redundancia de datos. Al presentarse la misma información en dos capas superpuestas, se satura la interfaz y se sobrecarga cognitivamente al usuario sin aportar valor real.
Recomendación	Implementar una validación de ruta: si el usuario ya se encuentra dentro de la sección principal de notificaciones/alertas, el dropdown de la campana superior debe desactivarse o simplemente no abrirse al hacer clic, quedando como un indicador visual estático.

Evidencia *Figura 1. Menú desplegable de notificaciones duplicando la información de la sección de fondo.*



• Hallazgo H-02

Campo	Detalle
Tarea evaluada	Búsqueda y filtrado de proveedores
Ubicación	Módulo de Proveedores - Barra de herramientas superior
Problema identificado	Inoperancia funcional de la barra de búsqueda y el botón "Filtrar"
Severidad	3
Heurística vulnerada	Retroalimentación del sistema / Flexibilidad y eficiencia de uso
Descripción	Los componentes de interacción en la sección de Proveedores (el campo de texto para buscar y el botón "Filtrar") actúan visualmente como elementos interactivos. Sin embargo, al ingresar datos o hacer clic en ellos, la tabla de registros permanece estática sin procesar el filtro ni emitir señales de carga.
Impacto para el usuario	Alta frustración y sensación de que la plataforma está rota. Si el volumen de proveedores crece, la incapacidad de buscar o filtrar obliga al escaneo manual, rompiendo la eficiencia del flujo de trabajo.

Campo	Detalle
Recomendación	Conectar el valor del <code><input></code> a un filtro reactivo en el frontend o disparar la petición correspondiente al backend. Si la opción avanzada de "Filtrar" aún no está desarrollada, se debe deshabilitar el botón visualmente y añadir un <i>tooltip</i> informativo: "Función en desarrollo".

Evidencia

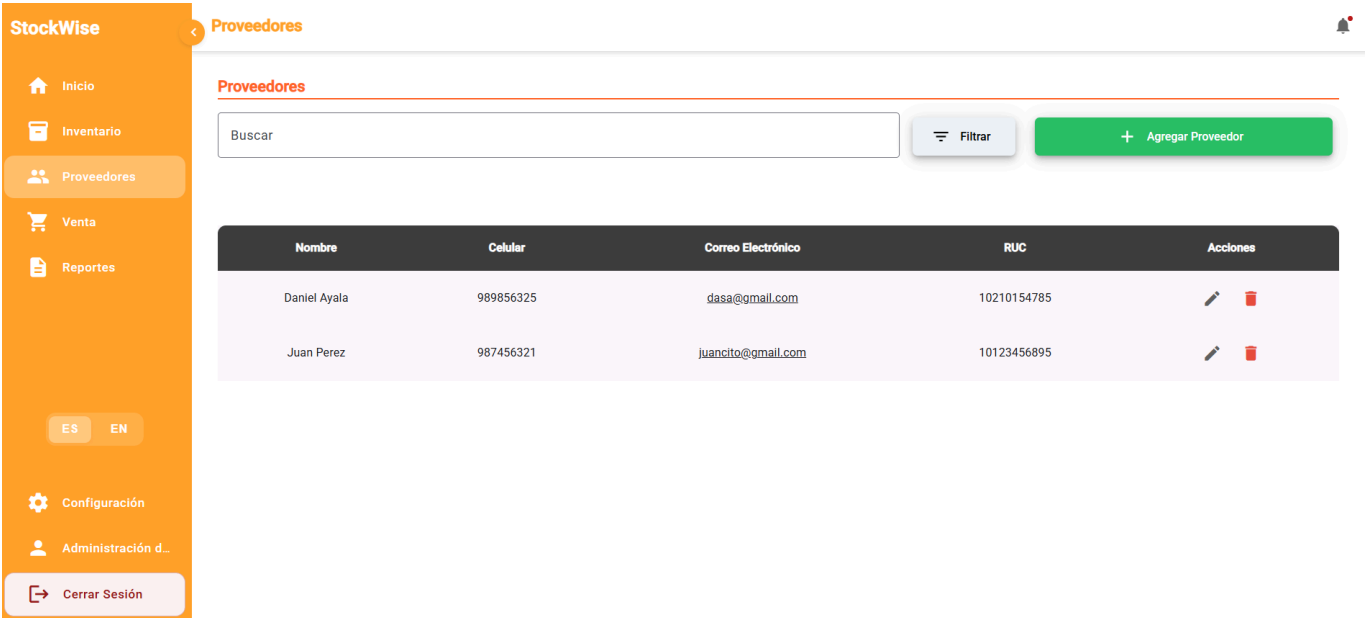


Figura 2. Módulo de proveedores con controles de búsqueda y filtrado inactivos.

• Hallazgo H-03

Campo	Detalle
Tarea evaluada	Visualización de Kits estructurados
Ubicación	Barra lateral derecha - Panel de visualización de Kits
Problema identificado	Falta de responsividad y superposición de textos (<i>overlapping</i>)
Severidad	3
Heurística vulnerada	Diseño estético y minimalista (Visual UI Bug)
Descripción	En la tarjeta de detalle de los componentes del kit (ej. "Pack Galletas Deli"), las cajas de texto no se adaptan correctamente al ancho reducido de la barra lateral. Esto ocasiona que los nombres largos de los productos colisionen y se encimen sobre las etiquetas de "Precio unitario" y "Total".
Impacto para el usuario	Perjudica gravemente la legibilidad de los costos de cada ítem del kit, lo que puede inducir a errores de cálculo visuales por parte del usuario y proyecta un acabado descuidado de la interfaz.

Campo	Detalle
Recomendación	Modificar las reglas CSS del contenedor interno del ítem. Se sugiere aplicar una propiedad flex-direction en columna (<code>flex-direction: column</code>) cuando el espacio sea estrecho para situar los precios debajo del nombre, o aplicar un límite de ancho con truncamiento de texto (<code>text-overflow: ellipsis</code>).

Evidencia

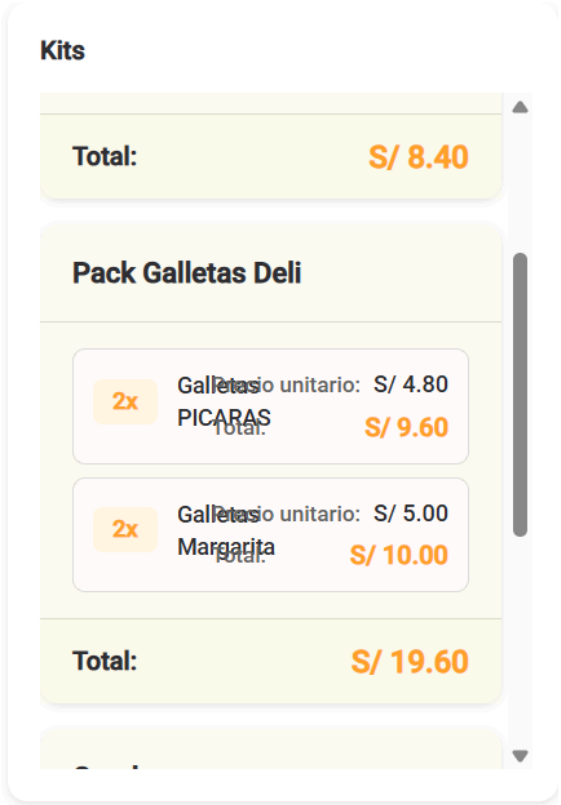


Figura 3. Superposición visual de textos y precios en el bloque del Kit.

- Hallazgo H-04

Campo	Detalle
Tarea evaluada	Gestión y edición de existencias del inventario
Ubicación	Módulo de Inventario - Tabla de Productos (Columna Acciones)
Problema identificado	Bloqueo funcional total al seleccionar las opciones del menú de acciones
Severidad	4
Heurística vulnerada	Control y libertad del usuario
Descripción	Al interactuar con el botón de tres puntos en la fila de cualquier producto, el menú contextual se despliega de manera correcta mostrando las acciones de "Ver detalle",

Campo	Detalle
	"Editar" y "Eliminar". No obstante, al hacer clic sobre cualquiera de las opciones, el sistema las ignora por completo y no ejecuta ninguna acción.
Impacto para el usuario	Bloqueo crítico del flujo de administración. El usuario pierde el control y la libertad sobre los datos del inventario, viéndose imposibilitado de actualizar precios, corregir datos o eliminar ítems obsoletos.
Recomendación	Verificar que las opciones del menú tengan correctamente enlazados sus manejadores de eventos (onClick). Adicionalmente, revisar que no exista un conflicto de capas (z-index) que coloque el menú detrás de una capa invisible, impidiendo registrar la pulsación del ratón.

Evidencia

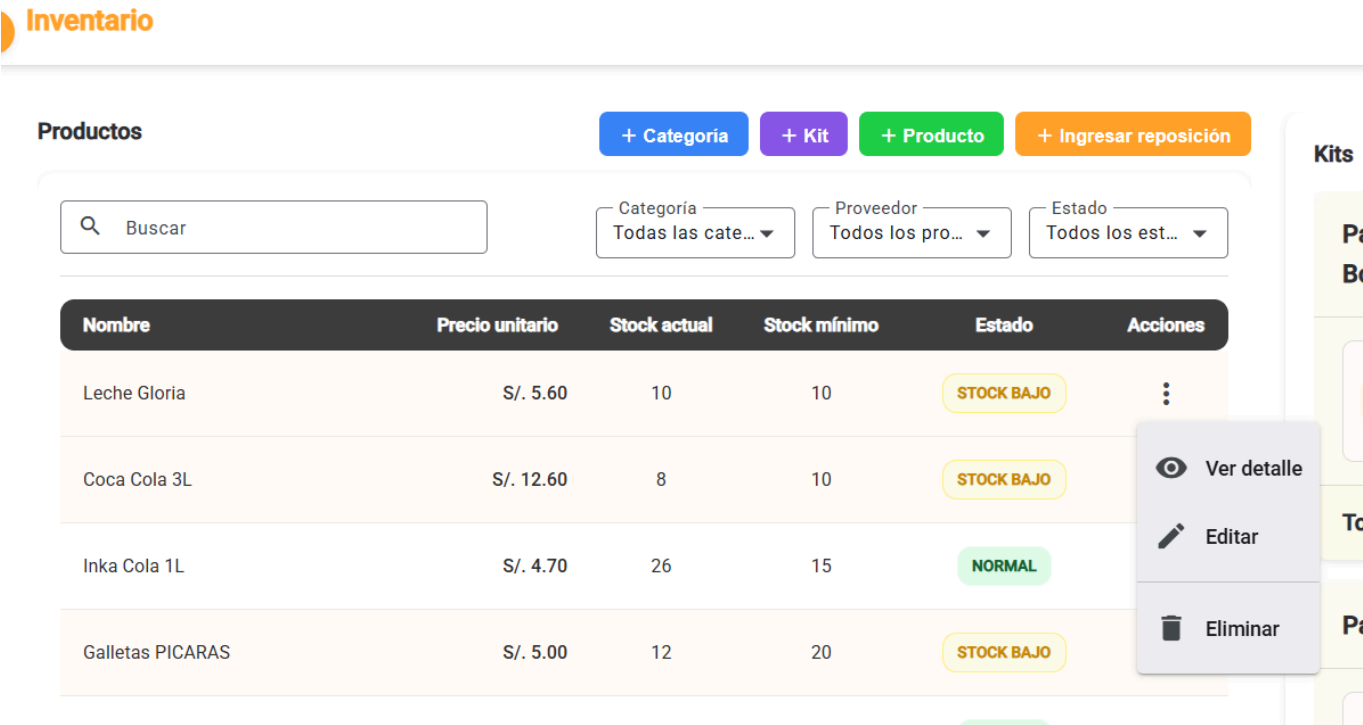


Figura 4. Menú de acciones desplegado pero inoperante al clic.

- Hallazgo H-05

Campo	Detalle
Tarea evaluada	Control de flujo de existencias y registro de ventas
Ubicación	Módulo de Gestión de Ventas - Borrador de salida de productos
Problema identificado	Permición de sobreventa de Kits sin stock disponible y manejo de error genérico
Severidad	4
Heurística vulnerada	Prevención de errores / Ayuda a reconocer y recuperarse de errores

Campo	Detalle
Descripción	El selector del borrador de salida permite añadir cantidades ilimitadas de un kit (ej. 14 unidades de "Combo gaseosas"), ignorando que los productos que lo componen individualmente tienen stock en cero en la tabla izquierda. Al intentar "Guardar", el sistema procesa la solicitud inválida y arroja un banner de error genérico: "Error al guardar la venta. Por favor, intente nuevamente".
Impacto para el usuario	Muy grave. Permite la venta ficticia de productos inexistentes y desorienta al usuario con un mensaje de error que sugiere un problema del servidor en lugar de explicar la causa real (falta de existencias). El usuario queda atrapado sin saber cómo corregir la acción.
Recomendación	1. En Frontend: Deshabilitar dinámicamente el botón + de incremento en el borrador si la cantidad del kit requiere más unidades de las disponibles en el stock real de cualquiera de sus componentes. 2. En Mensajería: Cambiar la respuesta del banner por un texto descriptivo: "No se pudo registrar la venta: Stock insuficiente en los componentes del Kit".

Evidencia

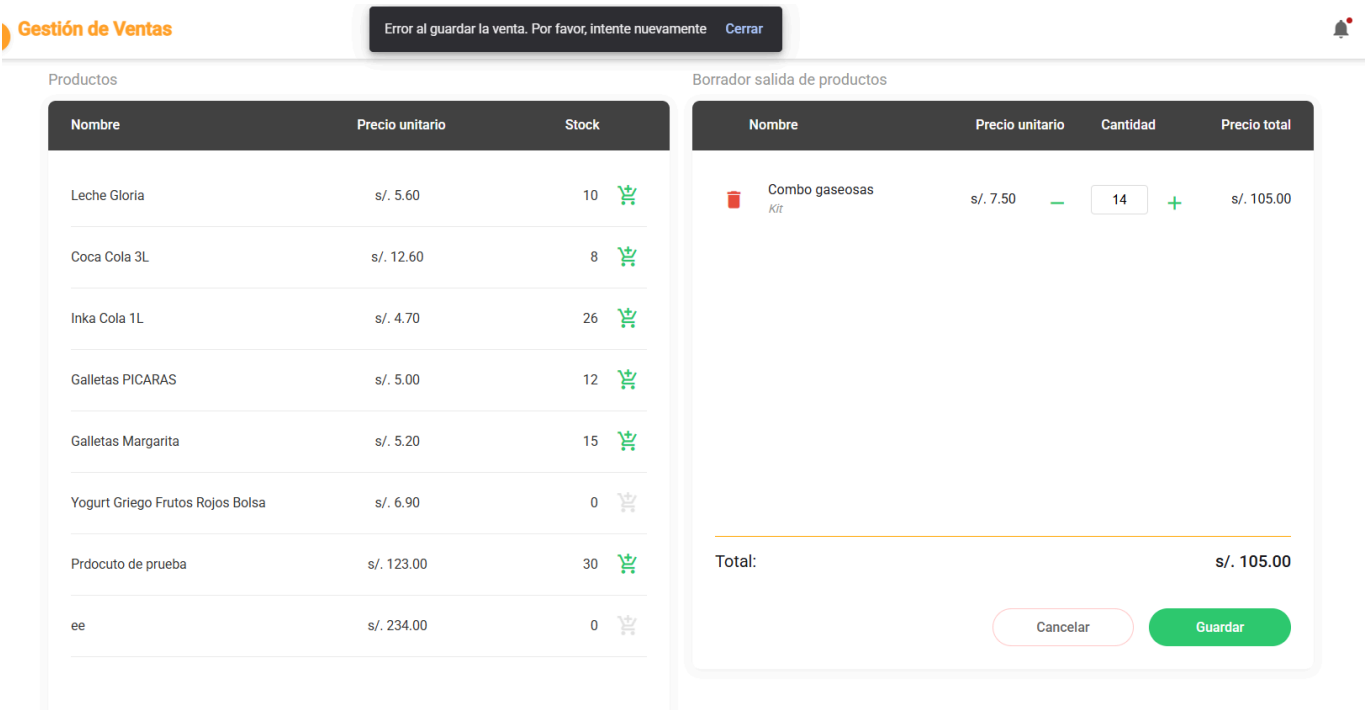


Figura 5. Ventana de ventas mostrando el incremento desmedido de un kit sin stock y el consecuente banner de error genérico.

6.4. Auditoria de Experiencias de Usuario.

La auditoría de experiencias de usuario fue llevada a cabo con el objetivo de obtener retroalimentación externa sobre la calidad de la interfaz y la arquitectura de información de StockWise, así como de brindar retroalimentación a otro equipo de trabajo. El proceso contempló dos dimensiones: la auditoría realizada a un grupo externo y la auditoría recibida por parte de otro grupo evaluador.

6.4.1. Auditoria realizada.

6.4.1.1. Información del grupo auditado.

Campo	Detalle
Grupo auditado	Grupo 2
Startup	EduLabs
Producto evaluado	Demy
Integrantes del equipo	Daniel Crispin, Rafael Dominguez, Henry Esteban, Renso Julca y Diego Vilca
Repositorio del proyecto	https://github.com/orgs/EduLabs-Experimentos/repositories
Landing Page	https://demy-edulabs.netlify.app/
Aplicación web	https://edulabs-experimentos.github.io/demy-admin-web/sign-up

6.4.1.2. Cronograma de auditoría realizada

Actividad	Fecha	Responsable	Duración estimada
Recepción del informe de auditoría externa	11/06/2026	Ronald Peralta	30 minutos
Revisión y análisis de hallazgos identificados	11/06/2026	Ronald Peralta	1 hora
Priorización de observaciones y acciones correctivas	12/06/2026	Ronald Peralta	30 minutos
Evaluación integral del flujo funcional de la aplicación	13/06/2026	Ronald Peralta	1 hora
Implementación de mejoras y corrección de incidencias	13/06/2026	Ronald Peralta	30 minutos

6.4.1.3. Contenido de la auditoría realizada

La presente auditoría tuvo como objetivo evaluar la usabilidad y experiencia de usuario del producto desarrollado por el grupo auditado. Para ello, se analizaron los principales flujos de interacción de la aplicación, identificando problemas relacionados con navegación, retroalimentación del sistema, accesibilidad, consistencia visual y comprensión de la interfaz.

Tareas evaluadas

Durante la auditoría se ejecutaron las siguientes tareas representativas del flujo de uso de la aplicación:

N.º	Tarea evaluada	Descripción
1	Navegación global	Interacción con la barra de navegación (Header) para retornar a la vista principal.
2	Visualización de contenido	Carga y renderizado de los recursos multimedia en las distintas secciones informativas.
3	Análisis de rendimiento	Evaluación de métricas de carga, accesibilidad y SEO utilizando la herramienta Google Lighthouse.
4	Arquitectura de la información	Recorrido por las secciones de la Landing Page para evaluar la carga cognitiva y relevancia del contenido.
5	Flujo de conversión	Búsqueda del enlace o botón principal (Call to Action) para iniciar el proceso de registro en la Web App.
6	Navegación interna	Exploración de las opciones secundarias del menú de navegación ("Configuración" y "Ayuda").
7	Registro de datos de usuario	Ingreso de información demográfica y de contacto en el formulario de la sección "ESTUDIANTES".
8	Gestión de disponibilidad	Configuración de rangos de tiempo (hora de inicio y fin) en la sección de "Horarios".
9	Adaptabilidad visual	Evaluación de la legibilidad de la interfaz al alternar el esquema de colores (Modo Oscuro) en la sección "Cobranzas".

Escala de severidad utilizada

Nivel	Descripción
1	Problema superficial que no afecta significativamente la experiencia del usuario.
2	Problema menor que genera cierta dificultad pero permite completar la tarea.
3	Problema importante que dificulta considerablemente la interacción.
4	Problema crítico que impide completar la tarea o continuar utilizando el sistema.

Resumen de hallazgos identificados

ID	Tarea	Problema identificado	Severidad	Heurística vulnerada
H-01	Navegación global	Logotipo inactivo sin redirección a la página de inicio.	2	Consistencia y estándares
H-02	Visualización de contenido	Imagen rota o no renderizada en la sección "Nosotros".	2	Diseño estético y minimalista

ID	Tarea	Problema identificado	Severidad	Heurística vulnerada
H-03	Análisis de rendimiento	Bajo puntaje de rendimiento (59/100) en la métrica de Lighthouse.	3	Flexibilidad y eficiencia de uso
H-04	Arquitectura de la información	Exceso de secciones y contenido redundante (sección "Pasos").	1	Diseño estético y minimalista
H-05	Flujo de conversión	Ausencia total de botón CTA hacia el registro de la aplicación web.	4	Control y libertad del usuario
H-06	Navegación interna	Enlaces inactivos en el menú de navegación principal.	3	Control y libertad del usuario
H-07	Registro de datos de usuario	Ausencia de validaciones lógicas en campos de teléfono y fecha de nacimiento.	3	Prevención de errores
H-08	Gestión de disponibilidad	Interfaz de selección de horas (Time Picker) poco intuitiva y demandante.	2	Flexibilidad y eficiencia de uso
H-09	Adaptabilidad visual	Deficiencia de contraste y pérdida de claridad en el modo oscuro (Módulo de facturas).	2	Diseño estético y minimalista

Detalle de hallazgos

Hallazgo H-01

Campo	Detalle
Tarea evaluada	Navegación global
Ubicación	Landing Page - Barra de navegación superior (Header)
Problema identificado	Logotipo de la marca inactivo sin hipervínculo al Home
Severidad	2
Heurística vulnerada	Consistencia y estándares
Descripción	Al navegar por las distintas secciones de la página, el usuario intenta hacer clic en el logotipo "Demy" para regresar rápidamente al inicio, pero este elemento es estático y no contiene un enlace (etiqueta <a>).
Impacto para el usuario	Genera fricción en la navegación. El usuario se ve obligado a hacer scroll manual hacia arriba o buscar alternativas para regresar, rompiendo una convención estándar de diseño web.

Campo	Detalle
Recomendación	Envolver el logotipo en una etiqueta de ancla (<a>) o componente de enrutamiento que apunte a la ruta raíz (/ o #home).

Evidencia

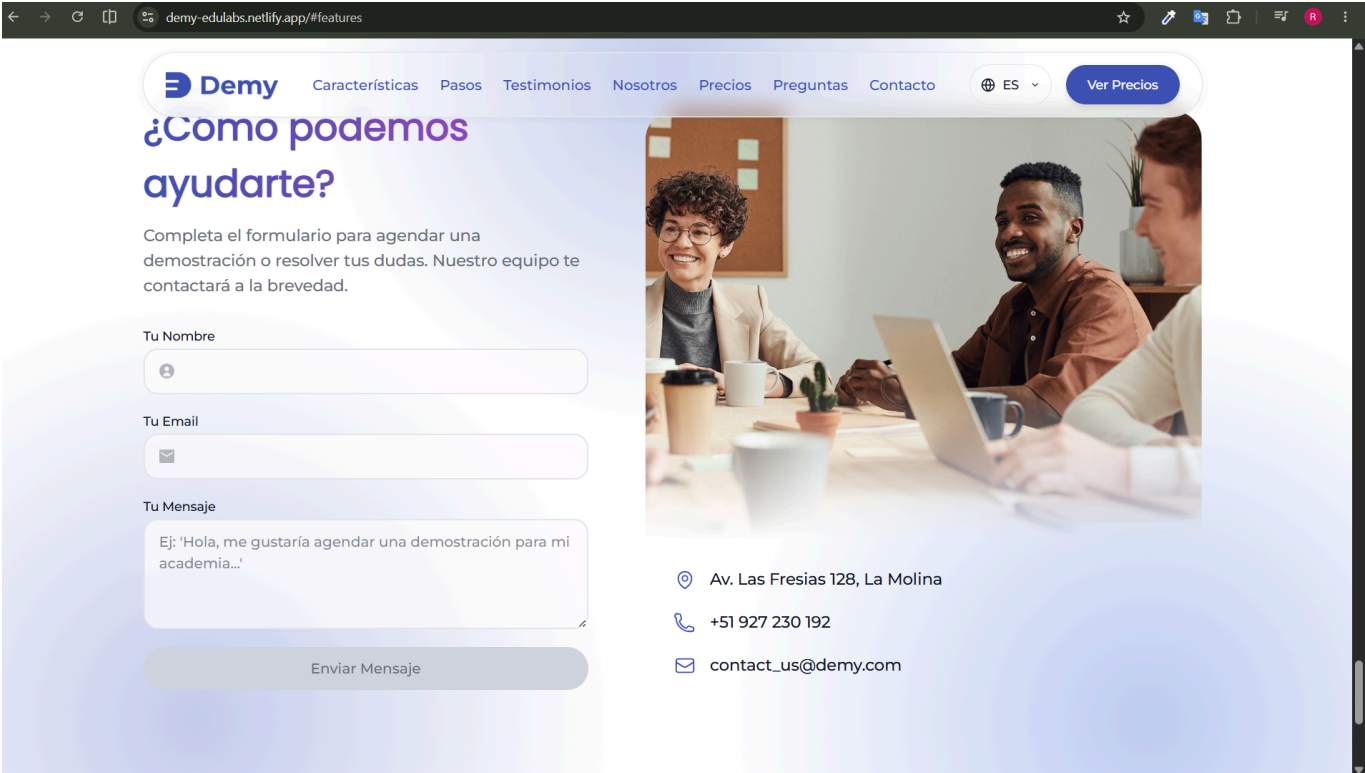


Figura 1. Barra de navegación donde se observa el logotipo estático.

Hallazgo H-02

Campo	Detalle
Tarea evaluada	Visualización de contenido
Ubicación	Landing Page - Sección "Nosotros"
Problema identificado	Elemento de imagen no disponible (Enlace roto)
Severidad	2
Heurística vulnerada	Diseño estético y minimalista / Prevención de errores
Descripción	Dentro del apartado "Nosotros", uno de los recursos gráficos no logra cargar correctamente, mostrando el icono estándar de "imagen rota" o un espacio en blanco en el navegador.
Impacto para el usuario	Disminuye la percepción de calidad y profesionalismo del producto. Puede generar desconfianza en el usuario sobre el mantenimiento del sitio.

Campo	Detalle
Recomendación	Verificar la ruta del atributo <code>src</code> en la etiqueta <code></code> . Asegurar que el archivo exista en el directorio de despliegue o proveer un atributo <code>alt</code> descriptivo junto con una imagen de respaldo (fallback).

Evidencia

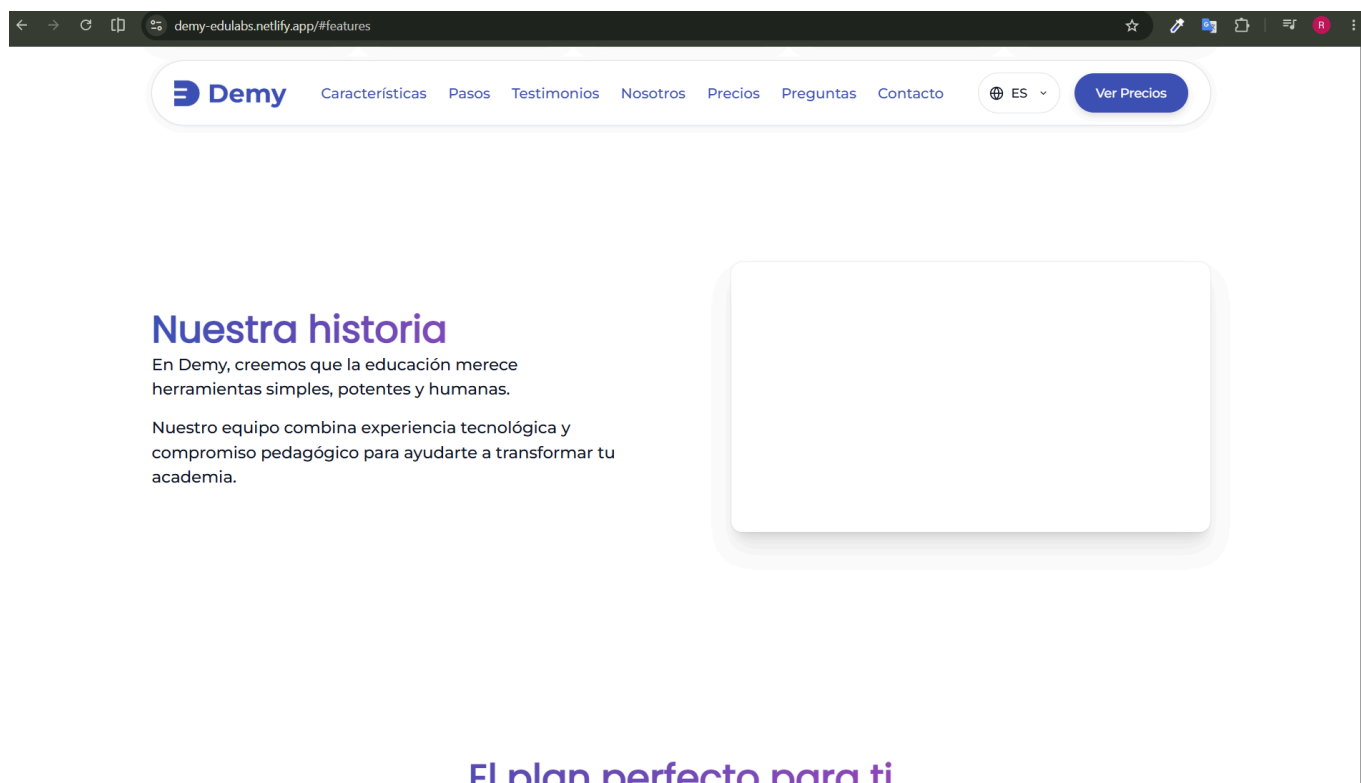


Figura 2. Sección "Nosotros" evidenciando el error de carga del recurso multimedia.

Hallazgo H-03

Campo	Detalle
Tarea evaluada	Análisis de rendimiento
Ubicación	Landing Page - Entorno global
Problema identificado	Puntaje deficiente de Performance (59/100) en Lighthouse
Severidad	3
Heurística vulnerada	Flexibilidad y eficiencia de uso
Descripción	La auditoría técnica con Google Lighthouse arroja resultados contrastantes: Accesibilidad (90), Buenas prácticas (77), SEO (91), pero el Rendimiento cae a 59/100, indicando tiempos de carga lentos para el hilo principal.

Campo	Detalle
Impacto para el usuario	Un bajo rendimiento impacta directamente en los Core Web Vitals (como el LCP). Los usuarios con conexiones inestables pueden abandonar la página antes de que termine de cargar.
Recomendación	Optimizar el tamaño y formato de las imágenes (usar WebP), diferir la carga de JavaScript no crítico y minificar los archivos estáticos CSS/JS para mejorar el First Contentful Paint (FCP).

Evidencia

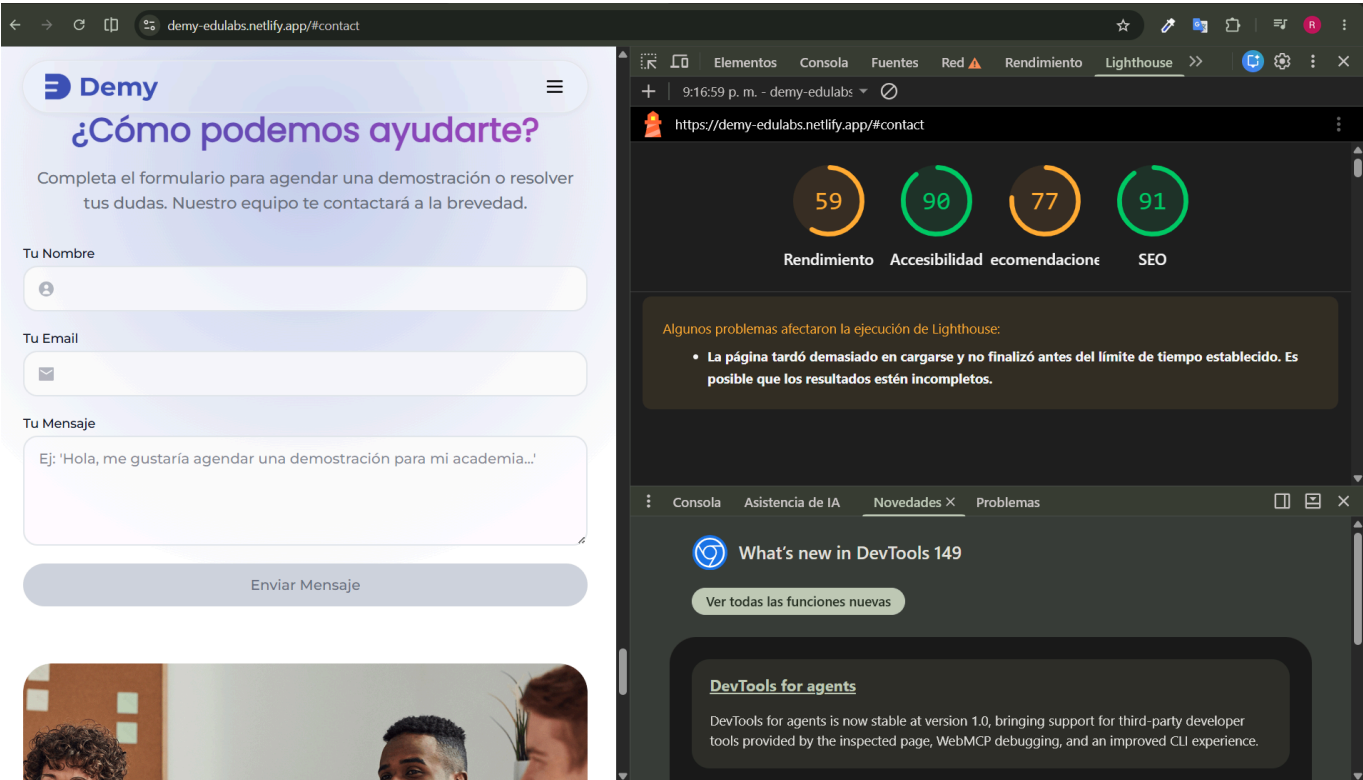


Figura 3. Resultados del análisis de Google Lighthouse.

Hallazgo H-04

Campo	Detalle
Tarea evaluada	Arquitectura de la información
Ubicación	Landing Page - Cuerpo de la página (Sección "Pasos")
Problema identificado	Sobrecarga de información y redundancia de secciones
Severidad	1
Heurística vulnerada	Diseño estético y minimalista

Campo	Detalle
Descripción	La página presenta un volumen excesivo de secciones verticales (Características, Pasos, Testimonios, Nosotros, Precios, Preguntas, Contacto). Específicamente, la sección "Pasos" describe un proceso que resulta ser demasiado trivial, sin aportar valor real.
Impacto para el usuario	Incrementa la fatiga visual y la carga cognitiva del usuario al tener que hacer demasiado scroll. Diluye la atención y desvía al usuario de la acción principal que debe realizar.
Recomendación	Simplificar la estructura de la Landing Page. Eliminar la sección "Pasos" y condensar u ocultar información secundaria en acordeones o enlaces internos para mantener un flujo más directo.

Evidencia

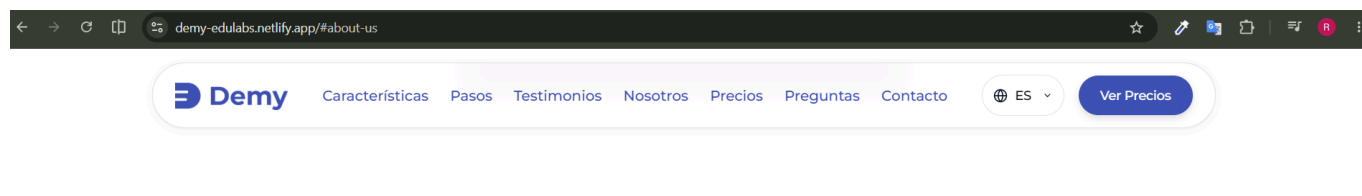


Figura 4. Sección "Pasos" que aporta baja relevancia al flujo de la página.

Hallazgo H-05

Campo	Detalle
Tarea evaluada	Flujo de conversión
Ubicación	Landing Page - Menú de navegación y Hero Section
Problema identificado	Ausencia de un botón de llamada a la acción (CTA) hacia la aplicación web
Severidad	4
Heurística vulnerada	Control y libertad del usuario / Prevención de errores
Descripción	El propósito principal de la Landing Page es captar usuarios, pero no existe un botón prominente (como "Regístrate" o "Ingresar a la App") que dirija al usuario al flujo de registro (/sign-up) de la Web App.
Impacto para el usuario	Impide por completo la conversión. El usuario interesado en el producto se queda estancado en la Landing Page sin un camino claro para comenzar a utilizar la aplicación, frustrando el objetivo de negocio.
Recomendación	Implementar botones CTA de alto contraste y tamaño adecuado tanto en la esquina superior derecha del menú de navegación como en la sección principal (Hero Section), enlazados directamente a la vista de registro.

Evidencia



Figura 5. Ausencia de botones de registro en el primer impacto visual de la página.

Hallazgo H-06

Campo	Detalle
Tarea evaluada	Navegación interna
Ubicación	Web App - Barra de navegación lateral/superior
Problema identificado	Secciones "Configuración" y "Ayuda" sin funcionalidad
Severidad	3
Heurística vulnerada	Control y libertad del usuario
Descripción	Al intentar acceder a las opciones de "Configuración" y "Ayuda" desde el menú principal, los enlaces no ejecutan ninguna acción, no redirigen a ninguna vista, ni muestran un mensaje indicando que la función está en construcción.
Impacto para el usuario	Genera confusión y frustración. El usuario percibe la aplicación como incompleta o defectuosa al no poder acceder a herramientas críticas de gestión de su cuenta o soporte.
Recomendación	Implementar las vistas correspondientes para estas rutas. Si las funcionalidades aún no están desarrolladas, se deben ocultar temporalmente los enlaces del menú o mostrar un modal informativo de "Próximamente" (Coming soon).

Evidencia

 Contabilidad

 Configuración

 Ayuda

 Cerrar Sesión

Figura 6. Elementos del menú de navegación que no presentan interactividad.

Hallazgo H-07

Campo	Detalle
Tarea evaluada	Registro de datos de usuario
Ubicación	Web App - Sección "ESTUDIANTES" (Formulario de creación/edición)
Problema identificado	Falta de validación lógica en inputs de teléfono y fecha de nacimiento
Severidad	3
Heurística vulnerada	Prevención de errores
Descripción	El formulario presenta dos fallas críticas de validación: 1) El campo de número de celular restringe la entrada estrictamente a 9 dígitos y no permite ingresar códigos de país (ej. +51), limitando la internacionalización. 2) El selector de fecha de cumpleaños permite seleccionar la fecha actual o fechas futuras.
Impacto para el usuario	Compromete la integridad de la base de datos al permitir el ingreso de información irreal o malformada. Limita el uso de la plataforma a un formato telefónico local, excluyendo posibles usuarios extranjeros.
Recomendación	Implementar validaciones de fecha (ej. <code>max="[fecha-actual]"</code> en el input de tipo date) para bloquear fechas futuras en los cumpleaños. Incorporar un componente de Input Phone que incluya un selector desplegable de códigos de país y ajuste la longitud esperada según la región.

Evidencia

Gestiona los estudiantes matriculados y su información personal

Editar Estudiante

Nombre

juli

Apellido

julios

DNI

84637586

Código

Ej. STU-001

Correo Electrónico

jul@gmail.com

Código

+234

Teléfono

11111112

Sexo

Female

Cumpleaños

12/06/2026

Dirección

nunca mas

Distrito

LIMA

Provincia / Depto

Lima

Departamento

Lima

Actualizar Estudiante

Buscar estudiantes...

Perfil

Correo Electrónico

Teléfono

Actions

jj

juli julios

✉

jul@gmail.com

☎

11111112

✎

🗑

<<< < 1 > >>>

Figura 7. Formulario aceptando datos incongruentes en los campos de contacto y nacimiento.

Hallazgo H-08

Campo	Detalle
Tarea evaluada	Gestión de disponibilidad
Ubicación	Web App - Sección "Horarios"
Problema identificado	Interfaz de selección de horas (Time Picker) incómoda y poco fluida
Severidad	2
Heurística vulnerada	Flexibilidad y eficiencia de uso
Descripción	El componente utilizado para ingresar la "Hora de inicio" y "Hora de fin" en las sesiones requiere múltiples clics o una manipulación poco ergonómica, dificultando la entrada rápida de datos, especialmente si se deben configurar múltiples horarios.
Impacto para el usuario	Incrementa innecesariamente el tiempo y el esfuerzo requerido para completar una tarea repetitiva, degradando la experiencia de uso (fricción operativa).
Recomendación	Reemplazar el componente actual por un Time Picker más intuitivo (por ejemplo, selectores visuales de reloj, campos con autocompletado en formato HH:MM, o incrementos rápidos de 15/30 minutos mediante botones).

Evidencia

Curso

Selecciona un curso

Salón

Selecciona un salón

Profesor

Selecciona un profesor

Día de la Semana

Selecciona un día

Hora Inicio

03:43

--:--

Agregar Sesión

22

43

23

44

00

45

01

46

02

47

03

48

04

49

Figura 8. Interfaz de configuración de horarios evidenciando un flujo de selección ineficiente.

Hallazgo H-09

Campo	Detalle
Tarea evaluada	Adaptabilidad visual
Ubicación	Web App - Sección "Cobranzas" (Apartado de facturas)
Problema identificado	Pérdida de claridad y legibilidad en el esquema de Modo Oscuro
Severidad	3
Heurística vulnerada	Diseño estético y minimalista (Accesibilidad)
Descripción	Al activar el tema oscuro (Dark Mode), los estilos CSS específicos del apartado de facturas no se adaptan correctamente. El texto y los elementos de la interfaz carecen del contraste necesario contra los fondos oscuros, volviendo la información difícil o imposible de leer.

Campo	Detalle
Impacto para el usuario	Impide la correcta visualización de datos financieros sensibles. Causa fatiga visual severa y vulnera las pautas de accesibilidad (WCAG) sobre el ratio de contraste mínimo requerido.
Recomendación	Auditar y ajustar la paleta de colores del Modo Oscuro utilizando variables CSS. Asegurar que los colores de texto principales (ej. blanco o gris claro) tengan un ratio de contraste de al menos 4.5:1 sobre los fondos oscuros correspondientes a las tablas y tarjetas de facturación.

Evidencia

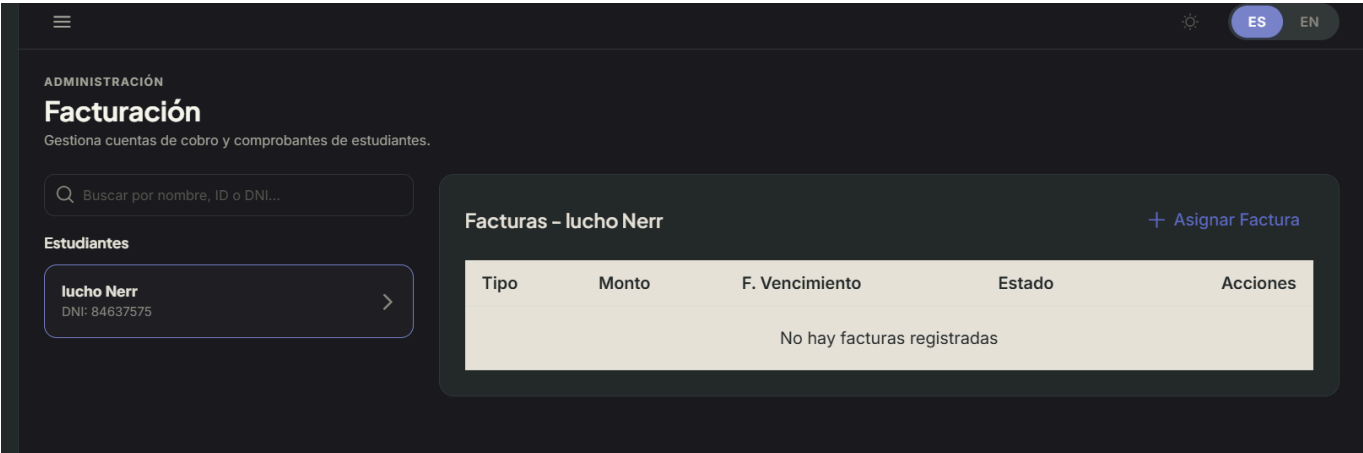


Figura 9. Problemas de contraste en el módulo de facturas al utilizar el tema oscuro.

6.4.2. Auditoria recibida.

6.4.2.1. Información del grupo auditor.

Campo	Detalle
Grupo auditado	Grupo 3
Startup	Stoq
Producto evaluado	StockWise
Integrantes del equipo	Ronald Peralta, Luciana Choquehuanca, Camila Rios, Fabiola Del Rocio y Roy Fernandes
Repositorio del proyecto	https://github.com/orgs/upc-1ASI0732-2610-16879-Stoq/repositories
Landing Page	https://stockwiselanding.netlify.app/
Aplicación web	https://stocktrack-frontend.vercel.app/auth/register

6.4.2.2. Cronograma de auditoría recibida.

Actividad	Fecha	Responsable	Duración estimada
Recepción del informe de auditoría externa de StockWise	11/06/2026	Rafael Dominguez	30 minutos
Revisión, análisis y catalogación de hallazgos	11/06/2026	Rafael Dominguez	1 hora
Priorización de observaciones según escala de severidad	12/06/2026	Diego Vilca	30 minutos
Asignación de tareas al equipo y planificación del plan correctivo	12/06/2026	Diego Vilca	1 hora
Implementación de mejoras y corrección de incidencias críticas	13/06/2026	Equipo EduLabs	6 horas
Validación final y pruebas de regresión de los flujos corregidos	14/06/2026	Equipo EduLabs	1 hora y 30 min

6.4.2.3. Contenido de auditoría recibida.

Hallazgo H-01

Campo	Detalle
Tarea evaluada	Inicio de sesión (Login)
Ubicación	Web App - Pantalla de Login
Problema identificado	Espera prolongada con spinner infinito sin mensaje de contexto ni manejo de tiempo de espera (timeout)
Severidad	2
Heurística vulnerada	Visibilidad del estado del sistema / Control y libertad del usuario
Descripción	Al realizar el primer intento de inicio de sesión, el sistema muestra correctamente un indicador de carga (spinner) en el botón. Sin embargo, debido a una alta latencia del servidor, este spinner se mantiene girando indefinidamente (más de 30 segundos) sin ofrecer ninguna actualización de estado. Al no existir un límite de tiempo de espera (timeout) ni un botón para cancelar la petición, el usuario percibe que la aplicación se ha "congelado", obligándolo a cerrar y recargar la página manualmente para poder continuar.
Impacto para el usuario	Causa frustración y desorientación al no saber si el proceso sigue activo, obligándolo a abandonar la tarea actual y realizar acciones manuales de recuperación (recarga de página) para desbloquear la interfaz.
Recomendación	Establecer un tiempo máximo de espera para la petición HTTP (por ejemplo, 10 o 15 segundos). Si el servidor no responde en ese lapso, se debe detener el spinner y

Campo	Detalle
	mostrar un mensaje de error claro al usuario (ej. "El servidor está tardando más de lo esperado, por favor intenta nuevamente"). Alternativamente, si se sabe que el servidor puede tener un "arranque en frío", mostrar un aviso temporal que diga "Conectando con el servidor, esto puede tomar unos segundos..."

Evidencia

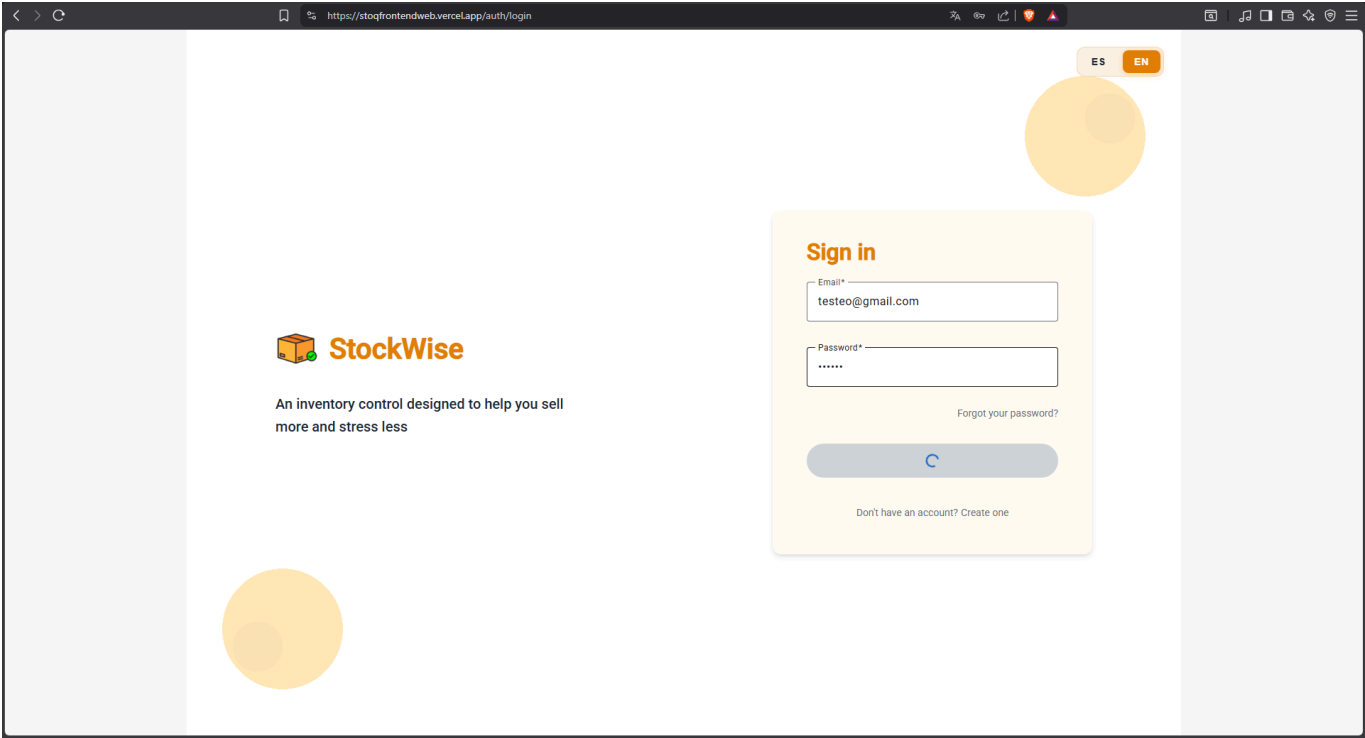


Figura 1. Interfaz de inicio de sesión evidenciando un indicador de carga indefinido y falta de manejo de timeout.

Hallazgo H-02

Campo	Detalle
Tarea evaluada	Recuperación de credenciales
Ubicación	Web App - Pantalla de Login (Enlace "¿Olvidaste tu contraseña?")
Problema identificado	El enlace de "Olvidaste tu contraseña" no dirige al flujo de recuperación, recargando la vista actual
Severidad	3
Heurística vulnerada	Control y libertad del usuario
Descripción	Al hacer clic en la opción "¿Olvidaste tu contraseña?", el sistema no redirige al usuario a la vista correspondiente para restablecer sus credenciales. En su lugar, el enlace simplemente recarga la página de login actual y altera la URL agregando un parámetro (ej. ?returnUrl=%2Fdashboard). Esto atrapa al usuario en un bucle sin salida si realmente ha perdido su contraseña, impidiendo su recuperación.

Campo	Detalle
Impacto para el usuario	Bloquea por completo el acceso a los usuarios que no recuerdan sus credenciales, generando un flujo roto que impide la autonomía del usuario para recuperar su cuenta.
Recomendación	Revisar el enrutamiento (routing) en el frontend de la aplicación web. Se debe asegurar que el enlace tenga la ruta correcta hacia el componente de recuperación (por ejemplo, /auth/recovery) en lugar de apuntar a la misma vista de login, eliminando la recarga innecesaria de la página.

Evidencia

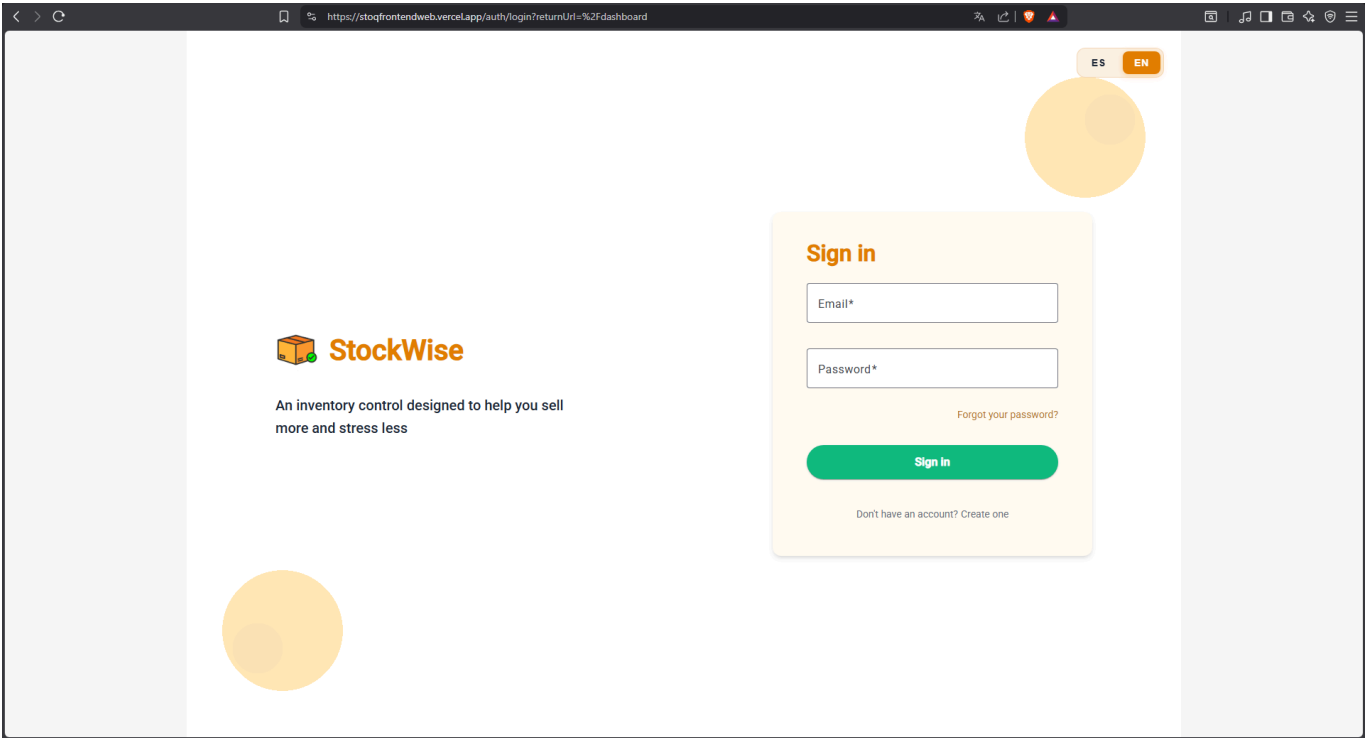


Figura 2. Enlace de recuperación de contraseña que genera un bucle de redirección en la misma pantalla.

Hallazgo H-03

Campo	Detalle
Tarea evaluada	Configuración de idioma / Interacción general
Ubicación	Web App - Esquina superior derecha de la interfaz
Problema identificado	Elemento interactivo superpuesto o muy cerca de un elemento gráfico decorativo
Severidad	1
Heurística vulnerada	Diseño estético y minimalista

Campo	Detalle
Descripción	En la esquina superior derecha, el selector de idioma (ES/EN) se encuentra visualmente sobrepuesto o demasiado pegado al elemento gráfico decorativo (el círculo amarillo de fondo). Esto genera ruido visual, reduce la claridad de la interfaz y da una apariencia poco pulida al diseño, aunque no impide la funcionalidad del botón.
Impacto para el usuario	Genera ruido visual y reduce la legibilidad de los controles, proyectando una imagen de falta de pulido estético y descuido en el diseño visual de la aplicación.
Recomendación	Ajustar los estilos CSS del contenedor del botón. Se debe aumentar el margen (margin) o el espaciado interno (padding) para separarlo del círculo decorativo, o en su defecto, ajustar el posicionamiento absoluto y el z-index de los elementos de fondo para que no interfieran con las áreas de interacción de los componentes superiores.

Evidencia



Figura 3. Superposición visual entre el selector de idioma y el elemento gráfico decorativo de fondo.

Hallazgo H-04

Campo	Detalle
Tarea evaluada	Navegación y persistencia de ruta
Ubicación	Web App - Sección Dashboard
Problema identificado	Redirección inesperada a la vista de "Ajustes" al refrescar el Dashboard
Severidad	3
Heurística vulnerada	Control y libertad del usuario

Campo	Detalle
Descripción	Estando en la vista principal del Dashboard, si el usuario recarga la página (F5 o botón de refresh del navegador), el sistema pierde el contexto de la ruta actual y lo redirige automáticamente a la sección de "Settings" (Ajustes). Esto interrumpe el flujo de trabajo del usuario, causándole desorientación y obligándolo a hacer clics adicionales para volver a la pantalla de inicio.
Impacto para el usuario	Interrumpe drásticamente el flujo de trabajo operativo, obligando al usuario a reorientarse y a realizar acciones repetitivas de navegación para regresar a su ubicación original.
Recomendación	Revisar la configuración del enrutador del frontend (ej. Vue Router, React Router). Se debe asegurar que el estado de la aplicación o el manejo de rutas privadas/autenticadas respete la URL actual (/dashboard) durante la recarga del navegador, en lugar de usar una redirección por defecto hacia /settings.

Evidencia

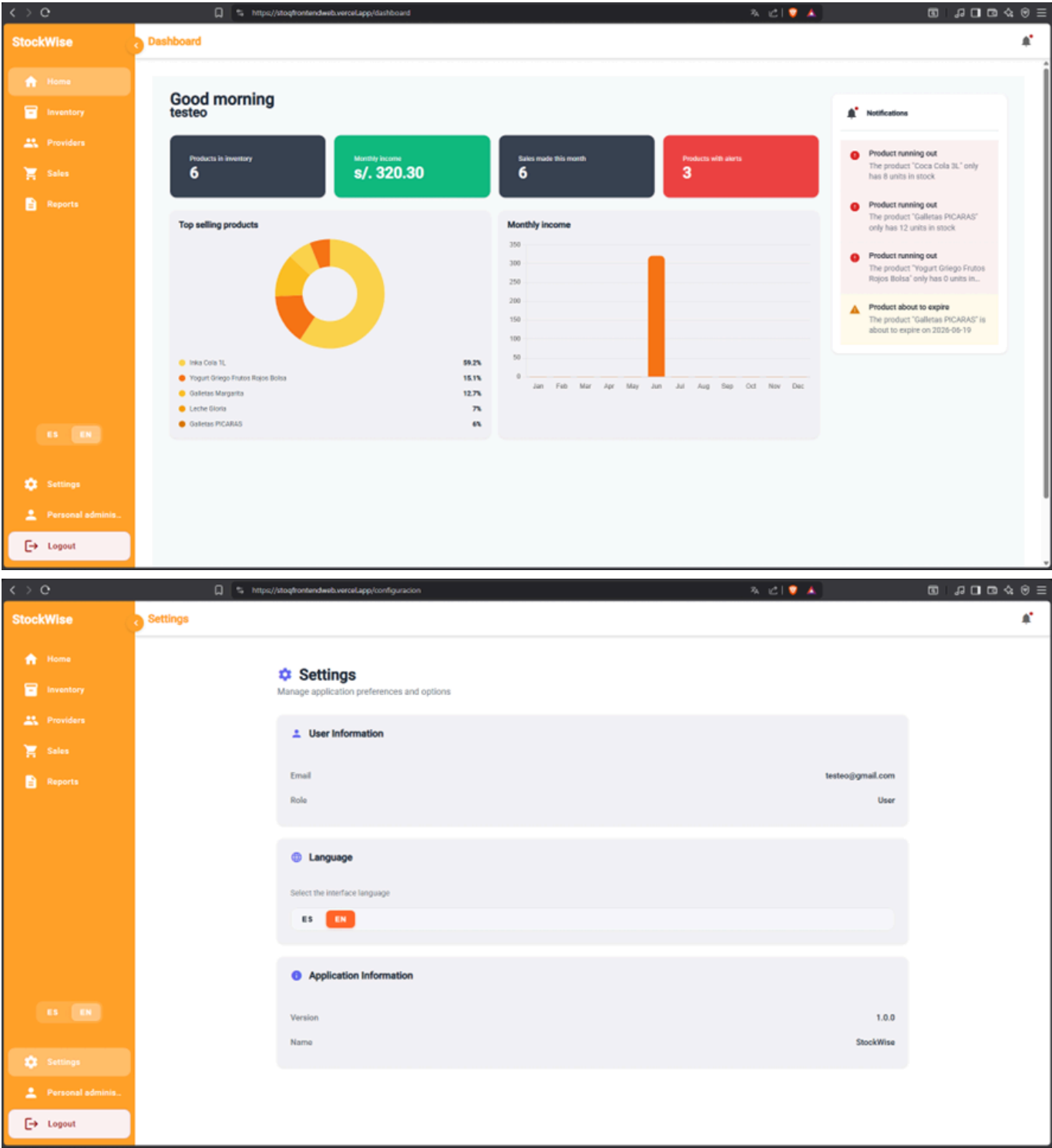


Figura 4. Pérdida del contexto de navegación al actualizar la página desde el Dashboard principal.

Hallazgo H-05

Campo	Detalle
Tarea evaluada	Configuración global (Cambio de idioma)
Ubicación	Web App - Menú lateral izquierdo (Sidebar)
Problema identificado	Control global (cambio de idioma) oculto al colapsar el menú lateral

Campo	Detalle
Severidad	2
Heurística vulnerada	Consistencia y estándares / Flexibilidad y eficiencia de uso
Descripción	El botón para alternar el idioma (ES/EN) está ubicado dentro del menú lateral izquierdo (sidebar). El problema de diseño radica en que, si el usuario decide colapsar este menú para tener más espacio de visualización, el control de idioma desaparece. Los controles de configuración global no deben depender de elementos colapsables de navegación específica.
Impacto para el usuario	Limita el acceso a funciones globales del sistema, forzando al usuario a desplegar elementos de navegación secundarios únicamente para poder interactuar con una preferencia básica de la interfaz.
Recomendación	Reubicar el selector de idiomas. El estándar en el diseño de interfaces web (UI) dicta que este tipo de controles globales se coloquen en la barra superior (Header o Topbar), preferiblemente alineado a la derecha, cerca del icono de notificaciones o del perfil del usuario, garantizando su visibilidad y acceso en todo momento.

Evidencia

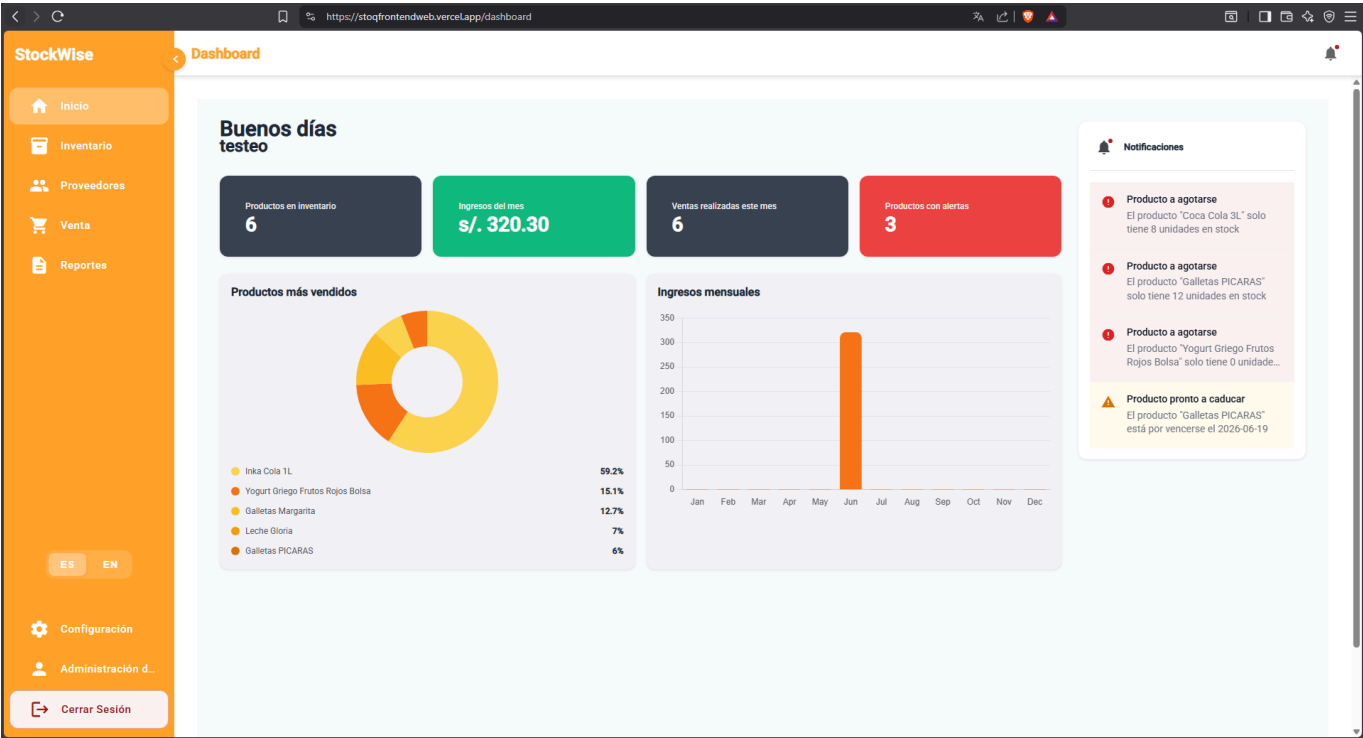


Figura 5. Ocultamiento del selector de idioma global debido al colapso del menú de navegación lateral.

Hallazgo H-06

Campo	Detalle
Tarea evaluada	Visualización de alertas y notificaciones

Campo	Detalle
Ubicación	Web App - Sección Dashboard (Panel derecho y Barra superior)
Problema identificado	Duplicidad innecesaria de la interfaz de notificaciones en la misma vista
Severidad	2
Heurística vulnerada	Diseño estético y minimalista
Descripción	La pantalla del Dashboard presenta redundancia de información. Existe un panel estático en el lado derecho que muestra una lista de notificaciones (ej. "Product running out") y, simultáneamente, en la barra superior hay un icono de campana que despliega exactamente la misma lista en un menú flotante. Esta duplicidad satura la interfaz con información repetida y desperdicia espacio valioso en la pantalla principal.
Impacto para el usuario	Sobrecarga cognitivamente al usuario debido a la redundancia visual, reduciendo además el espacio útil en pantalla que podría destinarse a información analítica relevante.
Recomendación	Eliminar el panel estático de notificaciones del cuerpo del Dashboard. Se recomienda mantener únicamente el icono de la campana en la barra superior con su respectivo menú desplegable (que es el estándar de la industria). El espacio liberado en el lado derecho del Dashboard puede aprovecharse para expandir los gráficos de métricas o incluir un nuevo indicador de negocio.

Evidencia

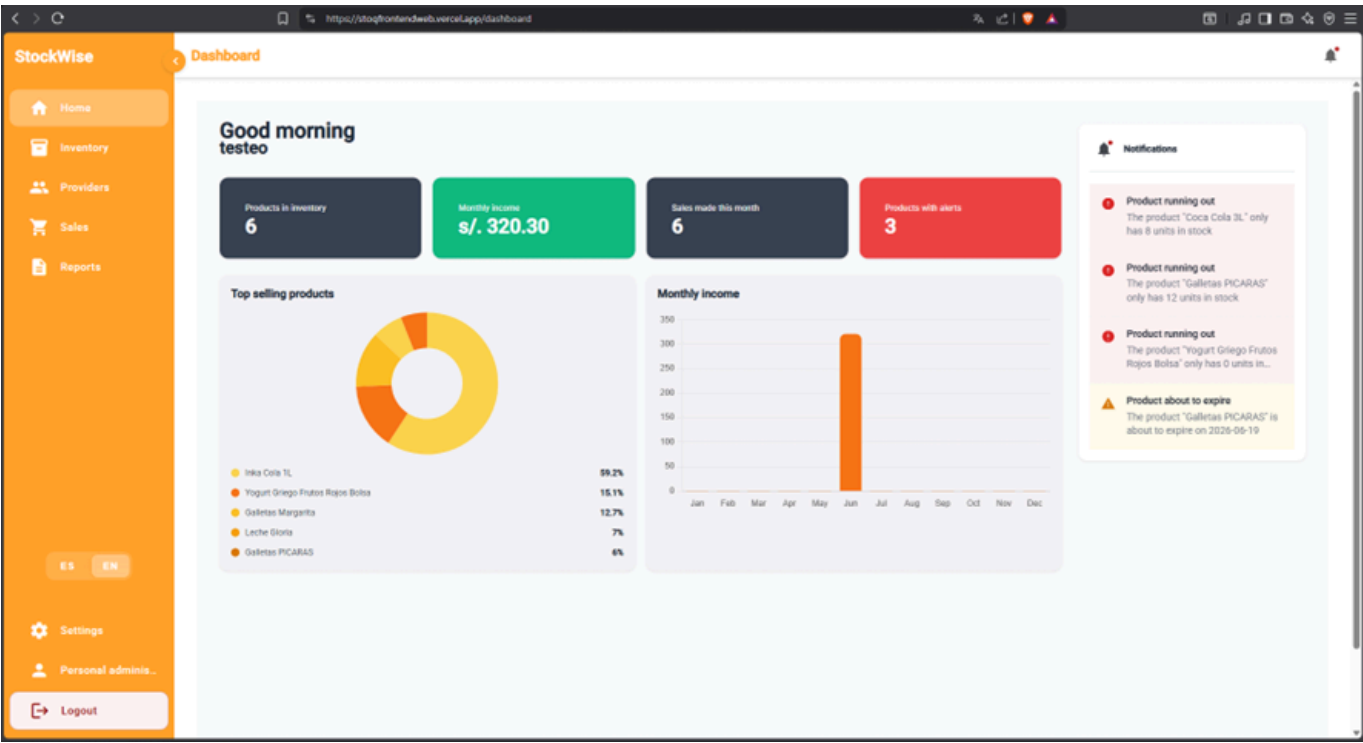


Figura 6. Redundancia de componentes informativos al mostrar notificaciones de forma paralela en dos secciones de la pantalla.

Hallazgo H-07

Campo	Detalle
Tarea evaluada	Navegación por menú lateral
Ubicación	Web App - Menú lateral de navegación (Sidebar)
Problema identificado	Texto truncado e incompleto en una opción del menú lateral
Severidad	1
Heurística vulnerada	Diseño estético y minimalista
Descripción	En la parte inferior del menú de navegación lateral (sidebar), la etiqueta del último botón ("Administración d...") o ("Personal adminis...") es demasiado larga para el ancho predeterminado del contenedor. Al no caber, el texto se corta abruptamente con puntos suspensivos. Aunque esto no impide hacer clic en el botón, da un aspecto visual poco profesional y descuidado a la interfaz.
Impacto para el usuario	Afecta negativamente la estética visual y la legibilidad inmediata de las opciones de menú, requiriendo que el usuario adivine o deduzca el significado completo de la etiqueta.
Recomendación	La mejor práctica en diseño de menús es utilizar etiquetas cortas y directas. Se recomienda cambiar el texto a una alternativa más concisa (por ejemplo, "Administración", "Personal" o "Usuarios"). Si por reglas de negocio es obligatorio mantener el texto original completo, se debe implementar un atributo de accesibilidad tipo tooltip (título emergente) nativo en HTML (title="Personal administration") o (title="Administración de personal") que se muestre cuando el usuario pase el cursor (hover) sobre el botón.

Evidencia



Figura 7. Etiqueta de navegación truncada por restricciones de ancho en el menú lateral.

Hallazgo H-08

Campo	Detalle
Tarea evaluada	Gestión de inventario (Edición de productos)
Ubicación	Web App - Menú contextual de acciones en listado de productos
Problema identificado	La acción "Editar" producto en el menú contextual no responde ni ofrece retroalimentación

Campo	Detalle
Severidad	4
Heurística vulnerada	Visibilidad del estado del sistema / Control y libertad del usuario
Descripción	Al abrir el menú de acciones (icono de tres puntos) en un producto específico y seleccionar la opción "Editar", la interfaz no responde de ninguna manera. No se abre ningún modal, ni se redirige a otra vista, ni se muestra ningún mensaje de error. Esta falta de respuesta bloquea por completo la capacidad del usuario para actualizar la información del inventario.
Impacto para el usuario	Bloqueo crítico de una función principal del sistema, rompiendo la confianza del usuario al no proveer ningún tipo de respuesta o indicativo de fallo tras ejecutar una acción.
Recomendación	Revisar el evento de clic asociado a la opción "Editar" en el código (ej. @click en Vue). Asegurar que el componente modal de edición esté correctamente importado y que la variable reactiva que controla su visibilidad cambie a verdadero (true). Además, verificar en la consola del navegador si existe algún error de JavaScript bloqueando la ejecución del evento.

Evidencia

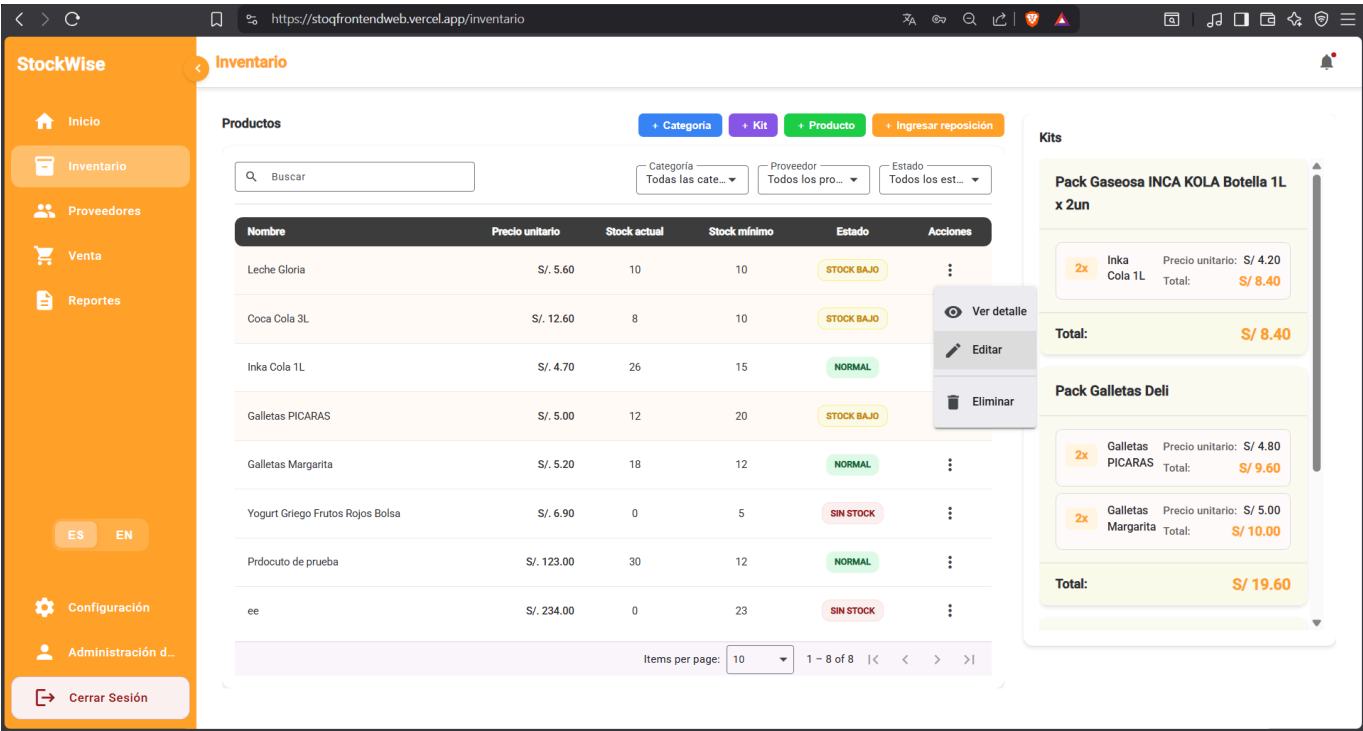


Figura 8. Interfaz de listado de productos donde la opción del menú contextual no gatilla acción alguna.

Hallazgo H-09

Campo	Detalle
Tarea evaluada	Gestión de inventario (Eliminación de productos)
Ubicación	Web App - Menú contextual de acciones en listado de productos
Problema identificado	Fallo en la funcionalidad de eliminación y uso de alertas nativas del navegador
Severidad	4
Heurística vulnerada	Prevención de errores / Diseño estético y minimalista
Descripción	Al intentar eliminar un producto mediante el menú de acciones, ocurren dos problemas graves: primero, el sistema lanza un cuadro de diálogo alert() nativo del navegador para confirmar la acción, lo cual rompe completamente la estética y consistencia del diseño de la aplicación. Segundo, tras confirmar la acción en dicho cuadro, el producto no se elimina del listado, fallando en su propósito principal.
Impacto para el usuario	Pérdida de integridad funcional (error crítico al no eliminar el elemento solicitado) acoplada a una experiencia visual deficiente que devalúa el estándar de diseño de la plataforma.
Recomendación	Reemplazar el uso de alert() o confirm() nativos de JavaScript por un componente modal de confirmación diseñado específicamente para la aplicación (que siga el sistema de diseño actual). En cuanto a la funcionalidad, se debe depurar la petición HTTP de tipo DELETE hacia el servidor para identificar por qué no se está completando la eliminación, y actualizar el estado local del listado (la tabla) para que el producto desaparezca de la vista inmediatamente después de una respuesta exitosa.

Evidencia

The screenshot displays the StockWise web application interface. A modal dialog box is open, asking for confirmation to delete the product 'Leche Gloria'. The dialog has 'Aceptar' (Accept) and 'Cancelar' (Cancel) buttons. In the background, the 'Inventario' (Inventory) page is visible, showing a table of products with columns: Nombre, Precio unitario, Stock actual, Stock mínimo, Estado, and Acciones. The table lists items like 'Leche Gloria', 'Coca Cola 3L', 'Inka Cola 1L', 'Galletas PICARAS', 'Galletas Margarita', 'Yogurt Griego Frutos Rojos Bolsa', 'Producto de prueba', and 'ee'. The 'Estado' column shows 'STOCK BAJO' (Low Stock) for the first three items and 'NORMAL' for the others. The 'Acciones' column has a dropdown menu with options: 'Ver detalle', 'Editar', and 'Eliminar'. On the right side, there are sections for 'Kits' showing product bundles and their totals, such as 'Pack Gaseosa INCA KOLA Botella 1L x 2un' with a total of S/ 8.40 and 'Pack Galletas Deli' with a total of S/ 19.60.

Figura 9. Cuadro de diálogo nativo del navegador interfiriendo con la interfaz gráfica en un flujo de eliminación fallido.

Hallazgo H-10

Campo	Detalle
Tarea evaluada	Creación y registro de "Kits"
Ubicación	Web App - Formulario de nuevo Kit
Problema identificado	Falta de validación y mensajería de error al intentar guardar un "Kit" con campos obligatorios vacíos
Severidad	3
Heurística vulnerada	Ayudar a los usuarios a reconocer, diagnosticar y recuperarse de errores
Descripción	En el formulario de creación de un nuevo "Kit", si el usuario omite un campo obligatorio (como el nombre del kit) e intenta guardar los cambios, el sistema simplemente falla silenciosamente. No se guarda el registro, pero tampoco se resalta el campo faltante ni se muestra ningún mensaje de advertencia. El usuario se queda sin saber qué hizo mal o por qué su acción no tuvo efecto.
Impacto para el usuario	Genera incertidumbre y desprotección operativa al procesar envíos inválidos de manera silenciosa, impidiendo que el usuario sepa cómo corregir la información errónea introducida.
Recomendación	Implementar validación de formularios en el frontend antes de enviar los datos. Se debe deshabilitar el botón de guardar si los campos obligatorios están vacíos o, preferiblemente, si el usuario hace clic en guardar, resaltar los campos faltantes en color rojo y mostrar un mensaje de error explícito debajo del campo (ej. "El nombre del kit es obligatorio"), indicándole claramente cómo corregir el problema.

Evidencia

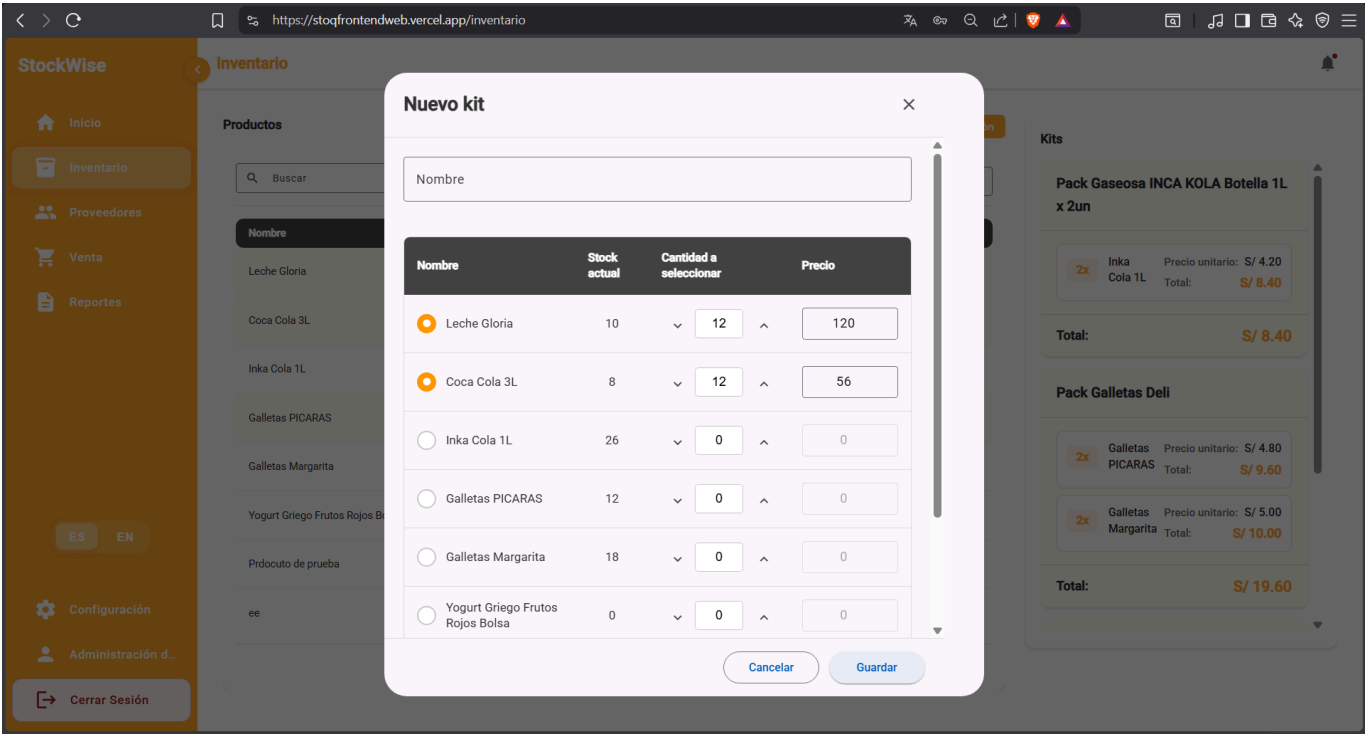


Figura 10. Formulario de creación procesando datos inválidos de forma silenciosa sin indicar campos obligatorios omitidos.

Hallazgo H-11

Campo	Detalle
Tarea evaluada	Visualización de la sección de Proveedores
Ubicación	Web App - Sección Proveedores
Problema identificado	Duplicidad innecesaria del título de la sección en la cabecera
Severidad	1
Heurística vulnerada	Diseño estético y minimalista
Descripción	En la vista actual, el título "Proveedores" aparece repetido dos veces de forma casi consecutiva: una vez en la barra superior (Topbar) junto al icono de notificaciones, y otra vez inmediatamente debajo, en rojo, como encabezado del contenedor principal. Esta redundancia no aporta valor informativo, genera ruido visual y desperdicia espacio vertical en la pantalla.
Impacto para el usuario	Sobrecarga la interfaz con información repetida de manera adyacente y reduce el área útil vertical destinada a la visualización de los datos operativos de los proveedores.
Recomendación	Eliminar el segundo título ("Proveedores" en texto rojo) del área de contenido para mantener un diseño más limpio, dejando únicamente el título de la barra superior como indicador global de la vista. Alternativamente, ese espacio inferior puede usarse

Campo	Detalle
	para un componente de breadcrumbs (migas de pan) si la navegación se vuelve más profunda.

Evidencia

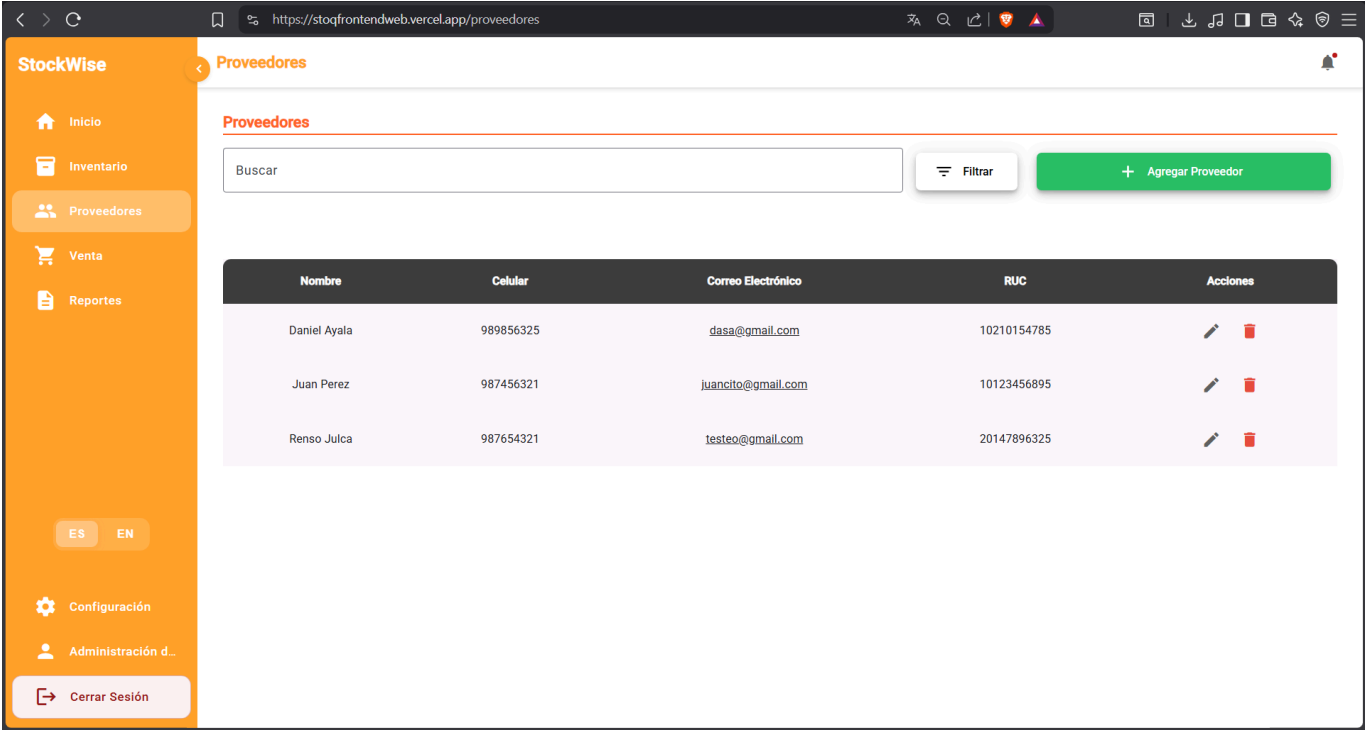


Figura 11. Redundancia visual por duplicidad del título identificador de la sección en la cabecera de la pantalla.

Hallazgo H-12

Campo	Detalle
Tarea evaluada	Búsqueda y filtrado de proveedores
Ubicación	Web App - Barra de búsqueda en la sección Proveedores
Problema identificado	Falta de interactividad y retroalimentación en el botón "Filtrar"
Severidad	3
Heurística vulnerada	Visibilidad del estado del sistema / Flexibilidad y eficiencia de uso
Descripción	El usuario puede ingresar texto en el campo de búsqueda ("Buscar"), pero al hacer clic en el botón contiguo de "Filtrar", el sistema no ejecuta ninguna acción. La tabla no se actualiza, la página no recarga y no se muestra ningún mensaje de error o estado de "cargando". El botón actúa como un elemento estático, lo que rompe la expectativa del usuario y frustra la tarea de búsqueda.

Campo	Detalle
Impacto para el usuario	Bloquea la capacidad de filtrar registros rápidamente, forzando al usuario a realizar una lectura visual manual y reduciendo drásticamente la eficiencia en la localización de datos.
Recomendación	Asegurar que el evento de clic del botón esté correctamente enlazado a la función de filtrado en el controlador del componente. Si la funcionalidad de búsqueda aún está en desarrollo, el botón debe estar visualmente deshabilitado (disabled) o, al hacer clic, debería mostrar una notificación tipo toast indicando que la función estará disponible próximamente.

Evidencia

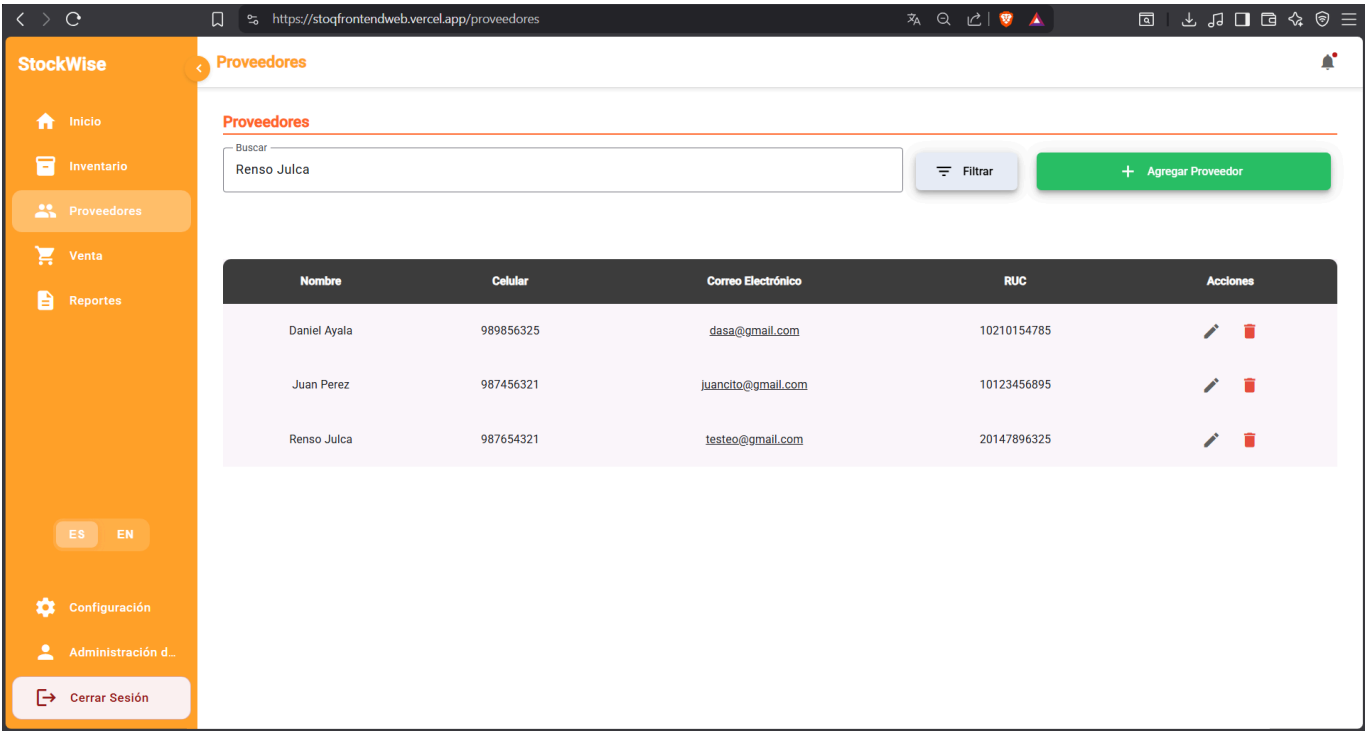


Figura 12. Componente de filtrado inactivo que no procesa el texto ingresado en el campo de búsqueda.

Hallazgo H-13

Campo	Detalle
Tarea evaluada	Gestión de proveedores (Eliminación de registros)
Ubicación	Web App - Listado de la sección Proveedores
Problema identificado	Uso de alertas nativas para confirmar eliminación y ausencia de estado de carga durante el procesamiento
Severidad	3
Heurística vulnerada	Consistencia y estándares / Visibilidad del estado del sistema

Campo	Detalle
Descripción	La acción de eliminar un proveedor presenta dos fallos de experiencia. Primero, invoca un cuadro de diálogo nativo del navegador (confirm()) en lugar de un modal propio, rompiendo la consistencia visual del sistema. Segundo, tras aceptar la alerta, el sistema tarda entre 3 y 5 segundos en remover el registro de la tabla sin mostrar ningún indicador visual de carga (spinner), dejando al usuario con la incertidumbre de si el clic funcionó o si el sistema se colgó.
Impacto para el usuario	Provoca incertidumbre operativa prolongada al no dar señales de que la solicitud está en proceso, lo que puede inducir al usuario a presionar el botón repetidamente o a asumir que la aplicación falló.
Recomendación	Al desarrollar sistemas CRUD integrales para el control de operaciones, el estándar en frameworks modernos exige mantener todo el flujo dentro de la interfaz gráfica propia. Se debe reemplazar la alerta nativa por un componente modal personalizado para la confirmación. Además, es obligatorio inyectar un estado de carga local (por ejemplo, deshabilitar el icono del basurero y cambiarlo por un spinner) durante los segundos que tarde la petición HTTP, actualizando la tabla de inmediato al recibir la respuesta exitosa 200 OK.

Evidencia

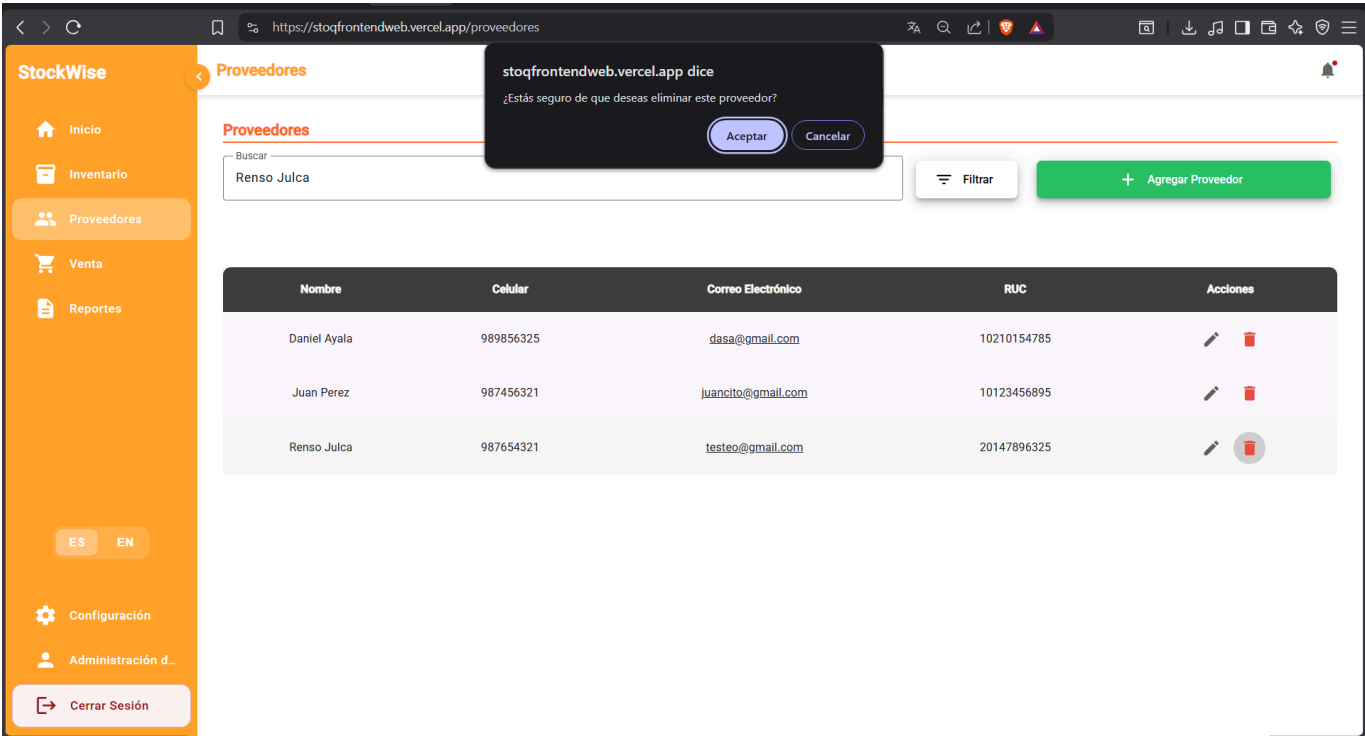


Figura 13. Flujo de eliminación que utiliza cuadros nativos de confirmación y procesa el borrado con retraso y sin retroalimentación.

Hallazgo H-14

Campo	Detalle
Tarea evaluada	Visualización de la sección de Ventas
Ubicación	Web App - Sección Gestión de Ventas
Problema identificado	Duplicidad innecesaria del título de la sección ("Gestión de Ventas")
Severidad	1
Heurística vulnerada	Diseño estético y minimalista
Descripción	Al igual que en la vista de Proveedores, la pantalla de Ventas presenta el título "Gestión de Ventas" repetido dos veces seguidas (en la barra superior y como encabezado del área de trabajo). Esto es redundante y ocupa espacio vertical que podría ser aprovechado para mostrar más productos en la tabla sin necesidad de hacer scroll.
Impacto para el usuario	Degrada la armonía visual de la interfaz mediante repetición de datos y limita artificialmente el espacio disponible para desplegar las filas de la tabla de ventas.
Recomendación	Mantener la consistencia en el diseño de las plantillas (layouts). Se debe remover el título secundario del contenedor principal y conservar únicamente el título de la barra superior.

Evidencia

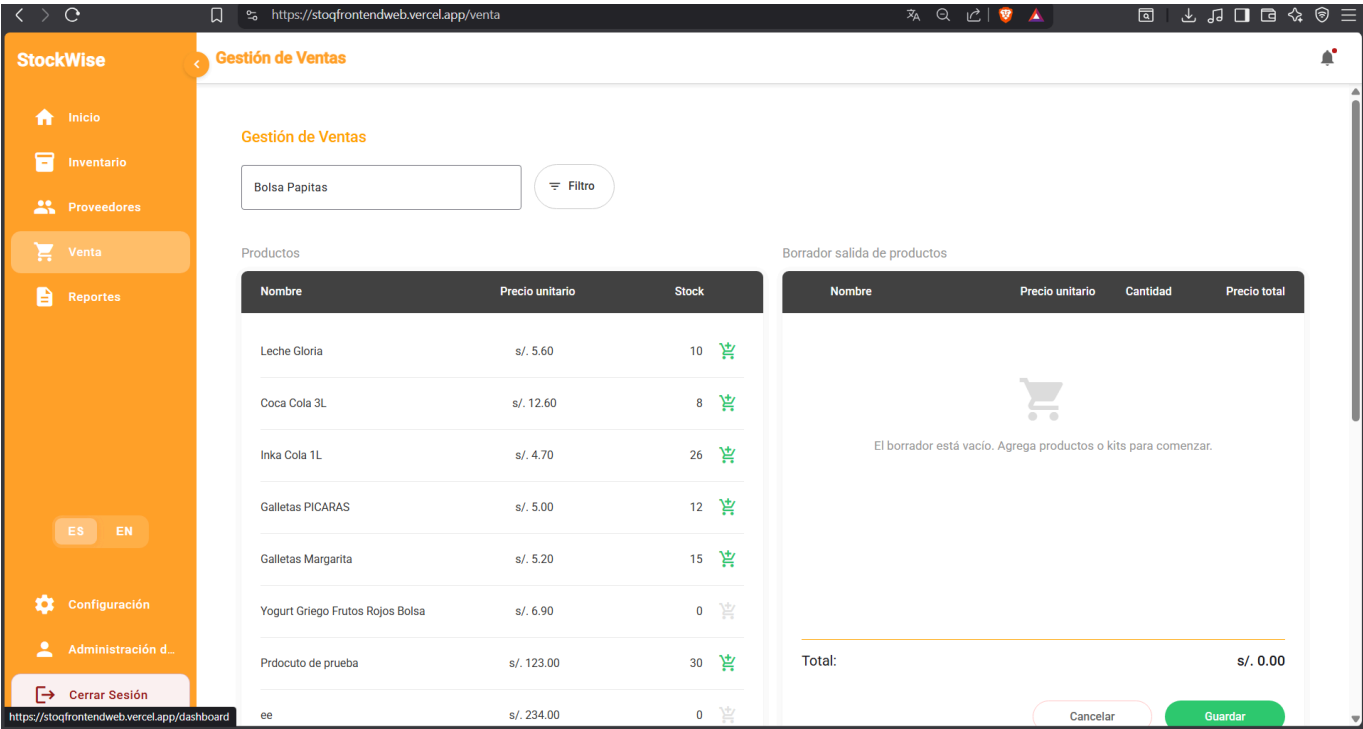


Figura 14. Pantalla de ventas con redundancia estructural en la ubicación y despliegue del título principal.

Campo	Detalle
Tarea evaluada	Búsqueda y filtrado de productos para la venta
Ubicación	Web App - Módulo de ventas (Barra de búsqueda)
Problema identificado	Falta de interactividad en el botón "Filtro" de la barra de búsqueda
Severidad	3
Heurística vulnerada	Visibilidad del estado del sistema / Flexibilidad y eficiencia de uso
Descripción	El usuario puede ingresar el nombre de un producto (ej. "Bolsa Papitas") en el input de texto, pero al presionar el botón "Filtro", la acción es ignorada por el sistema. La tabla de productos no se filtra ni se muestra ningún mensaje, lo que obliga al usuario a buscar el ítem manualmente entre toda la lista, reduciendo drásticamente la eficiencia en el proceso de venta.
Impacto para el usuario	Genera fricción operativa severa y retrasos en la atención al cliente, forzando búsquedas manuales ineficientes dentro de catálogos extensos de productos.
Recomendación	Vincular correctamente el evento @click (o equivalente según el framework utilizado) del botón al método de filtrado. Dado que los productos ya están listados en el DOM, se recomienda implementar un filtrado local (procesando el array de datos en el cliente) para que la búsqueda sea instantánea, en lugar de hacer una nueva petición al servidor.

Evidencia

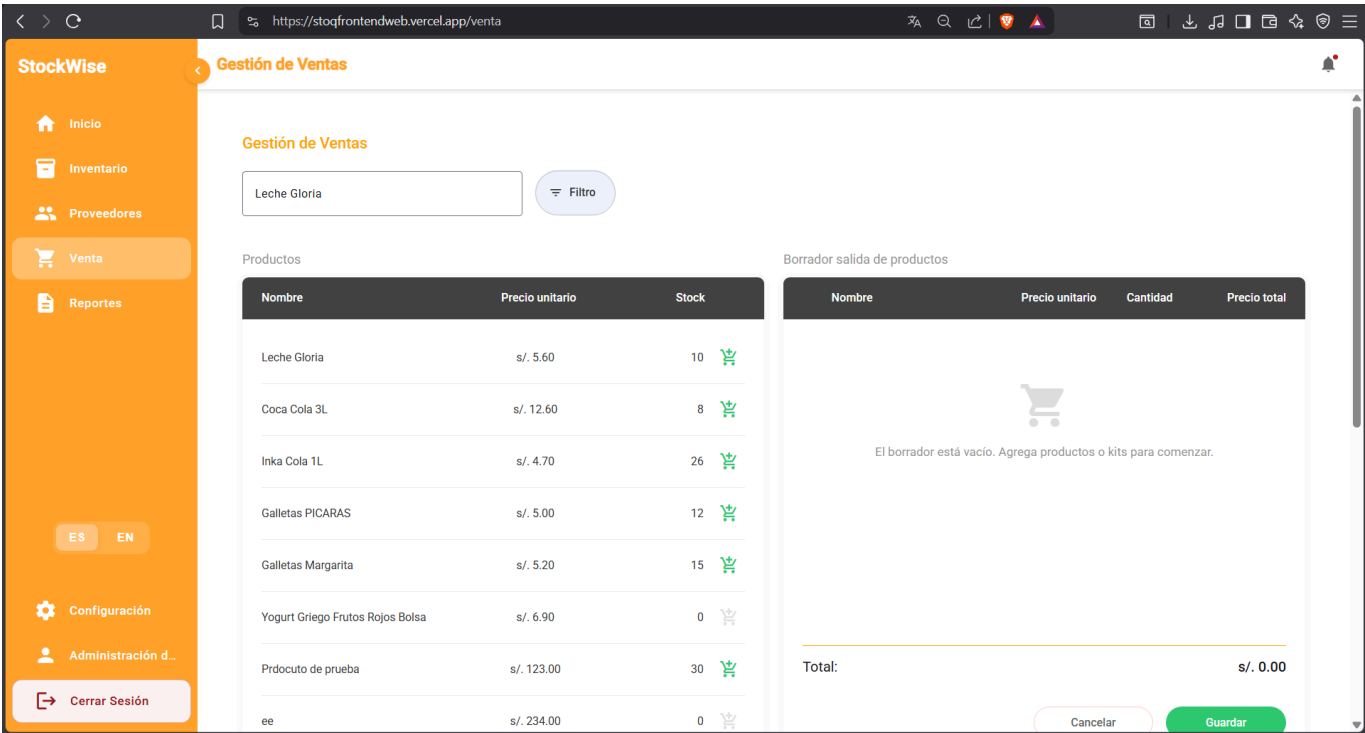


Figura 15. Botón de filtro de productos inerte ante las solicitudes de interacción del usuario.

Hallazgo H-16

Campo	Detalle
Tarea evaluada	Configuración del carrito de compras (Añadir/Modificar ítems)
Ubicación	Web App - Sección Ventas (Panel central de productos y Panel derecho de carrito)
Problema identificado	Latencia excesiva sin retroalimentación visual al agregar productos o modificar cantidades en el carrito
Severidad	3
Heurística vulnerada	Visibilidad del estado del sistema / Flexibilidad y eficiencia de uso
Descripción	Al hacer clic en el ícono verde para agregar un producto al "Borrador salida de productos" (carrito), o al intentar aumentar su cantidad, el sistema sufre una latencia aproximada de 3 segundos antes de reflejar el cambio en el panel derecho. Durante este tiempo, la interfaz no bloquea el botón ni muestra un indicador de carga, lo que genera confusión e invita al usuario a hacer múltiples clics accidentales pensando que la acción no funcionó.
Impacto para el usuario	Induce a la carga errónea o duplicada de cantidades en el carrito de compras debido a la falta de respuesta en tiempo real, entorpeciendo el flujo de checkout.
Recomendación	Implementar un patrón de diseño de Interfaz de Usuario Optimista (Optimistic UI). Cuando se desarrolla un sistema transaccional donde la agilidad operativa es clave, las actualizaciones de estado (como añadir a un carrito) deben reflejarse instantáneamente en la pantalla manipulando el estado local del frontend. La sincronización de estos datos con la base de datos a través de peticiones HTTP debe ocurrir en segundo plano. Adicionalmente, si es estrictamente necesario esperar al servidor, se debe cambiar el icono del carrito por un pequeño spinner de carga durante esos 3 segundos.

Evidencia

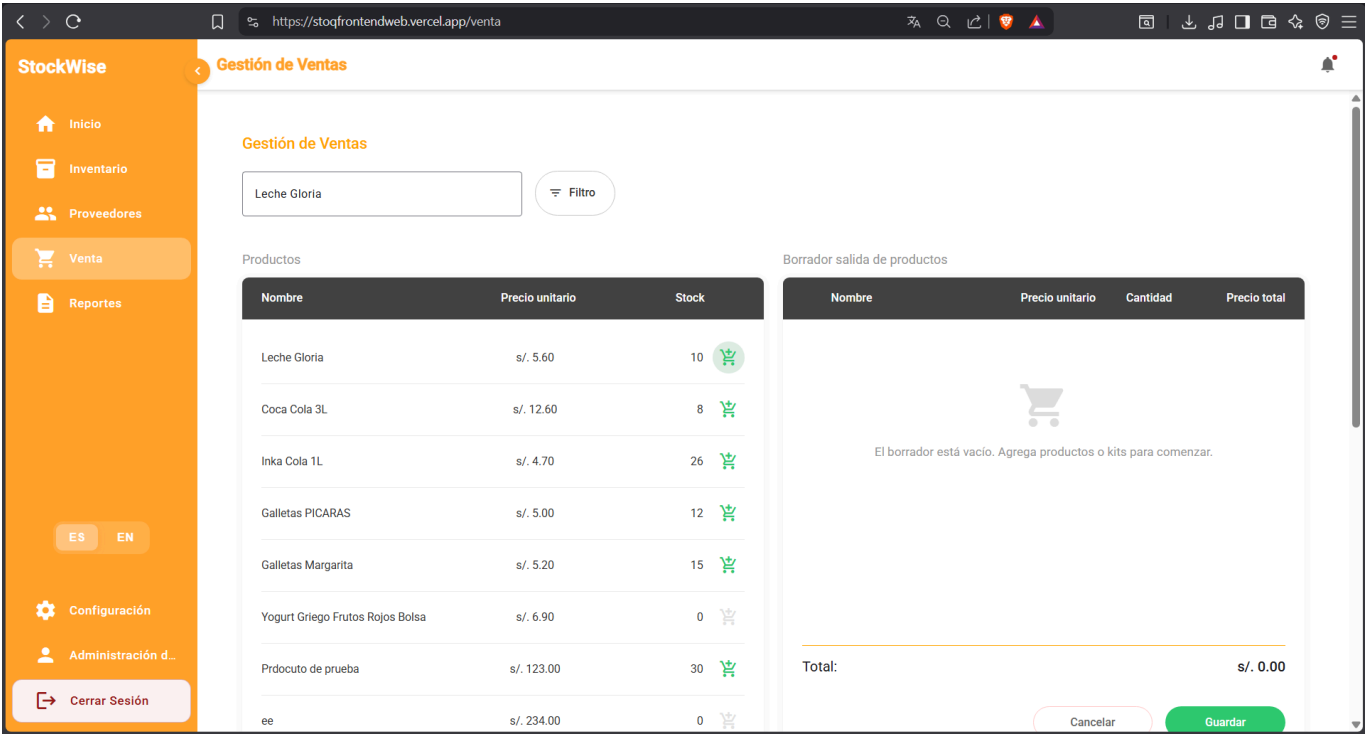


Figura 16. Retraso visual en la actualización del desglose de productos añadidos al carrito de salida.

Hallazgo H-17

Campo	Detalle
Tarea evaluada	Gestión de inventario multiidioma (Creación de categorías/productos)
Ubicación	Web App - Sección Inventario (Modales "New Category" y "New product" en idioma inglés)
Problema identificado	Exposición de variables internas de código (llaves de traducción) al cambiar el idioma a inglés
Severidad	3
Heurística vulnerada	Relación entre el sistema y el mundo real / Prevención de errores
Descripción	Al cambiar el idioma de la aplicación a inglés (botón "EN") y abrir los modales de creación en la sección de Inventario (como "New Category" o "New product"), el sistema falla al renderizar los textos. En lugar de mostrar lenguaje natural, expone las llaves o variables internas del diccionario de internacionalización (por ejemplo: inventory.newCategory, inventory.categoryNamePlaceholder, inventory.isActive). Esto confunde gravemente al usuario final, quien no tiene por qué entender la estructura del código, dificultando la correcta inserción de datos.
Impacto para el usuario	Rompe la comprensión de la interfaz de usuario, impidiendo que operadores angloparlantes entiendan qué datos introducir en los campos del formulario.
Recomendación	Revisar la configuración del paquete de internacionalización (i18n) en el código fuente. Se debe verificar el archivo del diccionario correspondiente al idioma inglés (ej. en.json

Campo	Detalle
	o en.js) e incluir todas las llaves faltantes que se están utilizando en la vista de inventario con sus respectivas traducciones legibles. Adicionalmente, configurar un valor de retroceso (fallback locale) para que, en caso de faltar una traducción en inglés, el sistema muestre el texto en español por defecto en lugar de la variable de código.

Evidencia

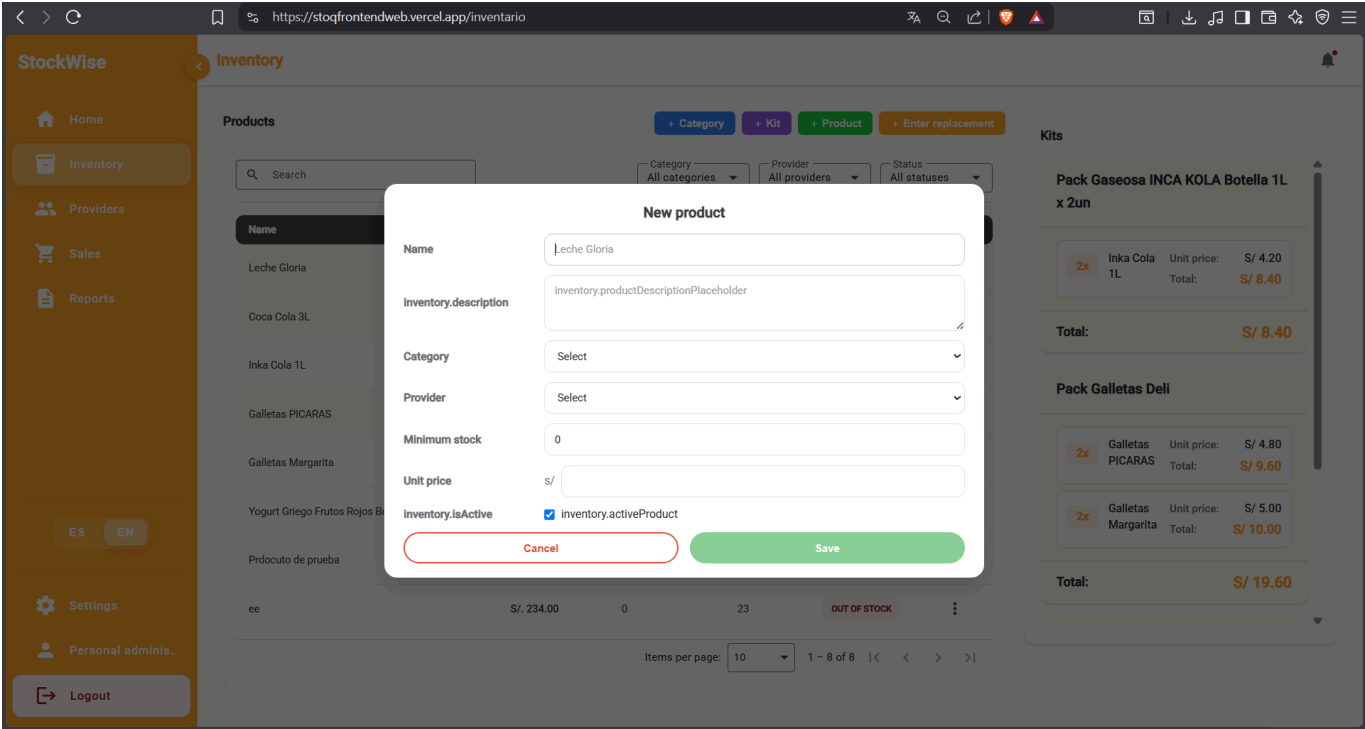


Figura 17. Despliegue de variables lógicas e identificadores del sistema de traducción en lugar de cadenas de texto adaptadas.

Hallazgo H-18

Campo	Detalle
Tarea evaluada	Visualización de la sección de Reportes
Ubicación	Web App - Sección Reports
Problema identificado	Duplicidad innecesaria del título de la sección ("Reports")
Severidad	1
Heurística vulnerada	Diseño estético y minimalista
Descripción	Siguiendo el mismo patrón de error visual encontrado en las vistas de Proveedores y Ventas, la pantalla de Reportes muestra el título duplicado: uno en la barra superior y otro en texto rojo justo debajo, en el área de contenido. Esta repetición es

Campo	Detalle
	redundante, no aporta nueva información al usuario y resta espacio valioso que podría utilizarse para visualizar los datos o los gráficos de los reportes.
Impacto para el usuario	Satura de manera reiterada la visualización del entorno de trabajo, restando espacio para gráficos analíticos complejos o tablas de datos consolidadas.
Recomendación	Estandarizar el diseño (layout) de las vistas principales del sistema. Se debe eliminar el título secundario (el texto rojo dentro del contenedor) en esta y todas las demás pantallas, centralizando la indicación de la vista actual únicamente en la barra de navegación superior.

Evidencia

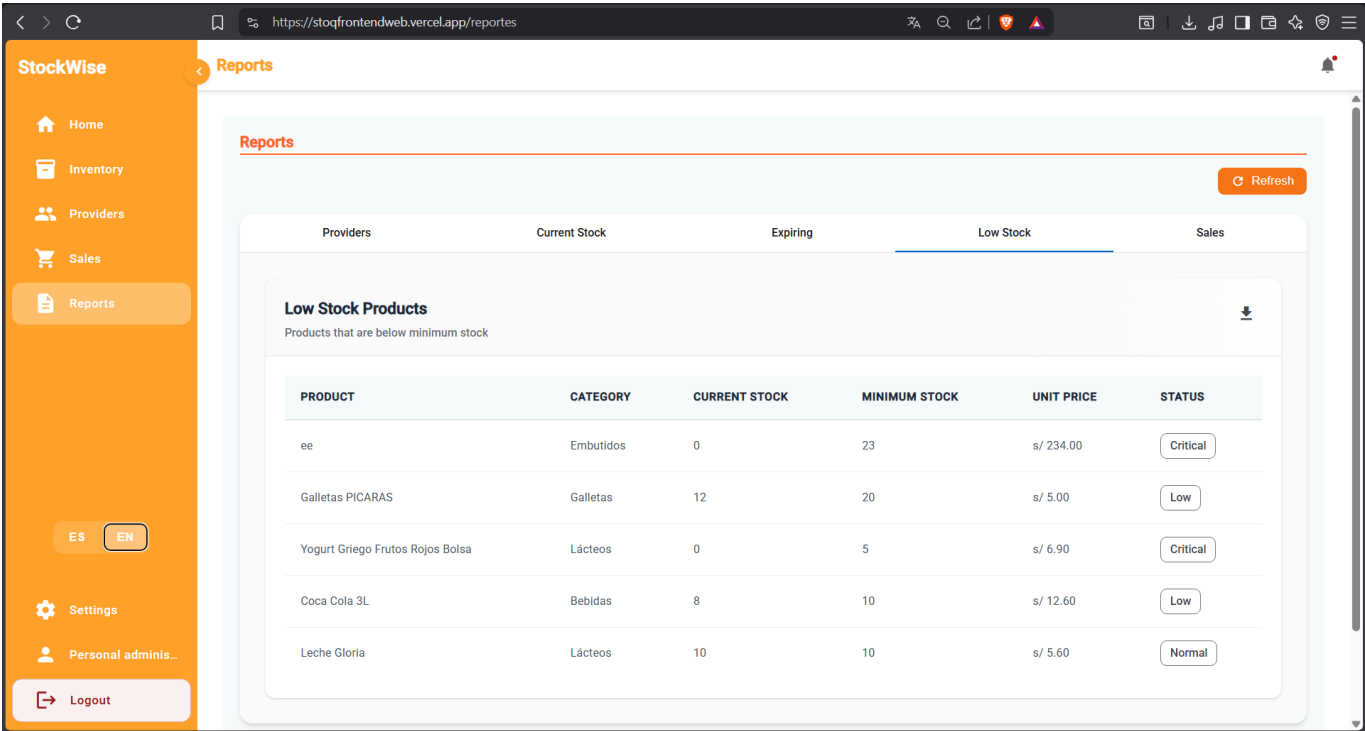


Figura 18. Sección analítica que presenta duplicidad estética de su título descriptivo.

Hallazgo H-19

Campo	Detalle
Tarea evaluada	Registro de personal y asignación de permisos
Ubicación	Web App - Modal "Nuevo Personal"
Problema identificado	Modal de creación de personal atascado en estado de carga indefinido tras una operación exitosa
Severidad	3
Heurística vulnerada	Visibilidad del estado del sistema / Prevención de errores

Campo	Detalle
Descripción	Al completar el formulario para "Nuevo Personal", asignar los permisos correspondientes y hacer clic en "Guardar", el botón cambia a un estado de carga (spinner) y se congela indefinidamente. La ventana modal nunca se cierra ni se muestra un mensaje de confirmación. Sin embargo, si el usuario observa la tabla atenuada en el fondo, puede notar que el nuevo registro sí fue creado y añadido al sistema. Esta falsa retroalimentación de "procesando" hace creer al administrador que el sistema falló, lo que puede inducirlo a cancelar, refrescar la página o intentar crear al mismo usuario duplicado.
Impacto para el usuario	Genera desconfianza e incertidumbre en operaciones críticas de personal, propiciando que el administrador intente registrar al mismo empleado varias veces de manera redundante.
Recomendación	Corregir la resolución de la promesa (Promise) de la petición HTTP en el frontend. La lógica del componente debe actualizar el estado de carga (isLoading = false) e invocar la función para cerrar el modal automáticamente tan pronto como el servidor devuelva un código de éxito (ej. 201 Created o 200 OK). Adicionalmente, se debe limpiar el formulario para futuras inserciones y mostrar una notificación temporal (toast) que confirme explícitamente: "Usuario creado con éxito".

Evidencia

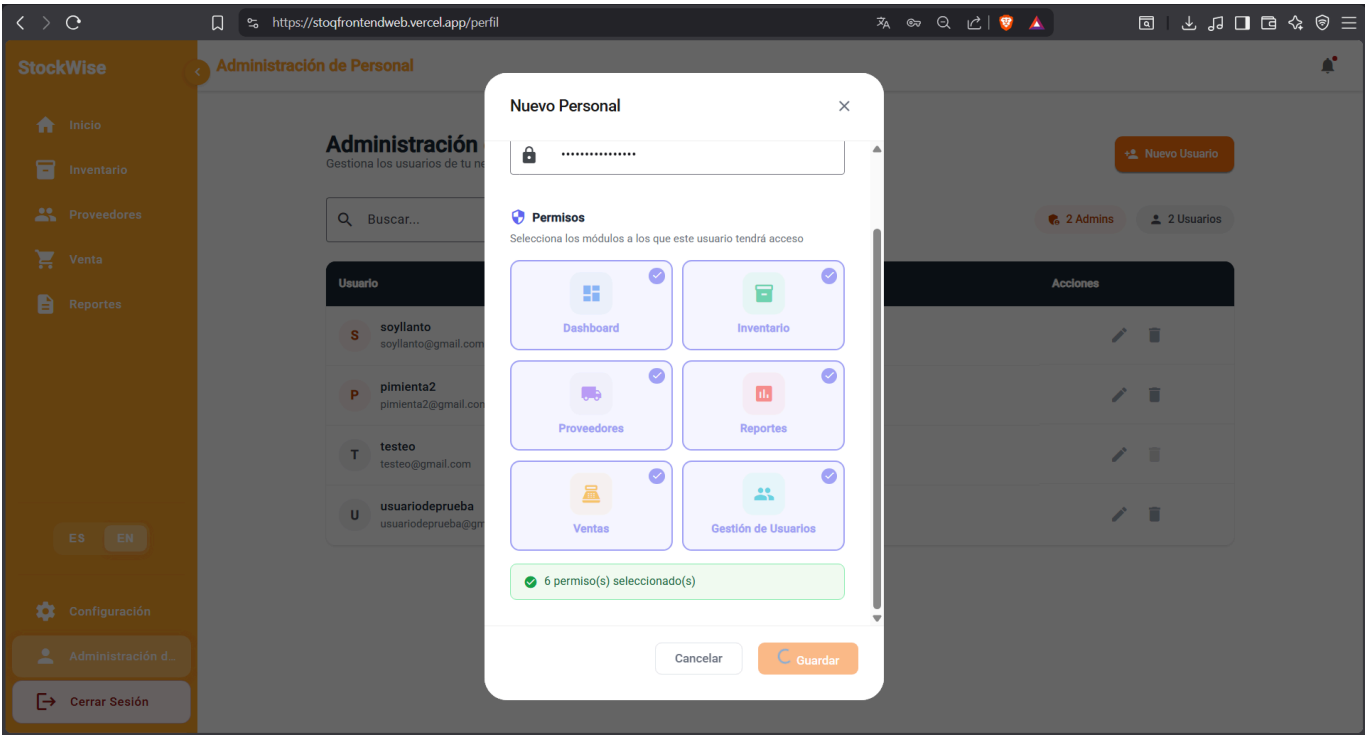


Figura 19. Bloqueo visual indefinido del formulario de alta a pesar del correcto registro de la entidad en segundo plano.

6.4.2.4. Resumen de modificaciones para subsanar hallazgos.

A partir de los hallazgos recibidos en la auditoría externa, el equipo Stoq priorizó los recursos del Sprint actual en resolver las fallas críticas de severidad alta (3 y 4), las cuales impactaban directamente la operabilidad y

transaccionalidad de StockWise.

#	Hallazgo original	Severidad	Acción tomada	Estado
2	El enlace de "Olvidaste tu contraseña" no dirige al flujo de recuperación, recargando la vista actual.	3	Se corrigió el enrutamiento (routing) en el frontend, asignando la ruta <code>/auth/recovery</code> al enlace para dirigir correctamente al usuario al flujo de restablecimiento de credenciales.	Completado
4	Redirección inesperada a la vista de "Ajustes" al refrescar el Dashboard.	3	Se reconfiguraron los guards y las condiciones del enrutador para asegurar que el sistema retenga y respete la URL activa (<code>/dashboard</code>) tras un refresco de página.	Completado
8	La acción "Editar" producto en el menú contextual no responde ni ofrece retroalimentación.	4	Se corrigió el evento de escucha (<code>@click</code>) en el menú contextual del inventario y se reparó la variable reactiva que controla la apertura del modal de edición.	Completado
9	Fallo en la funcionalidad de eliminación y uso de alertas nativas del navegador en Productos.	4	Se reemplazó la alerta nativa por un modal de confirmación integrado en la UI y se depuró la petición HTTP <code>DELETE</code> junto con la actualización inmediata del estado local de la tabla.	Completado
10	Falta de validación y mensajería de error al intentar guardar un "Kit" con campos obligatorios vacíos.	3	Se integró validación de formularios en el frontend que resalta los campos vacíos obligatorios en rojo, bloquea el botón de guardado y despliega mensajes guía específicos.	Completado
12	Falta de interactividad y retroalimentación en el botón "Filtrar".	3	Se vinculó la función de filtrado al evento de clic del botón y se configuró un estado visual deshabilitado (<code>disabled</code>) en caso de que el campo de búsqueda se encuentre vacío.	Completado
13	Uso de alertas nativas para confirmar eliminación y ausencia de estado de carga durante el procesamiento en Proveedores.	3	Se sustituyó la confirmación nativa por un componente modal y se inyectó un spinner de carga local en la tabla para indicar que la petición de borrado está en proceso.	Completado
15	Falta de interactividad en el botón "Filtro" de la barra de búsqueda de Ventas.	3	Se enlazó el botón de filtrado a un método de procesamiento local sobre el arreglo de datos del cliente para	Completado

#	Hallazgo original	Severidad	Acción tomada	Estado
			ofrecer búsquedas e iteraciones instantáneas.	
16	Latencia excesiva sin retroalimentación visual al agregar productos o modificar cantidades en el carrito.	3	Se aplicó un patrón de Interfaz de Usuario Optimista (Optimistic UI) para reflejar los cambios en el carrito de inmediato en el cliente mientras la petición se procesa en segundo plano.	Completado
17	Exposición de variables internas de código (llaves de traducción) al cambiar el idioma a inglés.	3	Se completaron los diccionarios de internacionalización (<code>en.json</code>) en las vistas de inventario y se configuró un <i>fallback locale</i> hacia el español para prevenir la visualización de código.	Completado
19	Modal de creación de personal atascado en estado de carga indefinido tras una operación exitosa.	3	Se corrigió la resolución de la promesa en el envío del formulario para forzar el cierre automático del modal, limpiar los inputs y disparar una notificación toast de éxito.	Completado

Los cambios de mayor prioridad (severidad 3 o 4) fueron atendidos en su totalidad y desplegados antes del cierre del Sprint actual. Por otro lado, los hallazgos de severidad menor (1 y 2), asociados principalmente a duplicidades visuales y ajustes menores de diseño estético, fueron trasladados al Product Backlog para ser planificados en futuras iteraciones de mantenimiento UI.

Capítulo VII: DevOps Practices

7.1. Continuous Integration

7.1.1. Tools and Practices

7.1.2. Build & Test Suite Pipeline Components

7.2. Continuous Delivery

7.2.1. Tools and Practices

7.2.2. Stages Deployment Pipeline Components

7.3. Continuous deployment

7.3.1. Tools and Practices

7.3.2. Production Deployment Pipeline Components

7.4. Continuous Monitoring

7.4.1. Tools and Practices

7.4.2. Monitoring Pipeline Components

7.4.3. Alerting Pipeline Components

7.4.4. Notification Pipeline Components

Part III: Experiment-Driven Lifecycle

Capitulo VIII: Experiment-Driven Development

8.1. Experiment Planning

8.1.1. As-Is Summary

8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims

8.1.3. Experiment-Ready Questions

8.1.4. Question Backlog

8.1.5. Experiment Cards

8.2. Experiment Design

8.2.1. Hypotheses

8.2.2. Domain Business Metrics

8.2.3. Measures

8.2.4. Conditions

8.2.5. Scale Calculations and Decisions

8.2.6. Methods Selection

8.2.7. Data Analytics: Goals, KPIs and Metrics Selection

8.2.8. Web and Mobile Tracking Plan

8.3. Experimentation

8.3.1. To-Be User Stories

8.3.2. To-Be Product Backlog

8.3.3. Pipeline-supported, Experiment-Driven To-Be Software Platform Lifecycle

8.3.3.1. To-Be Sprint Backlogs

8.3.3.2. Implemented To-Be Landing Page Evidence

5.2.3. Implemented Frontend-Web Application Evidence

8.3.3.4. Implemented To-Be Native-Mobile Application Evidence

8.3.3.5. Implemented To-Be RESTful API and/or Serverless Backend Evidence

8.3.3.6. Team Collaboration Insights

8.3.4. To-Be Validation Interviews

8.3.4.1. Diseno de Entrevistas

8.3.4.2. Registro de Entrevistas

8.4. Experiment Aftermath & Analysis

8.4.1. Analysis and Interpretation of Results

8.4.2. Re-scored and Re-prioritized Question Backlog

8.5. Continuous Learning

8.5.1. Shareback Session Artifacts: Learning Workflow

8.6. To-Be Software Platform Pre-launch

8.6.1. About-the-Product Intro Video

Conclusiones

Conclusiones y recomendaciones

Bibliografia

Anexos
